Dokumentasi Resmi & Sintaks Python (3.12/3.13 Up-to-Date)

Bahasa: Python | Total Chunk/Dokumen: 258 | Disiapkan untuk RAG System

[python_001] Python - Getting Started & Basics

An Informal Introduction to Python: Overview

Sumber: ht

An Informal Introduction to Python
In the following examples, input and output are distinguished by the presence or absence of prompts (:term:`>>>` and ...): to repeat the example, you must type everything after the prompt, when the prompt appears; lines that do not begin with a prompt are output from the interpreter. Note that a secondary prompt on a line by itself in an example means you must type a blank line; this is used to end a multi-line command.
You can use the "Copy" button (it appears in the upper-right corner when hovering over or tapping a code example), which strips prompts and omits output, to copy and paste the input lines into your interpreter.
Many of the examples in this manual, even those entered at the interactive prompt, include comments. Comments in Python start with the hash character, ``#``, and extend to the end of the physical line. A comment may appear at the start of a line or following whitespace or code, but not within a string literal. A hash character within a string literal is just a hash character.
Since comments are to clarify code and are not interpreted by Python, they may be omitted when typing in examples.
Some examples

Contoh Kode / Perintah CLI:

```
# this is the first comment
spam = 1  # and this is the second comment
# ... and now a third!
text = "# This is not a comment because it's inside quotes."
```

[python_002] Python - Getting Started & Basics

An Informal Introduction to Python: Using Python as a Calculator

Sumber: ht

Let's try some simple Python commands. Start the interpreter and wait for the primary prompt, ``>>>``. (It shouldn't take long.)

[python_003] Python - Getting Started & Basics

An Informal Introduction to Python: Numbers

Sumber: ht

The interpreter acts as a simple calculator: you can type an expression into it and it will write the value. Expression syntax is straightforward: the operators ``+``, ``-``, ``*`` and ``/`` can be used to perform arithmetic; parentheses (``()``) can be used for grouping.
For example
The integer numbers (e.g. ``2``, ``4``, ``20``) have type int, the ones with a fractional part (e.g. ``5.0``, ``1.6``) have type float. We will see more about numeric types later in the tutorial.
Division (``/``) always returns a float. To do floor division and

get an integer result you can use the ``//`` operator; to calculate
the remainder you can use ``%``
With Python, it is possible to use the ``**`` operator to calculate powers [#]_
The equal sign (``=``) is used to assign a value to a variable. Afterwards, no
result is displayed before the next interactive prompt
If a variable is not "defined" (assigned a value), trying to use it will
give you an error
There is full support for floating point; operators with mixed type operands
convert the integer operand to floating point
In interactive mode, the last printed expression is assigned to the variable
``_``. This means that when you are using Python as a desk calculator, it is
somewhat easier to continue calculations, for example
This variable should be treated as read-only by the user. Don't explicitly
assign a value to it --- you would create an independent local variable with the
same name masking the built-in variable with its magic behavior.
In addition to int and float, Python supports other types of
numbers, such as ~decimal.Decimal and ~fractions.Fraction.
Python also has built-in support for typesnumeric,
and uses the ``j`` or ``J`` suffix to indicate the imaginary part
(e.g. ``3+5j``).

Contoh Kode / Perintah CLI:

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5  # division always returns a floating-point number
1.6

# ----------------------------------------

>>> 17 / 3  # classic division returns a float
5.666666666666667
>>>
>>> 17 // 3  # floor division discards the fractional part
5
>>> 17 % 3  # the % operator returns the remainder of the division
2
>>> 5 * 3 + 2  # floored quotient * divisor + remainder
17

# ----------------------------------------

>>> 5 ** 2  # 5 squared
25
>>> 2 ** 7  # 2 to the power of 7
128
```

[python_004] Python - Getting Started & Basics

An Informal Introduction to Python: Text

Sumber: ht

Python can manipulate text (represented by type str, so-called
"strings") as well as numbers. This includes characters "``!``", words
"``rabbit``", names "``Paris``", sentences "``Got your back.``", etc.
"``Yay! :)``". They can be enclosed in single quotes (``'...'``) or double
quotes (``"..."``) with the same result [#]_.
>>> 'spam eggs' # single quotes
'spam eggs'
>>> "Paris rabbit got your back :)! Yay!" # double quotes
'Paris rabbit got your back :)! Yay!'
>>> '1975' # digits and numerals enclosed in quotes are also strings
'1975'
To quote a quote, we need to "escape" it, by preceding it with ``\``.
Alternatively, we can use the other type of quotation marks
In the Python shell, the string definition and output string can look
different. The print function produces a more readable output, by
omitting the enclosing quotes and by printing escaped and special characters
If you don't want characters prefaced by ``\`` to be interpreted as
special characters, you can use *raw strings* by adding an ``r`` before
the first quote
There is one subtle aspect to raw strings: a raw string may not end in
an odd number of ``\`` characters; see
faq-programming-raw-string-backslash for more information
and workarounds.
String literals can span multiple lines. One way is using triple-quotes:
``"""..."""`` or ``'''...'''``. End-of-line characters are automatically
included in the string, but it's possible to prevent this by adding a ``\`` at
the end of the line. In the following example, the initial newline is not
included
Strings can be concatenated (glued together) with the ``+`` operator, and
repeated with ``*``
Two or more *string literals* (i.e. the ones enclosed between quotes) next
to each other are automatically concatenated.
This feature is particularly useful when you

Contoh Kode / Perintah CLI:

```
>>> 'doesn\'t'  # use \' to escape the single quote...
"doesn't"
>>> "doesn't"  # ...or use double quotes instead
"doesn't"
>>> '"Yes," they said.'
'"Yes," they said.'
>>> "\"Yes,\" they said."
'"Yes," they said.'
>>> '"Isn\'t," they said.'
'"Isn\'t," they said.'

# ----------------------------------------

>>> s = 'First line.\nSecond line.'  # \n means newline
>>> s  # without print(), special characters are included in the string
'First line.\nSecond line.'
>>> print(s)  # with print(), special characters are interpreted, so \n produces new line
```

```
First line.
Second line.

# -----------------------------------------

>>> print('C:\this\name')  # here \t means tab, \n means newline
C:      his
ame
>>> print(r'C:\this\name')  # note the r before the quote
C:\this\name
```

[python_005] Python - Getting Started & Basics

An Informal Introduction to Python: Lists

Sumber: ht

Python knows a number of *compound* data types, used to group together other values. The most versatile is the *list*, which can be written as a list of comma-separated values (items) between square brackets. Lists might contain items of different types, but usually the items all have the same type.
Like strings (and all other built-in sequence types), lists can be indexed and sliced
Lists also support operations like concatenation
Unlike strings, which are immutable, lists are a mutable type, i.e. it is possible to change their content
You can also add new items at the end of the list, by using the list.append *method* (we will see more about methods later)
Simple assignment in Python never copies data. When you assign a list to a variable, the variable refers to the *existing list*.
Any changes you make to the list through one variable will be seen through all other variables that refer to it.
All slice operations return a new list containing the requested elements. This means that the following slice returns a shallow_vs_deep_copy of the list
Assignment to slices is also possible, and this can even change the size of the list or clear it entirely
The built-in function len also applies to lists
It is possible to nest lists (create lists containing other lists), for example

Contoh Kode / Perintah CLI:

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]

# -----------------------------------------

>>> squares[0]  # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:]  # slicing returns a new list
[9, 16, 25]

# -----------------------------------------

>>> squares + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

[python_006] Python - Getting Started & Basics

An Informal Introduction to Python: First Steps Towards Programming

Sumber: ht

Of course, we can use Python for more complicated tasks than adding two and two together. For instance, we can write an initial sub-sequence of the `Fibonacci series <https://en.wikipedia.org/wiki/Fibonacci_sequence>`_ as follows
This example introduces several new features.
* The first line contains a *multiple assignment*: the variables ``a`` and ``b`` simultaneously get the new values 0 and 1. On the last line this is used again, demonstrating that the expressions on the right-hand side are all evaluated first before any of the assignments take place. The right-hand side expressions are evaluated from the left to the right.
* The while loop executes as long as the condition (here: ``a < 10``) remains true. In Python, like in C, any non-zero integer value is true; zero is false. The condition may also be a string or list value, in fact any sequence; anything with a non-zero length is true, empty sequences are false. The test used in the example is a simple comparison. The standard comparison operators are written the same as in C: ``<`` (less than), ``>`` (greater than), ``==`` (equal to), ``<=`` (less than or equal to), ``>=`` (greater than or equal to) and ``!=`` (not equal to).
* The *body* of the loop is *indented*: indentation is Python's way of grouping statements. At the interactive prompt, the default REPL automatically indents continuation lines after compound statement headers like ``if`` or ``while``. In the basic REPL (invoked with PYTHON_BASIC_REPL) or in older Python versions, you have to type a tab or space(s) for each indented line manually. In practice you will prepare more complicated input for Python with a text editor; all decent text editors have an auto-indent facility. When a compound statement is entered interactively, it must be fo

Contoh Kode / Perintah CLI:

```
>>> # Fibonacci series:
>>> # the sum of two elements defines the next
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a+b
...
0
1
1
2
3
5
8

# ----------------------------------------

>>> i = 256*256
>>> print('The value of i is', i)
The value of i is 65536
The keyword argument *end* can be used to avoid the newline after the output,

# ----------------------------------------
```

```
>>> a, b = 0, 1
>>> while a < 1000:
...     print(a, end=',')
...     a, b = b, a+b
...
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

[python_007] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Overview

Sumber: ht

More Control Flow Tools
As well as the while statement just introduced, Python uses a few more
that we will encounter in this chapter.

[python_008] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): if Statements

Sumber: ht

Perhaps the most well-known statement type is the if statement. For
example
There can be zero or more elif parts, and the else part is
optional. The keyword 'elif' is short for 'else if', and is useful
to avoid excessive indentation. An if ... elif ...
elif ... sequence is a substitute for the ``switch`` or
``case`` statements found in other languages.
If you're comparing the same value to several constants, or checking for specific types or
attributes, you may also find the match statement useful. For more
details see tut-match.

Contoh Kode / Perintah CLI:

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
More
```

[python_009] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): for Statements

Sumber: ht

pair: statement; for
The for statement in Python differs a bit from what you may be used
to in C or Pascal. Rather than always iterating over an arithmetic progression
of numbers (like in Pascal), or giving the user the ability to define both the
iteration step and halting condition (as C), Python's for statement
iterates over the items of any sequence (a list or a string), in the order that
they appear in the sequence. For example (no pun intended):

.. One suggestion was to give a real C example here, but that may only serve to
confuse non-C programmers.
Code that modifies a collection while iterating over that same collection can
be tricky to get right. Instead, it is usually more straight-forward to loop
over a copy of the collection or to create a new collection

Contoh Kode / Perintah CLI:

```
>>> # Measure some strings:
>>> words = ['cat', 'window', 'defenestrate']
>>> for w in words:
...     print(w, len(w))
...
cat 3
window 6
defenestrate 12

# ----------------------------------------

# Create a sample collection
users = {'Hans': 'active', 'Éléonore': 'inactive', '???': 'active'}
# Strategy:  Iterate over a copy
for user, status in users.copy().items():
if status == 'inactive':
del users[user]
# Strategy:  Create a new collection
active_users = {}
for user, status in users.items():
if status == 'active':
active_users[user] = status
```

[python_010] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): The range Function

Sumber: ht

If you do need to iterate over a sequence of numbers, the built-in function
range comes in handy. It generates arithmetic progressions
The given end point is never part of the generated sequence; ``range(10)`` generates
10 values, the legal indices for items of a sequence of length 10. It
is possible to let the range start at another number, or to specify a different
increment (even negative; sometimes this is called the 'step')
To iterate over the indices of a sequence, you can combine range and
len as follows
In most such cases, however, it is convenient to use the enumerate
function, see tut-loopidioms.
A strange thing happens if you just print a range
In many ways the object returned by range behaves as if it is a list,
but in fact it isn't. It is an object which returns the successive items of
the desired sequence when you iterate over it, but it doesn't really make
the list, thus saving space.
We say such an object is iterable, that is, suitable as a target for
functions and constructs that expect something from which they can
obtain successive items until the supply is exhausted. We have seen that
the for statement is such a construct, while an example of a function
that takes an iterable is sum
Later we will see more functions that return iterables and take iterables as
arguments. In chapter tut-structures, we will discuss list in more

detail.

Contoh Kode / Perintah CLI:

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4

# ----------------------------------------

>>> list(range(5, 10))
[5, 6, 7, 8, 9]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(-10, -100, -30))
[-10, -40, -70]

# ----------------------------------------

>>> a = ['Mary', 'had', 'a', 'little', 'lamb']
>>> for i in range(len(a)):
...     print(i, a[i])
...
0 Mary
1 had
2 a
3 little
4 lamb
```

[python_011] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): break and continue Statements

Sumber: ht

The break statement breaks out of the innermost enclosing
for or while loop
The continue statement continues with the next
iteration of the loop

Contoh Kode / Perintah CLI:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(f"{n} equals {x} * {n//x}")
...             break
...
4 equals 2 * 2
6 equals 2 * 3
8 equals 2 * 4
9 equals 3 * 3

# ----------------------------------------

>>> for num in range(2, 10):
...     if num % 2 == 0:
...         print(f"Found an even number {num}")
...         continue
...     print(f"Found an odd number {num}")
```

```
...
Found an even number 2
Found an odd number 3
Found an even number 4
Found an odd number 5
Found an even number 6
Found an odd number 7
Found an even number 8
Found an odd number 9
```

[python_012] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): else Clauses on Loops

Sumber: ht

In a for or while loop the break statement
may be paired with an else clause. If the loop finishes without
executing the break, the else clause executes.
In a for loop, the else clause is executed
after the loop finishes its final iteration, that is, if no break occurred.
In a while loop, it's executed after the loop's condition becomes false.
In either kind of loop, the else clause is **not** executed if the
loop was terminated by a break. Of course, other ways of ending the
loop early, such as a return or a raised exception, will also skip
execution of the else clause.
This is exemplified in the following for loop,
which searches for prime numbers
(Yes, this is the correct code. Look closely: the ``else`` clause belongs to
the ``for`` loop, **not** the ``if`` statement.)
One way to think of the else clause is to imagine it paired with the ``if``
inside the loop. As the loop executes, it will run a sequence like
if/if/if/else. The ``if`` is inside the loop, encountered a number of times. If
the condition is ever true, a ``break`` will happen. If the condition is never
true, the ``else`` clause outside the loop will execute.
When used with a loop, the ``else`` clause has more in common with the ``else``
clause of a try statement than it does with that of ``if``
statements: a ``try`` statement's ``else`` clause runs when no exception
occurs, and a loop's ``else`` clause runs when no ``break`` occurs. For more on
the ``try`` statement and exceptions, see tut-handling.

Contoh Kode / Perintah CLI:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(n, 'equals', x, '*', n//x)
...             break
...     else:
...         # loop fell through without finding a factor
...         print(n, 'is a prime number')
...
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

[python_013] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): pass Statements

Sumber: ht

The pass statement does nothing. It can be used when a statement is
required syntactically but the program requires no action. For example
This is commonly used for creating minimal classes
Another place pass can be used is as a place-holder for a function or
conditional body when you are working on new code, allowing you to keep thinking
at a more abstract level. The pass is silently ignored
For this last case, many people use the ellipsis literal ... instead of
pass. This use has no special meaning to Python, and is not part of
the language definition (you could use any constant expression here), but
... is used conventionally as a placeholder body as well.
See bltin-ellipsis-object.

Contoh Kode / Perintah CLI:

```
>>> while True:
...     pass  # Busy-wait for keyboard interrupt (Ctrl+C)
...

# ----------------------------------------

>>> class MyEmptyClass:
...     pass
...

# ----------------------------------------

>>> def initlog(*args):
...     pass   # Remember to implement this!
...
```

[python_014] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): match Statements

Sumber: ht

A match statement takes an expression and compares its value to successive
patterns given as one or more case blocks. This is superficially
similar to a switch statement in C, Java or JavaScript (and many
other languages), but it's more similar to pattern matching in
languages like Rust or Haskell. Only the first pattern that matches
gets executed and it can also extract components (sequence elements
or object attributes) from the value into variables. If no case matches,
none of the branches is executed.
The simplest form compares a subject value against one or more literals
Note the last block: the "variable name" ``_`` acts as a *wildcard* and
never fails to match.
You can combine several literals in a single pattern using ``|`` ("or")
Patterns can look like unpacking assignments, and can be used to bind
variables
Study that one carefully! The first pattern has two literals, and can
be thought of as an extension of the literal pattern shown above. But
the next two patterns combine a literal and a variable, and the
variable *binds* a value from the subject (``point``). The fourth
pattern captures two values, which makes it conceptually similar to

the unpacking assignment ``(x, y) = point``.
If you are using classes to structure your data
you can use the class name followed by an argument list resembling a
constructor, but with the ability to capture attributes into variables
You can use positional parameters with some builtin classes that provide an
ordering for their attributes (e.g. dataclasses). You can also define a specific
position for attributes in patterns by setting the ``__match_args__`` special
attribute in your classes. If it's set to ("x", "y"), the following patterns are all
equivalent (and all bind the ``y`` attribute to the ``var`` variable)
A reco

Contoh Kode / Perintah CLI:

```
def http_error(status):
match status:
case 400:
return "Bad request"
case 404:
return "Not found"
case 418:
return "I'm a teapot"
case _:
return "Something's wrong with the internet"

# ----------------------------------------

# point is an (x, y) tuple
match point:
case (0, 0):
print("Origin")
case (0, y):
print(f"Y={y}")
case (x, 0):
print(f"X={x}")
case (x, y):
print(f"X={x}, Y={y}")
case _:
raise ValueError("Not a point")

# ----------------------------------------

class Point:
def __init__(self, x, y):
self.x = x
self.y = y
def where_is(point):
match point:
case Point(x=0, y=0):
print("Origin")
case Point(x=0, y=y):
print(f"Y={y}")
case Point(x=x, y=0):
print(f"X={x}")
case Point():
print("Somewhere else")
case _:
print("Not a point")
```

[python_015] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Defining Functions

Sumber: ht

We can create a function that writes the Fibonacci series to an arbitrary
boundary
single: documentation strings
single: docstrings
single: strings, documentation
The keyword def introduces a function *definition*. It must be
followed by the function name and the parenthesized list of formal parameters.
The statements that form the body of the function start at the next line, and
must be indented.
The first statement of the function body can optionally be a string literal;
this string literal is the function's documentation string, or docstring.
(More about docstrings can be found in the section tut-docstrings.)
There are tools which use docstrings to automatically produce online or printed
documentation, or to let the user interactively browse through code; it's good
practice to include docstrings in code that you write, so make a habit of it.
The *execution* of a function introduces a new symbol table used for the local
variables of the function. More precisely, all variable assignments in a
function store the value in the local symbol table; whereas variable references
first look in the local symbol table, then in the local symbol tables of
enclosing functions, then in the global symbol table, and finally in the table
of built-in names. Thus, global variables and variables of enclosing functions
cannot be directly assigned a value within a function (unless, for global
variables, named in a global statement, or, for variables of enclosing
functions, named in a nonlocal statement), although they may be
referenced.
The actual parameters (arguments) to a function call are introduced in the local
symbol table of the called function when it is called; thus, arguments are
passed using *call by value* (where the *value* is always an object *reference*,
not the value of the

Contoh Kode / Perintah CLI:

```
>>> def fib(n):    # write Fibonacci series less than n
...     """Print a Fibonacci series less than n."""
...     a, b = 0, 1
...     while a < n:
...         print(a, end=' ')
...         a, b = b, a+b
...     print()
...
>>> # Now call the function we just defined:
>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597

# ----------------------------------------

>>> fib
<function fib at 10042ed0>
>>> f = fib
>>> f(100)
0 1 1 2 3 5 8 13 21 34 55 89

# ----------------------------------------
```

```
>>> fib(0)
>>> print(fib(0))
None
```

[python_016] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): More on Defining Functions

Sumber: ht

It is also possible to define functions with a variable number of arguments.
There are three forms, which can be combined.

[python_017] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Default Argument Values

Sumber: ht

The most useful form is to specify a default value for one or more arguments.
This creates a function that can be called with fewer arguments than it is
defined to allow. For example
This function can be called in several ways:
* giving only the mandatory argument:
``ask_ok('Do you really want to quit?')``
* giving one of the optional arguments:
``ask_ok('OK to overwrite the file?', 2)``
* or even giving all arguments:
``ask_ok('OK to overwrite the file?', 2, 'Come on, only yes or no!')``
This example also introduces the in keyword. This tests whether or
not a sequence contains a certain value.
The default values are evaluated at the point of function definition in the
defining scope, so that
will print ``5``.
Important warning: The default value is evaluated only once. This makes a
difference when the default is a mutable object such as a list, dictionary, or
instances of most classes. For example, the following function accumulates the
arguments passed to it on subsequent calls
This will print
If you don't want the default to be shared between subsequent calls, you can
write the function like this instead

Contoh Kode / Perintah CLI:

```
def ask_ok(prompt, retries=4, reminder='Please try again!'):
while True:
reply = input(prompt)
if reply in {'y', 'ye', 'yes'}:
return True
if reply in {'n', 'no', 'nop', 'nope'}:
return False
retries = retries - 1
if retries < 0:
raise ValueError('invalid user response')
print(reminder)

# --------------------------------------

i = 5
def f(arg=i):
print(arg)
```

```
i = 6
f()

# --------------------------------------

def f(a, L=[]):
L.append(a)
return L
print(f(1))
print(f(2))
print(f(3))
```

[python_018] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Keyword Arguments

Sumber: ht

Functions can also be called using keyword argument
of the form ``kwarg=value``. For instance, the following function
accepts one required argument (``voltage``) and three optional arguments
(``state``, ``action``, and ``type``). This function can be called in any
of the following ways
but all the following calls would be invalid
In a function call, keyword arguments must follow positional arguments.
All the keyword arguments passed must match one of the arguments
accepted by the function (e.g. ``actor`` is not a valid argument for the
``parrot`` function), and their order is not important. This also includes
non-optional arguments (e.g. ``parrot(voltage=1000)`` is valid too).
No argument may receive a value more than once.
Here's an example that fails due to this restriction
When a final formal parameter of the form ``**name`` is present, it receives a
dictionary (see typesmapping) containing all keyword arguments except for
those corresponding to a formal parameter. This may be combined with a formal
parameter of the form ``*name`` (described in the next subsection) which
receives a tut-tuples containing the positional
arguments beyond the formal parameter list. (``*name`` must occur
before ``**name``.) For example, if we define a function like this
It could be called like this
and of course it would print:
-- Do you have any Limburger ?
-- I'm sorry, we're all out of Limburger
It's very runny, sir.

Contoh Kode / Perintah CLI:

```
def parrot(voltage, state='a stiff', action='voom', type='Norwegian Blue'):
print("-- This parrot wouldn't", action, end=' ')
print("if you put", voltage, "volts through it.")
print("-- Lovely plumage, the", type)
print("-- It's", state, "!")

# --------------------------------------

parrot(1000)                                          # 1 positional argument
parrot(voltage=1000)                                  # 1 keyword argument
parrot(voltage=1000000, action='VOOOOOM')             # 2 keyword arguments
parrot(action='VOOOOOM', voltage=1000000)             # 2 keyword arguments
parrot('a million', 'bereft of life', 'jump')         # 3 positional arguments
parrot('a thousand', state='pushing up the daisies')  # 1 positional, 1 keyword
```

```
# ----------------------------------------

parrot()                     # required argument missing
parrot(voltage=5.0, 'dead')  # non-keyword argument after a keyword argument
parrot(110, voltage=220)     # duplicate value for the same argument
parrot(actor='John Cleese')  # unknown keyword argument
```

[python_019] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): It's really very, VERY runny, sir.

Sumber: ht

shopkeeper : Michael Palin
client : John Cleese
sketch : Cheese Shop Sketch
Note that the order in which the keyword arguments are printed is guaranteed
to match the order in which they were provided in the function call.

[python_020] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Special parameters

Sumber: ht

By default, arguments may be passed to a Python function either by position
or explicitly by keyword. For readability and performance, it makes sense to
restrict the way arguments can be passed so that a developer need only look
at the function definition to determine if items are passed by position, by
position or keyword, or by keyword.
A function definition may look like:
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
---------- ---------- ----------
| | |
| Positional or keyword |
| - Keyword only
-- Positional only
where ``/`` and ``*`` are optional. If used, these symbols indicate the kind of
parameter by how the arguments may be passed to the function:
positional-only, positional-or-keyword, and keyword-only. Keyword parameters
are also referred to as named parameters.

[python_021] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Positional-or-Keyword Arguments

Sumber: ht

If ``/`` and ``*`` are not present in the function definition, arguments may
be passed to a function by position or by keyword.

[python_022] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Positional-Only Parameters

Sumber: ht

Looking at this in a bit more detail, it is possible to mark certain parameters as *positional-only*. If *positional-only*, the parameters' order matters, and the parameters cannot be passed by keyword. Positional-only parameters are placed before a ``/`` (forward-slash). The ``/`` is used to logically separate the positional-only parameters from the rest of the parameters. If there is no ``/`` in the function definition, there are no positional-only parameters.
Parameters following the ``/`` may be *positional-or-keyword* or *keyword-only*.

[python_023] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Keyword-Only Arguments

Sumber: ht

To mark parameters as *keyword-only*, indicating the parameters must be passed by keyword argument, place an ``*`` in the arguments list just before the first *keyword-only* parameter.

[python_024] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Function Examples

Sumber: ht

Consider the following example function definitions paying close attention to the markers ``/`` and ``*``
The first function definition, ``standard_arg``, the most familiar form, places no restrictions on the calling convention and arguments may be passed by position or keyword
The second function ``pos_only_arg`` is restricted to only use positional parameters as there is a ``/`` in the function definition
The third function ``kwd_only_arg`` only allows keyword arguments as indicated by a ``*`` in the function definition
And the last uses all three calling conventions in the same function definition
Finally, consider this function definition which has a potential collision between the positional argument ``name`` and ``**kwds`` which has ``name`` as a key
There is no possible call that will make it return ``True`` as the keyword ``'name'`` will always bind to the first parameter. For example
But using ``/`` (positional only arguments), it is possible since it allows ``name`` as a positional argument and ``'name'`` as a key in the keyword arguments
In other words, the names of positional-only parameters can be used in ``**kwds`` without ambiguity.

Contoh Kode / Perintah CLI:

```
>>> def standard_arg(arg):
...     print(arg)
...
>>> def pos_only_arg(arg, /):
...     print(arg)
...
>>> def kwd_only_arg(*, arg):
...     print(arg)
...
>>> def combined_example(pos_only, /, standard, *, kwd_only):
```

```
...     print(pos_only, standard, kwd_only)

# -----------------------------------------

>>> standard_arg(2)
2
>>> standard_arg(arg=2)
2

# -----------------------------------------

>>> pos_only_arg(1)
1
>>> pos_only_arg(arg=1)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: pos_only_arg() got some positional-only arguments passed as keyword arguments: 'arg'
```

[python_025] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Recap

Sumber: ht

The use case will determine which parameters to use in the function definition
As guidance:
* Use positional-only if you want the name of the parameters to not be
available to the user. This is useful when parameter names have no real
meaning, if you want to enforce the order of the arguments when the function
is called or if you need to take some positional parameters and arbitrary
keywords.
* Use keyword-only when names have meaning and the function definition is
more understandable by being explicit with names or you want to prevent
users relying on the position of the argument being passed.
* For an API, use positional-only to prevent breaking API changes
if the parameter's name is modified in the future.

Contoh Kode / Perintah CLI:

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
```

[python_026] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Arbitrary Argument Lists

Sumber: ht

single: * (asterisk); in function calls
Finally, the least frequently used option is to specify that a function can be
called with an arbitrary number of arguments. These arguments will be wrapped
up in a tuple (see tut-tuples). Before the variable number of arguments,
zero or more normal arguments may occur.
Normally, these *variadic* arguments will be last in the list of formal
parameters, because they scoop up all remaining input arguments that are
passed to the function. Any formal parameters which occur after the ``*args``
parameter are 'keyword-only' arguments, meaning that they can only be used as
keywords rather than positional arguments.

Contoh Kode / Perintah CLI:

```
def write_multiple_items(file, separator, *args):
file.write(separator.join(args))
```

```
# ----------------------------------------

>>> def concat(*args, sep="/"):
...     return sep.join(args)
...
>>> concat("earth", "mars", "venus")
'earth/mars/venus'
>>> concat("earth", "mars", "venus", sep=".")
'earth.mars.venus'
```

[python_027] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Unpacking Argument Lists

Sumber: ht

The reverse situation occurs when the arguments are already in a list or tuple
but need to be unpacked for a function call requiring separate positional
arguments. For instance, the built-in range function expects separate
start and *stop* arguments. If they are not available separately, write the
function call with the ``*``\ -operator to unpack the arguments out of a list
or tuple
single: **; in function calls
In the same fashion, dictionaries can deliver keyword arguments with the
``**``\ -operator

Contoh Kode / Perintah CLI:

```
>>> list(range(3, 6))            # normal call with separate arguments
[3, 4, 5]
>>> args = [3, 6]
>>> list(range(*args))            # call with arguments unpacked from a list
[3, 4, 5]

# ----------------------------------------

>>> def parrot(voltage, state='a stiff', action='voom'):
...     print("-- This parrot wouldn't", action, end=' ')
...     print("if you put", voltage, "volts through it.", end=' ')
...     print("E's", state, "!")
...
>>> d = {"voltage": "four million", "state": "bleedin' demised", "action": "VOOM"}
>>> parrot(**d)
-- This parrot wouldn't VOOM if you put four million volts through it. E's bleedin' demised !
```

[python_028] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Lambda Expressions

Sumber: ht

Small anonymous functions can be created with the lambda keyword.
This function returns the sum of its two arguments: ``lambda a, b: a+b``.
Lambda functions can be used wherever function objects are required. They are
syntactically restricted to a single expression. Semantically, they are just
syntactic sugar for a normal function definition. Like nested function
definitions, lambda functions can reference variables from the containing
scope
The above example uses a lambda expression to return a function. Another use
is to pass a small function as an argument. For instance, list.sort
takes a sorting key function *key* which can be a lambda function

Contoh Kode / Perintah CLI:

```
>>> def make_incrementor(n):
...     return lambda x: x + n
...
>>> f = make_incrementor(42)
>>> f(0)
42
>>> f(1)
43

# --------------------------------------

>>> pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
>>> pairs.sort(key=lambda pair: pair[1])
>>> pairs
[(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]
```

[python_029] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Documentation Strings

Sumber: ht

single: docstrings
single: documentation strings
single: strings, documentation
Here are some conventions about the content and formatting of documentation strings.
The first line should always be a short, concise summary of the object's purpose. For brevity, it should not explicitly state the object's name or type, since these are available by other means (except if the name happens to be a verb describing a function's operation). This line should begin with a capital letter and end with a period.
If there are more lines in the documentation string, the second line should be blank, visually separating the summary from the rest of the description. The following lines should be one or more paragraphs describing the object's calling conventions, its side effects, etc.
The Python parser strips indentation from multi-line string literals when they serve as module, class, or function docstrings.
Here is an example of a multi-line docstring

Contoh Kode / Perintah CLI:

```
>>> def my_function():
...     """Do nothing, but document it.
...
...     No, really, it doesn't do anything:
...
...         >>> my_function()
...         >>>
...     """
...     pass
...
>>> print(my_function.__doc__)
Do nothing, but document it.
No, really, it doesn't do anything:
>>> my_function()
>>>
```

[python_030] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Function Annotations

Sumber: ht

pair: function; annotations
single: ->; function annotations
single: : (colon); function annotations
function are completely optional metadata
information about the types used by user-defined functions (see 3107 and
484 for more information).
function annotation are stored in the ~object.__annotations__
attribute of the function as a dictionary and have no effect on any other part of the
function. Parameter annotations are defined by a colon after the parameter name, followed
by an expression evaluating to the value of the annotation. Return annotations are
defined by a literal ``->``, followed by an expression, between the parameter
list and the colon denoting the end of the def statement. The
following example has a required argument, an optional argument, and the return
value annotated

Contoh Kode / Perintah CLI:

```
>>> def f(ham: str, eggs: str = 'eggs') -> str:
...     print("Annotations:", f.__annotations__)
...     print("Arguments:", ham, eggs)
...     return ham + ' and ' + eggs
...
>>> f('spam')
Annotations: {'ham': <class 'str'>, 'return': <class 'str'>, 'eggs': <class 'str'>}
Arguments: spam eggs
'spam and eggs'
```

[python_031] Python - Control Flow & Pattern Matching

More Control Flow Tools (if, for, match-case, functions): Intermezzo: Coding Style

Sumber: ht

Now that you are about to write longer, more complex pieces of Python, it is a
good time to talk about *coding style*. Most languages can be written (or more
concisely, *formatted*) in different styles; some are more readable than others.
Making it easy for others to read your code is always a good idea, and adopting
a nice coding style helps tremendously for that.
For Python, 8 has emerged as the style guide that most projects adhere to;
it promotes a very readable and eye-pleasing coding style. Every Python
developer should read it at some point; here are the most important points
extracted for you:
* Use 4-space indentation, and no tabs.
4 spaces are a good compromise between small indentation (allows greater
nesting depth) and large indentation (easier to read). Tabs introduce
confusion, and are best left out.
* Wrap lines so that they don't exceed 79 characters.
This helps users with small displays and makes it possible to have several
code files side-by-side on larger displays.
* Use blank lines to separate functions and classes, and larger blocks of
code inside functions.
* When possible, put comments on a line of their own.
* Use docstrings.
* Use spaces around operators and after commas, but not directly inside

bracketing constructs: ``a = f(1, 2) + g(3, 4)``.
* Name your classes and functions consistently; the convention is to use
``UpperCamelCase`` for classes and ``lowercase_with_underscores`` for functions
and methods. Always use ``self`` as the name for the first method argument
(see tut-firstclasses for more on classes and methods).
* Don't use fancy encodings if your code is meant to be used in international
environments. Python's default, UTF-8, or even plain ASCII work best in any
case.
* Likewise, don't use non-ASCII characters in identifiers if

[python_032] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Overview

Sumber: ht

Data Structures
This chapter describes some things you've learned about already in more detail,
and adds some new things as well.

[python_033] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): More on Lists

Sumber: ht

The typesseq-list data type has some more methods. Here are all
of the methods of list objects:
:noindex:
Add an item to the end of the list. Similar to ``a[len(a):] = [x]``.
:noindex:
Extend the list by appending all the items from the iterable. Similar to
``a[len(a):] = iterable``.
:noindex:
Insert an item at a given position. The first argument is the index of the
element before which to insert, so ``a.insert(0, x)`` inserts at the front of
the list, and ``a.insert(len(a), x)`` is equivalent to ``a.append(x)``.
:noindex:
Remove the first item from the list whose value is equal to *value*. It raises a
ValueError if there is no such item.
:noindex:
Remove the item at the given position in the list, and return it. If no index
is specified, ``a.pop()`` removes and returns the last item in the list.
It raises an IndexError if the list is empty or the index is
outside the list range.
:noindex:
Remove all items from the list. Similar to ``del a[:]``.
:noindex:
Return zero-based index of the first occurrence of *value* in the list.
Raises a ValueError if there is no such item.
The optional arguments *start* and *end* are interpreted as in the slice
notation and are used to limit the search to a particular subsequence of
the list. The returned index is computed relative to the beginning of the full
sequence rather than the *start* argument.
:noindex:
Return the number of times *value* appears in the list.
:noindex:

Sort the items of the list in place (the arguments can be used for sort
customization, see sorted for their explanation).
:noindex:
Reverse the elements of the list in place.
:noindex:
Return a shallow copy of the list. Similar to ``a[:]``.
An example that uses most of the list methods
You might have noticed that methods like ``insert``, ``remove`` or ``sor

Contoh Kode / Perintah CLI:

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.count('tangerine')
0
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)  # Find next banana starting at position 4
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange', 'grape']
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'
```

[python_034] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Using Lists as Stacks

Sumber: ht

The list methods make it very easy to use a list as a stack, where the last
element added is the first element retrieved ("last-in, first-out"). To add an
item to the top of the stack, use ~list.append. To retrieve an item from the
top of the stack, use ~list.pop without an explicit index. For example

Contoh Kode / Perintah CLI:

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```

[python_035] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Using Lists as Queues

Sumber: ht

It is also possible to use a list as a queue, where the first element added is
the first element retrieved ("first-in, first-out"); however, lists are not
efficient for this purpose. While appends and pops from the end of list are
fast, doing inserts or pops from the beginning of a list is slow (because all
of the other elements have to be shifted by one). See
time-complexity for more information.
To implement a queue, use collections.deque which was designed to
have fast appends and pops from both ends. For example

Contoh Kode / Perintah CLI:

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # Terry arrives
>>> queue.append("Graham")          # Graham arrives
>>> queue.popleft()                 # The first to arrive now leaves
'Eric'
>>> queue.popleft()                 # The second to arrive now leaves
'John'
>>> queue                           # Remaining queue in order of arrival
deque(['Michael', 'Terry', 'Graham'])
```

[python_036] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): List Comprehensions

Sumber: ht

List comprehensions provide a concise way to create lists.
Common applications are to make new lists where each element is the result of
some operations applied to each member of another sequence or iterable, or to
create a subsequence of those elements that satisfy a certain condition.
For example, assume we want to create a list of squares, like
Note that this creates (or overwrites) a variable named ``x`` that still exists
after the loop completes. We can calculate the list of squares without any
side effects using
or, equivalently
which is more concise and readable.
A list comprehension consists of brackets containing an expression followed
by a for clause, then zero or more for or if
clauses. The result will be a new list resulting from evaluating the expression
in the context of the for and if clauses which follow it.
For example, this listcomp combines the elements of two lists if they are not
equal
and it's equivalent to
Note how the order of the for and if statements is the
same in both these snippets.
If the expression is a tuple (e.g. the ``(x, y)`` in the previous example),
it must be parenthesized.
List comprehensions can contain complex expressions and nested functions

Contoh Kode / Perintah CLI:

```
>>> squares = []
>>> for x in range(10):
...     squares.append(x**2)
```

```
...
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# ----------------------------------------

squares = list(map(lambda x: x**2, range(10)))

# ----------------------------------------

squares = [x**2 for x in range(10)]
```

[python_037] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Nested List Comprehensions

Sumber: ht

The initial expression in a list comprehension can be any arbitrary expression,
including another list comprehension.
Consider the following example of a 3x4 matrix implemented as a list of
3 lists of length 4
The following list comprehension will transpose rows and columns
As we saw in the previous section, the inner list comprehension is evaluated in
the context of the for that follows it, so this example is
equivalent to
which, in turn, is the same as
In the real world, you should prefer built-in functions to complex flow statements.
The zip function would do a great job for this use case
See tut-unpacking-arguments for details on the asterisk in this line.

Contoh Kode / Perintah CLI:

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]

# ----------------------------------------

>>> [[row[i] for row in matrix] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

# ----------------------------------------

>>> transposed = []
>>> for i in range(4):
...     transposed.append([row[i] for row in matrix])
...
>>> transposed
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

[python_038] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Unpacking in Lists and List Comprehensions

Sumber: ht

The section on tut-unpacking-arguments describes the use of ``*`` to
"unpack" the elements of an iterable object, providing each one separately as
an argument to a function. Unpacking can also be used in other contexts, for
example, when creating lists. When specifying elements of a list, prefixing an
expression by a ``*`` will unpack the result of that expression, adding each of
its elements to the list we're creating
This only works if the expression following the ``*`` evaluates to an iterable
object; trying to unpack a non-iterable object will raise an exception
Unpacking can also be used in list comprehensions, as a way to build a new list
representing the concatenation of an arbitrary number of iterables
Note that the effect is that each element from ``x`` is unpacked. This works
for arbitrary iterable objects, not just lists
But if the objects in ``x`` are not iterable, this expression would again raise
an exception.

Contoh Kode / Perintah CLI:

```
>>> x = [1, 2, 3]
>>> [0, *x, 4, 5, 6]
[0, 1, 2, 3, 4, 5, 6]

# ----------------------------------------

>>> x = 1
>>> [0, *x, 2, 3, 4]
Traceback (most recent call last):
File "<python-input-1>", line 1, in <module>
[0, *x, 2, 3, 4]
TypeError: Value after * must be an iterable, not int

# ----------------------------------------

>>> x = [[1, 2, 3], [4, 5, 6], [], [7], [8, 9]]
>>> [*element for element in x]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

[python_039] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): The del statement

Sumber: ht

There is a way to remove an item from a list given its index instead of its
value: the del statement. This differs from the ~list.pop method
which returns a value. The del statement can also be used to remove
slices from a list or clear the entire list (which we did earlier by assignment
of an empty list to the slice). For example
del can also be used to delete entire variables
Referencing the name ``a`` hereafter is an error (at least until another value
is assigned to it). We'll find other uses for del later.

Contoh Kode / Perintah CLI:

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
```

```
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]

# ----------------------------------------

>>> del a
```

[python_040] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Tuples and Sequences

Sumber: ht

We saw that lists and strings have many common properties, such as indexing and slicing operations. They are two examples of *sequence* data types (see typesseq). Since Python is an evolving language, other sequence data types may be added. There is also another standard sequence data type: the *tuple*.
A tuple consists of a number of values separated by commas, for instance
As you see, on output tuples are always enclosed in parentheses, so that nested tuples are interpreted correctly; they may be input with or without surrounding parentheses, although often parentheses are necessary anyway (if the tuple is part of a larger expression). It is not possible to assign to the individual items of a tuple, however it is possible to create tuples which contain mutable objects, such as lists.
Though tuples may seem similar to lists, they are often used in different situations and for different purposes.
Tuples are immutable, and usually contain a heterogeneous sequence of elements that are accessed via unpacking (see later in this section) or indexing (or even by attribute in the case of collections.namedtuple).
Lists are mutable, and their elements are usually homogeneous and are accessed by iterating over the list.
A special problem is the construction of tuples containing 0 or 1 items: the syntax has some extra quirks to accommodate these. Empty tuples are constructed by an empty pair of parentheses; a tuple with one item is constructed by following a value with a comma (it is not sufficient to enclose a single value in parentheses). Ugly, but effective. For example
The statement ``t = 12345, 54321, 'hello!'`` is an example of *tuple packing*: the values ``12345``, ``54321`` and ``'hello!'`` are packed together in a tuple.
The reverse operation is also possible
This i

Contoh Kode / Perintah CLI:

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> # Tuples may be nested:
>>> u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> # Tuples are immutable:
```

```
>>> t[0] = 88888
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # but they can contain mutable objects:
>>> v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
>>> # they support unpacking just like lists:
>>> x = [1, 2, 3]
>>> 0, *x, 4
(0, 1, 2, 3, 4)

# ----------------------------------------

>>> empty = ()
>>> singleton = 'hello',    # <-- note trailing comma
>>> len(empty)
0
>>> len(singleton)
1
>>> singleton
('hello',)

# ----------------------------------------

>>> x, y, z = t
```

[python_041] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Sets

Sumber: ht

Python also includes a data type for types-set. A set is
an unordered collection with no duplicate elements. Basic uses include
membership testing and eliminating duplicate entries. Set objects also
support mathematical operations like union, intersection, difference, and
symmetric difference.
Curly braces or the set function can be used to create sets. Note: to
create an empty set you have to use ``set()``, not ``{}``; the latter creates an
empty dictionary, a data structure that we discuss in the next section.
Because sets are unordered, iterating over them or printing them can
produce the elements in a different order than you expect.
Here is a brief demonstration
Similarly to tut-listcomps, set comprehensions
are also supported, including comprehensions with unpacking

Contoh Kode / Perintah CLI:

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)                      # show that duplicates have been removed
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                 # fast membership testing
True
>>> 'crabgrass' in basket
False
>>> # Demonstrate set operations on unique letters from two words
>>>
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                  # unique letters in a
```

```
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                              # letters in a but not in b
{'r', 'd', 'b'}
>>> a | b                              # letters in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                              # letters in both a and b
{'a', 'c'}
>>> a ^ b                              # letters in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}

# --------------------------------------

>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
>>> fruits = [{'apple', 'avocado', 'apricot'}, {'banana', 'blueberry'}]
>>> {*fruit for fruit in fruits}
{'blueberry', 'banana', 'avocado', 'apple', 'apricot'}
```

[python_042] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Dictionaries

Sumber: ht

Another useful data type built into Python is the *dictionary* (see typesmapping). Dictionaries are sometimes found in other languages as "associative memories" or "associative arrays". Unlike sequences, which are indexed by a range of numbers, dictionaries are indexed by *keys*, which can be any immutable type; strings and numbers can always be keys. Tuples can be used as keys if they contain only strings, numbers, or tuples; if a tuple contains any mutable object either directly or indirectly, it cannot be used as a key. You can't use lists as keys, since lists can be modified in place using index assignments, slice assignments, or methods like ~list.append and ~list.extend.
It is best to think of a dictionary as a set of *key: value* pairs, with the requirement that the keys are unique (within one dictionary). A pair of braces creates an empty dictionary: ``{}``. Placing a comma-separated list of key:value pairs within the braces adds initial key:value pairs to the dictionary; this is also the way dictionaries are written on output.
The main operations on a dictionary are storing a value with some key and extracting the value given the key. It is also possible to delete a key:value pair with ``del``. If you store using a key that is already in use, the old value associated with that key is forgotten.
Extracting a value for a non-existent key by subscripting (``d[key]``) raises a KeyError. To avoid getting this error when trying to access a possibly non-existent key, use the ~dict.get method instead, which returns ``None`` (or a specified default value) if the key is not in the dictionary.
Performing ``list(d)`` on a dictionary returns a list of all the keys used in the dictionary, in insertion order (if you want it sorted, just use ``sorted(d)`` instead). To check

Contoh Kode / Perintah CLI:

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
```

```
>>> tel['irv']
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
KeyError: 'irv'
>>> print(tel.get('irv'))
None
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False

# ----------------------------------------

>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}

# ----------------------------------------

>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

[python_043] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Looping Techniques

Sumber: ht

When looping through dictionaries, the key and corresponding value can be retrieved at the same time using the ~dict.items method.
When looping through a sequence, the position index and corresponding value can be retrieved at the same time using the enumerate function.
To loop over two or more sequences at the same time, the entries can be paired with the zip function.
To loop over a sequence in reverse, first specify the sequence in a forward direction and then call the reversed function.
To loop over a sequence in sorted order, use the sorted function which returns a new sorted list while leaving the source unaltered.
Using set on a sequence eliminates duplicate elements. The use of sorted in combination with set over a sequence is an idiomatic way to loop over unique elements of the sequence in sorted order.
It is sometimes tempting to change a list while you are looping over it; however, it is often simpler and safer to create a new list instead.

Contoh Kode / Perintah CLI:

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():
...     print(k, v)
...
gallahad the pure
robin the brave

# ----------------------------------------
```

```
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
...
0 tic
1 tac
2 toe

# --------------------------------------

>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print('What is your {0}?  It is {1}.'.format(q, a))
...
What is your name?  It is lancelot.
What is your quest?  It is the holy grail.
What is your favorite color?  It is blue.
```

[python_044] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): More on Conditions

Sumber: ht

The conditions used in ``while`` and ``if`` statements can contain any operators, not just comparisons.
The comparison operators ``in`` and ``not in`` are membership tests that determine whether a value is in (or not in) a container. The operators ``is`` and ``is not`` compare whether two objects are really the same object. All comparison operators have the same priority, which is lower than that of all numerical operators.
Comparisons can be chained. For example, ``a < b == c`` tests whether ``a`` is less than ``b`` and moreover ``b`` equals ``c``.
Comparisons may be combined using the Boolean operators ``and`` and ``or``, and the outcome of a comparison (or of any other Boolean expression) may be negated with ``not``. These have lower priorities than comparison operators; between them, ``not`` has the highest priority and ``or`` the lowest, so that ``A and not B or C`` is equivalent to ``(A and (not B)) or C``. As always, parentheses can be used to express the desired composition.
The Boolean operators ``and`` and ``or`` are so-called *short-circuit* operators: their arguments are evaluated from left to right, and evaluation stops as soon as the outcome is determined. For example, if ``A`` and ``C`` are true but ``B`` is false, ``A and B and C`` does not evaluate the expression ``C``. When used as a general value and not as a Boolean, the return value of a short-circuit operator is the last evaluated argument.
It is possible to assign the result of a comparison or other Boolean expression to a variable. For example,
Note that in Python, unlike C, assignment inside expressions must be done explicitly with the
why-can-t-i-use-an-assignment-in-an-expression ``:=``.
This avoids a common class of problems encountered in C programs: typing ``=`` in an expression when

Contoh Kode / Perintah CLI:

```
>>> string1, string2, string3 = '', 'Trondheim', 'Hammer Dance'
>>> non_null = string1 or string2 or string3
>>> non_null
'Trondheim'
```

[python_045] Python - Data Structures

Data Structures (Lists, Tuples, Sets, Dicts, Comprehensions): Comparing Sequences and Other Types

Sumber: ht

Sequence objects typically may be compared to other objects with the same sequence type. The comparison uses *lexicographical* ordering: first the first two items are compared, and if they differ this determines the outcome of the comparison; if they are equal, the next two items are compared, and so on, until either sequence is exhausted. If two items to be compared are themselves sequences of the same type, the lexicographical comparison is carried out recursively. If all items of two sequences compare equal, the sequences are considered equal. If one sequence is an initial sub-sequence of the other, the shorter sequence is the smaller (lesser) one. Lexicographical ordering for strings uses the Unicode code point number to order individual characters. Some examples of comparisons between sequences of the same type Note that comparing objects of different types with ``<`` or ``>`` is legal provided that the objects have appropriate comparison methods. For example, mixed numeric types are compared according to their numeric value, so 0 equals 0.0, etc. Otherwise, rather than providing an arbitrary ordering, the interpreter will raise a TypeError exception.
.. [#] Other languages may return the mutated object, which allows method chaining, such as ``d->insert("a")->remove("b")->sort();``.

Contoh Kode / Perintah CLI:

```
(1, 2, 3)              < (1, 2, 4)
[1, 2, 3]              < [1, 2, 4]
'ABC' < 'C' < 'Pascal' < 'Python'
(1, 2, 3, 4)           < (1, 2, 4)
(1, 2)                 < (1, 2, -1)
(1, 2, 3)             == (1.0, 2.0, 3.0)
(1, 2, ('aa', 'ab'))   < (1, 2, ('abc', 'a'), 4)
```

[python_046] Python - Modules & Packages

Modules and Packages: Overview

Sumber: ht

Modules
If you quit from the Python interpreter and enter it again, the definitions you have made (functions and variables) are lost. Therefore, if you want to write a somewhat longer program, you are better off using a text editor to prepare the input for the interpreter and running it with that file as input instead. This is known as creating a *script*. As your program gets longer, you may want to split it into several files for easier maintenance. You may also want to use a handy function that you've written in several programs without copying its definition into each program.
To support this, Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a *module*; definitions from a module can be *imported* into other modules or into the *main* module (the collection of variables that you have access to in a script executed at the top level and in calculator mode).
A module is a file containing Python definitions and statements. The file name is the module name with the suffix .py appended. Within a module, the module's name (as a string) is available as the value of the global variable

``__name__``. For instance, use your favorite text editor to create a file
called fibo.py in the current directory with the following contents
Now enter the Python interpreter and import this module with the following
command
This does not add the names of the functions defined in ``fibo`` directly to
the current namespace (see tut-scopes for more details);
it only adds the module name ``fibo`` there. Using
the module name you can access the functions
If you intend to use a function often you can assign it to a local name

Contoh Kode / Perintah CLI:

```
# Fibonacci numbers module
def fib(n):
"""Write Fibonacci series up to n."""
a, b = 0, 1
while a < n:
print(a, end=' ')
a, b = b, a+b
print()
def fib2(n):
"""Return Fibonacci series up to n."""
result = []
a, b = 0, 1
while a < n:
result.append(a)
a, b = b, a+b
return result

# --------------------------------------

>>> import fibo

# --------------------------------------

>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

[python_047] Python - Modules & Packages

Modules and Packages: More on Modules

Sumber: ht

A module can contain executable statements as well as function definitions.
These statements are intended to initialize the module. They are executed only
the *first* time the module name is encountered in an import statement. [#]_
(They are also run if the file is executed as a script.)
Each module has its own private namespace, which is used as the global namespace
by all functions defined in the module. Thus, the author of a module can
use global variables in the module without worrying about accidental clashes
with a user's global variables. On the other hand, if you know what you are
doing you can touch a module's global variables with the same notation used to
refer to its functions, ``modname.itemname``.
Modules can import other modules. It is customary but not required to place all
import statements at the beginning of a module (or script, for that

matter). The imported module names, if placed at the top level of a module (outside any functions or classes), are added to the module's global namespace.
There is a variant of the import statement that imports names from a module directly into the importing module's namespace. For example
This does not introduce the module name from which the imports are taken in the local namespace (so in the example, ``fibo`` is not defined).
There is even a variant to import all names that a module defines
This imports all names except those beginning with an underscore (``_``).
In most cases Python programmers do not use this facility since it introduces an unknown set of names into the interpreter, possibly hiding some things you have already defined.
Note that in general the practice of importing ``*`` from a module or package is frowned upon, since it often causes poorly readable code. However, it is okay to use it to save

Contoh Kode / Perintah CLI:

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

# ----------------------------------------

>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

# ----------------------------------------

>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

[python_048] Python - Modules & Packages

Modules and Packages: Executing modules as scripts

Sumber: ht

When you run a Python module with
the code in the module will be executed, just as if you imported it, but with the ``__name__`` set to ``"__main__"``. That means that by adding this code at the end of your module
you can make the file usable as a script as well as an importable module, because the code that parses the command line only runs if the module is executed as the "main" file:
$ python fibo.py 50
0 1 1 2 3 5 8 13 21 34
If the module is imported, the code is not run
This is often used either to provide a convenient user interface to a module, or for testing purposes (running the module as a script executes a test suite).

Contoh Kode / Perintah CLI:

```
if __name__ == "__main__":
import sys
fib(int(sys.argv[1]))

# ----------------------------------------

>>> import fibo
```

```
>>>
```

[python_049] Python - Modules & Packages

Modules and Packages: The Module Search Path

Sumber: ht

When a module named spam is imported, the interpreter first searches for
a built-in module with that name. These module names are listed in
sys.builtin_module_names. If not found, it then searches for a file
named spam.py in a list of directories given by the variable
sys.path. sys.path is initialized from these locations:
* The directory containing the input script (or the current directory when no
file is specified).
* PYTHONPATH (a list of directory names, with the same syntax as the
shell variable PATH).
* The installation-dependent default (by convention including a
``site-packages`` directory, handled by the site module).
More details are at sys-path-init.
On file systems which support symlinks, the directory containing the input
script is calculated after the symlink is followed. In other words the
directory containing the symlink is **not** added to the module search path.
After initialization, Python programs can modify sys.path. The
directory containing the script being run is placed at the beginning of the
search path, ahead of the standard library path. This means that scripts in that
directory will be loaded instead of modules of the same name in the library
directory. This is an error unless the replacement is intended. See section
tut-standardmodules for more information.
.. %
Do we need stuff on zip files etc. ? DUBOIS

[python_050] Python - Modules & Packages

Modules and Packages: "Compiled" Python files

Sumber: ht

To speed up loading modules, Python caches the compiled version of each module
in the ``__pycache__`` directory under the name module.{version}.pyc,
where the version encodes the format of the compiled file; it generally contains
the Python version number. For example, in CPython release 3.3 the compiled
version of spam.py would be cached as ``__pycache__/spam.cpython-33.pyc``. This
naming convention allows compiled modules from different releases and different
versions of Python to coexist.
Python checks the modification date of the source against the compiled version
to see if it's out of date and needs to be recompiled. This is a completely
automatic process. Also, the compiled modules are platform-independent, so the
same library can be shared among systems with different architectures.
Python does not check the cache in two circumstances. First, it always
recompiles and does not store the result for the module that's loaded directly
from the command line. Second, it does not check the cache if there is no
source module. To support a non-source (compiled only) distribution, the
compiled module must be in the source directory, and there must not be a source
module.
Some tips for experts:

* You can use the -O or -OO switches on the Python command
to reduce the size of a compiled module. The ``-O`` switch removes assert
statements, the ``-OO`` switch removes both assert statements and __doc__
strings. Since some programs may rely on having these available, you should
only use this option if you know what you're doing. "Optimized" modules have
an ``opt-`` tag and are usually smaller. Future releases may
change the effects of optimization.
* A program doesn't run any faster when it is read from a ``.pyc``
file than when it is read from a ``.py`` file; the only

[python_051] Python - Modules & Packages

Modules and Packages: Standard Modules

Sumber: ht

Python comes with a library of standard modules, described in a separate
document, the Python Library Reference ("Library Reference" hereafter). Some
modules are built into the interpreter; these provide access to operations that
are not part of the core of the language but are nevertheless built in, either
for efficiency or to provide access to operating system primitives such as
system calls. The set of such modules is a configuration option which also
depends on the underlying platform. For example, the winreg module is only
provided on Windows systems. One particular module deserves some attention:
sys, which is built into every Python interpreter. The variables
``sys.ps1`` and ``sys.ps2`` define the strings used as primary and secondary
prompts
These two variables are only defined if the interpreter is in interactive mode.
The variable ``sys.path`` is a list of strings that determines the interpreter's
search path for modules. It is initialized to a default path taken from the
environment variable PYTHONPATH, or from a built-in default if
PYTHONPATH is not set. You can modify it using standard list
operations

Contoh Kode / Perintah CLI:

```
>>> import sys
>>> sys.ps1
'>>> '
>>> sys.ps2
'... '
>>> sys.ps1 = 'C> '
C> print('Yuck!')
Yuck!
C>

# -----------------------------------------

>>> import sys
>>> sys.path.append('/ufs/guido/lib/python')
```

[python_052] Python - Modules & Packages

Modules and Packages: The dir Function

Sumber: ht

The built-in function dir is used to find out which names a module defines. It returns a sorted list of strings
Without arguments, dir lists the names you have defined currently
Note that it lists all types of names: variables, modules, functions, etc.
dir does not list the names of built-in functions and variables. If you want a list of those, they are defined in the standard module builtins

Contoh Kode / Perintah CLI:

```
>>> import fibo, sys
>>> dir(fibo)
['__name__', 'fib', 'fib2']
>>> dir(sys)  # doctest: +NORMALIZE_WHITESPACE
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthook__',
'__interactivehook__', '__loader__', '__name__', '__package__', '__spec__',
'__stderr__', '__stdin__', '__stdout__', '__unraisablehook__',
'_clear_type_cache', '_current_frames', '_debugmallocstats', '_framework',
'_getframe', '_git', '_home', '_xoptions', 'abiflags', 'addaudithook',
'api_version', 'argv', 'audit', 'base_exec_prefix', 'base_prefix',
'breakpointhook', 'builtin_module_names', 'byteorder', 'call_tracing',
'callstats', 'copyright', 'displayhook', 'dont_write_bytecode', 'exc_info',
'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'float_info',
'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_depth',
'getallocatedblocks', 'getdefaultencoding', 'getdlopenflags',
'getfilesystemencodeerrors', 'getfilesystemencoding', 'getprofile',
'getrecursionlimit', 'getrefcount', 'getsizeof', 'getswitchinterval',
'gettrace', 'hash_info', 'hexversion', 'implementation', 'int_info',
'intern', 'is_finalizing', 'last_traceback', 'last_type', 'last_value',
'maxsize', 'maxunicode', 'meta_path', 'modules', 'path', 'path_hooks',
'path_importer_cache', 'platform', 'prefix', 'ps1', 'ps2', 'pycache_prefix',
'set_asyncgen_hooks', 'set_coroutine_origin_tracking_depth', 'setdlopenflags',
'setprofile', 'setrecursionlimit', 'setswitchinterval', 'settrace', 'stderr',
'stdin', 'stdout', 'thread_info', 'unraisablehook', 'version', 'version_info',
'warnoptions']

# ----------------------------------------

>>> a = [1, 2, 3, 4, 5]
>>> import fibo
>>> fib = fibo.fib
>>> dir()
['__builtins__', '__name__', 'a', 'fib', 'fibo', 'sys']

# ----------------------------------------

>>> import builtins
>>> dir(builtins)  # doctest: +NORMALIZE_WHITESPACE
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning',
'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False',
'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented',
```

```
'NotImplementedError', 'OSError', 'OverflowError',
'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError',
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'ZeroDivisionError', '_', '__build_class__',
'__debug__', '__doc__', '__import__', '__name__', '__package__', 'abs',
'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable',
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit',
'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr',
'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass',
'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview',
'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice',
'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars',
'zip']
```

[python_053] Python - Modules & Packages

Modules and Packages: Packages

Sumber: ht

Packages are a way of structuring Python's module namespace by using "dotted module names". For example, the module name A.B designates a submodule named ``B`` in a package named ``A``. Just like the use of modules saves the authors of different modules from having to worry about each other's global variable names, the use of dotted module names saves the authors of multi-module packages like NumPy or Pillow from having to worry about each other's module names.

Suppose you want to design a collection of modules (a "package") for the uniform handling of sound files and sound data. There are many different sound file formats (usually recognized by their extension, for example: .wav, .aiff, .au), so you may need to create and maintain a growing collection of modules for the conversion between the various file formats. There are also many different operations you might want to perform on sound data (such as mixing, adding echo, applying an equalizer function, creating an artificial stereo effect), so in addition you will be writing a never-ending stream of modules to perform these operations. Here's a possible structure for your package (expressed in terms of a hierarchical filesystem):

sound/ Top-level package
__init__.py Initialize the sound package
formats/ Subpackage for file format conversions
__init__.py
wavread.py
wavwrite.py
aiffread.py
aiffwrite.py
auread.py
auwrite.py
...
effects/ Subpackage for sound effects
__init__.py
echo.py

surround.py
reverse.py
...
filters/ Subpackage for filters
__init__.py
equalizer.py
vocoder.py
karaoke.py
...
When importing the package, Python searches through the directories on
``sys.path`` looking for the package subdirectory.
The __init__.py

Contoh Kode / Perintah CLI:

```
import sound.effects.echo

# ----------------------------------------

sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)

# ----------------------------------------

from sound.effects import echo
```

[python_054] Python - Modules & Packages

Modules and Packages: Importing * From a Package

Sumber: ht

Now what happens when the user writes ``from sound.effects import *``? Ideally, one would hope that this somehow goes out to the filesystem, finds which submodules are present in the package, and imports them all. This could take a long time and importing sub-modules might have unwanted side-effects that should only happen when the sub-module is explicitly imported.
The only solution is for the package author to provide an explicit index of the package. The import statement uses the following convention: if a package's __init__.py code defines a list named ``__all__``, it is taken to be the list of module names that should be imported when ``from package import *`` is encountered. It is up to the package author to keep this list up-to-date when a new version of the package is released. Package authors may also decide not to support it, if they don't see a use for importing * from their package. For example, the file sound/effects/__init__.py could contain the following code
This would mean that ``from sound.effects import *`` would import the three named submodules of the sound.effects package.
Be aware that submodules might become shadowed by locally defined names. For example, if you added a ``reverse`` function to the sound/effects/__init__.py file, the ``from sound.effects import *`` would only import the two submodules ``echo`` and ``surround``, but *not* the ``reverse`` submodule, because it is shadowed by the locally defined ``reverse`` function
If ``__all__`` is not defined, the statement ``from sound.effects import *`` does *not* import all submodules from the package sound.effects into the current namespace; it only ensures that the package sound.effects has been imported (possibly running any initialization code in __init__.py)

and then imports whateve

Contoh Kode / Perintah CLI:

```
__all__ = ["echo", "surround", "reverse"]

# ----------------------------------------

__all__ = [
"echo",      # refers to the 'echo.py' file
"surround",  # refers to the 'surround.py' file
"reverse",   # !!! refers to the 'reverse' function now !!!
]
def reverse(msg: str):  # <-- this name shadows the 'reverse.py' submodule
return msg[::-1]    #     in the case of a 'from sound.effects import *'

# ----------------------------------------

import sound.effects.echo
import sound.effects.surround
from sound.effects import *
```

[python_055] Python - Modules & Packages

Modules and Packages: Intra-package References

Sumber: ht

When packages are structured into subpackages (as with the sound package
in the example), you can use absolute imports to refer to submodules of siblings
packages. For example, if the module sound.filters.vocoder needs to use
the echo module in the sound.effects package, it can use ``from
sound.effects import echo``.
You can also write relative imports, with the ``from module import name`` form
of import statement. These imports use leading dots to indicate the current and
parent packages involved in the relative import. From the surround
module for example, you might use
Note that relative imports are based on the name of the current module's package.
Since the main module does not have a package, modules intended for use
as the main module of a Python application must always use absolute imports.
.. [#] In fact function definitions are also 'statements' that are 'executed'; the
execution of a module-level function definition adds the function name to
the module's global namespace.

Contoh Kode / Perintah CLI:

```
from . import echo
from .. import formats
from ..filters import equalizer
```

[python_056] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Overview

Sumber: ht

Input and output
There are several ways to present the output of a program; data can be printed
in a human-readable form, or written to a file for future use. This chapter will
discuss some of the possibilities.

[python_057] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Fancier output formatting

Sumber: ht

So far we've encountered two ways of writing values: *expression statements* and
the print function. (A third way is using the ~io.TextIOBase.write method
of file objects; the standard output file can be referenced as ``sys.stdout``.
See the Library Reference for more information on this.)
Often you'll want more control over the formatting of your output than simply
printing space-separated values. There are several ways to format output.
* To use tut-f-strings, begin a string
with ``f`` or ``F`` before the opening quotation mark or triple quotation mark.
Inside this string, you can write a Python expression between ``{`` and ``}``
characters that can refer to variables or literal values.
* The str.format method of strings requires more manual
effort. You'll still use ``{`` and ``}`` to mark where a variable
will be substituted and can provide detailed formatting directives,
but you'll also need to provide the information to be formatted. In the following code
block there are two examples of how to format variables:
The example also prints ``percentage`` multiplied by 100, with 2 decimal
places and followed by a percent sign (see formatspec for details).
* Finally, you can do all the string handling yourself by using string slicing and
concatenation operations to create any layout you can imagine. The
string type has some methods that perform useful operations for padding
strings to a given column width.
When you don't need fancy output but just want a quick display of some
variables for debugging purposes, you can convert any value to a string with
the repr or str functions.
The str function is meant to return representations of values which are
fairly human-readable, while repr is meant to generate representations
which can be read by the interpreter (or will force

Contoh Kode / Perintah CLI:

```
>>> year = 2016
>>> event = 'Referendum'
>>> f'Results of the {year} {event}'
'Results of the 2016 Referendum'

# ----------------------------------------

>>> yes_votes = 42_572_654
>>> total_votes = 85_705_149
>>> percentage = yes_votes / total_votes
>>> '{:-9} YES votes  {:2.2%}'.format(yes_votes, percentage)
' 42572654 YES votes  49.67%'
Notice how the ``yes_votes`` are padded with spaces and a negative sign only for negative numbers.

# ----------------------------------------

>>> s = 'Hello, world.'
>>> str(s)
'Hello, world.'
>>> repr(s)
"'Hello, world.'"
>>> str(1/7)
'0.14285714285714285'
>>> x = 10 * 3.25
```

```
>>> y = 200 * 200
>>> s = 'The value of x is ' + repr(x) + ', and y is ' + repr(y) + '...'
>>> print(s)
The value of x is 32.5, and y is 40000...
>>> # The repr() of a string adds string quotes and backslashes:
>>> hello = 'hello, world\n'
>>> hellos = repr(hello)
>>> print(hellos)
'hello, world\n'
>>> # The argument to repr() may be any Python object:
>>> repr((x, y, ('spam', 'eggs')))
"(32.5, 40000, ('spam', 'eggs'))"
```

[python_058] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Formatted string literals

Sumber: ht

f-strings (also called f-strings for
short) let you include the value of Python expressions inside a string by
prefixing the string with ``f`` or ``F`` and writing expressions as
``{expression}``.
An optional format specifier can follow the expression. This allows greater
control over how the value is formatted. The following example rounds pi to
three places after the decimal
Passing an integer after the ``':'`` will cause that field to be a minimum
number of characters wide. This is useful for making columns line up.
Other modifiers can be used to convert the value before it is formatted.
``!a`` applies ascii, ``!s`` applies str, and ``!r``
applies repr
The ``=`` specifier can be used to expand an expression to the text of the
expression, an equal sign, then the representation of the evaluated expression:
>>> bugs = 'roaches'
>>> count = 13
>>> area = 'living room'
>>> print(f'Debugging {bugs=} {count=} {area=}')
Debugging bugs='roaches' count=13 area='living room'
See bpo-36817-whatsnew for more information
on the ``=`` specifier. For a reference on these format specifications, see
the reference guide for the formatspec.

Contoh Kode / Perintah CLI:

```
>>> import math
>>> print(f'The value of pi is approximately {math.pi:.3f}.')
The value of pi is approximately 3.142.

# ----------------------------------------

>>> table = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 7678}
>>> for name, phone in table.items():
...     print(f'{name:10} ==> {phone:10d}')
...
Sjoerd     ==>       4127
Jack       ==>       4098
Dcab       ==>       7678

# ----------------------------------------
```

```
>>> animals = 'eels'
>>> print(f'My hovercraft is full of {animals}.')
My hovercraft is full of eels.
>>> print(f'My hovercraft is full of {animals!r}.')
My hovercraft is full of 'eels'.
```

[python_059] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): The string format() method

Sumber: ht

Basic usage of the str.format method looks like this
The brackets and characters within them (called format fields) are replaced with
the objects passed into the str.format method. A number in the
brackets can be used to refer to the position of the object passed into the
str.format method.
If keyword arguments are used in the str.format method, their values
are referred to by using the name of the argument.
Positional and keyword arguments can be arbitrarily combined
If you have a really long format string that you don't want to split up, it
would be nice if you could reference the variables to be formatted by name
instead of by position. This can be done by simply passing the dict and using
square brackets ``'[]'`` to access the keys.
This could also be done by passing the ``table`` dictionary as keyword arguments with the ``**``
notation.
This is particularly useful in combination with the built-in function
vars, which returns a dictionary containing all local variables
As an example, the following lines produce a tidily aligned
set of columns giving integers and their squares and cubes
For a complete overview of string formatting with str.format, see
formatstrings.

Contoh Kode / Perintah CLI:

```
>>> print('We are the {} who say "{}!"'.format('knights', 'Ni'))
We are the knights who say "Ni!"

# ----------------------------------------

>>> print('{0} and {1}'.format('spam', 'eggs'))
spam and eggs
>>> print('{1} and {0}'.format('spam', 'eggs'))
eggs and spam

# ----------------------------------------

>>> print('This {food} is {adjective}.'.format(
...       food='spam', adjective='absolutely horrible'))
This spam is absolutely horrible.
```

[python_060] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Manual string formatting

Sumber: ht

Here's the same table of squares and cubes, formatted manually
(Note that the one space between each column was added by the
way print works: it always adds spaces between its arguments.)
The str.rjust method of string objects right-justifies a string in a
field of a given width by padding it with spaces on the left. There are
similar methods str.ljust and str.center. These methods do
not write anything, they just return a new string. If the input string is too
long, they don't truncate it, but return it unchanged; this will mess up your
column lay-out but that's usually better than the alternative, which would be
lying about a value. (If you really want truncation you can always add a
slice operation, as in ``x.ljust(n)[:n]``.)
There is another method, str.zfill, which pads a numeric string on the
left with zeros. It understands about plus and minus signs

Contoh Kode / Perintah CLI:

```
>>> for x in range(1, 11):
...     print(repr(x).rjust(2), repr(x*x).rjust(3), end=' ')
...     # Note use of 'end' on previous line
...     print(repr(x*x*x).rjust(4))
...
1   1    1
2   4    8
3   9   27
4  16   64
5  25  125
6  36  216
7  49  343
8  64  512
9  81  729
10 100 1000

# ----------------------------------------

>>> '12'.zfill(5)
'00012'
>>> '-3.14'.zfill(7)
'-003.14'
>>> '3.14159265359'.zfill(5)
'3.14159265359'
```

[python_061] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Old string formatting

Sumber: ht

The % operator (modulo) can also be used for string formatting.
Given ``format % values`` (where *format* is a string),
``%`` conversion specifications in *format* are replaced with
zero or more elements of *values*.
This operation is commonly known as string
interpolation. For example
More information can be found in the old-string-formatting section.

Contoh Kode / Perintah CLI:

```
>>> import math
```

```
>>> print('The value of pi is approximately %5.3f.' % math.pi)
The value of pi is approximately 3.142.
```

[python_062] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Reading and writing files

Sumber: ht

pair: built-in function; open
pair: object; file
open returns a file object, and is most commonly used with
two positional arguments: ``open(filename, mode)``
.. XXX str(f) is <io.TextIOWrapper object at 0x82e8dc4>
>>> print(f)
<open file 'workfile', mode 'w' at 80a0960>
The first argument is a string containing the filename. The second argument is
another string containing a few characters describing the way in which the file
will be used. *mode* can be ``'r'`` when the file will only be read, ``'w'``
for only writing (an existing file with the same name will be erased), and
``'a'`` opens the file for appending; any data written to the file is
automatically added to the end. ``'r+'`` opens the file for both reading and
writing. The *mode* argument is optional; ``'r'`` will be assumed if it's
omitted.
Normally, files are opened in text mode, that means, you read and write
strings from and to the file, which are encoded in a specific *encoding*.
If *encoding* is not specified, the default is UTF-8 (see open).
Appending a ``'b'`` to the mode opens the file in binary mode.
Binary mode data is read and written as bytes objects.
You can not specify *encoding* when opening file in binary mode.
In text mode, the default when reading is to convert platform-specific line
endings (``\n`` on Unix, ``\r\n`` on Windows) to just ``\n``. When writing in
text mode, the default is to convert occurrences of ``\n`` back to
platform-specific line endings. This behind-the-scenes modification
to file data is fine for text files, but will corrupt binary data like that in
JPEG or EXE files. Be very careful to use binary mode when
reading and writing such files.
It is good practice to use the with keyword when dealing
with file objects. The advantage is that the file is properly closed
a

Contoh Kode / Perintah CLI:

```
>>> f = open('workfile', 'w')

# ----------------------------------------

>>> with open('workfile') as f:
...     read_data = f.read()
>>> # We can check that the file has been automatically closed.
>>> f.closed
True

# ----------------------------------------

>>> f.close()
>>> f.read()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
```

```
ValueError: I/O operation on closed file.
```

[python_063] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Methods of file objects

Sumber: ht

The rest of the examples in this section will assume that a file object called
``f`` has already been created.
To read a file's contents, call ``f.read(size)``, which reads some quantity of
data and returns it as a string (in text mode) or bytes object (in binary mode).
size is an optional numeric argument. When *size* is omitted or negative, the
entire contents of the file will be read and returned; it's your problem if the
file is twice as large as your machine's memory. Otherwise, at most *size*
characters (in text mode) or *size* bytes (in binary mode) are read and returned.
If the end of the file has been reached, ``f.read()`` will return an empty
string (``''``).
``f.readline()`` reads a single line from the file; a newline character (``\n``)
is left at the end of the string, and is only omitted on the last line of the
file if the file doesn't end in a newline. This makes the return value
unambiguous; if ``f.readline()`` returns an empty string, the end of the file
has been reached, while a blank line is represented by ``\n``, a string
containing only a single newline.
For reading lines from a file, you can loop over the file object. This is memory
efficient, fast, and leads to simple code
If you want to read all the lines of a file in a list you can also use
``list(f)`` or ``f.readlines()``.
``f.write(string)`` writes the contents of *string* to the file, returning
the number of characters written.
Other types of objects need to be converted -- either to a string (in text mode)
or a bytes object (in binary mode) -- before writing them
``f.tell()`` returns an integer giving the file object's current position in the file
represented as number of bytes from the beginning of the file when in binary mode and
an opaque number when in text mode.
To change the file

Contoh Kode / Perintah CLI:

```
>>> f.read()
'This is the entire file.\n'
>>> f.read()
''

# ----------------------------------------

>>> f.readline()
'This is the first line of the file.\n'
>>> f.readline()
'Second line of the file\n'
>>> f.readline()
''

# ----------------------------------------

>>> for line in f:
...     print(line, end='')
...
This is the first line of the file.
```

```
Second line of the file
```

[python_064] Python - Input & Output (F-Strings)

Input and Output (Formatted String Literals, Files): Saving structured data with json

Sumber: ht

Strings can easily be written to and read from a file. Numbers take a bit more
effort, since the ~io.TextIOBase.read method only returns strings, which will have to
be passed to a function like int, which takes a string like ``'123'``
and returns its numeric value 123. When you want to save more complex data
types like nested lists and dictionaries, parsing and serializing by hand
becomes complicated.
Rather than having users constantly writing and debugging code to save
complicated data types to files, Python allows you to use the popular data
interchange format called `JSON (JavaScript Object Notation)
<https://json.org>`_. The standard module called json can take Python
data hierarchies, and convert them to string representations; this process is
called serializing. Reconstructing the data from the string representation
is called deserializing. Between serializing and deserializing, the
string representing the object may have been stored in a file or data, or
sent over a network connection to some distant machine.
The JSON format is commonly used by modern applications to allow for data
exchange. Many programmers are already familiar with it, which makes
it a good choice for interoperability.
If you have an object ``x``, you can view its JSON string representation with a
simple line of code
Another variant of the ~json.dumps function, called ~json.dump,
simply serializes the object to a text file. So if ``f`` is a
text file object opened for writing, we can do this
To decode the object again, if ``f`` is a binary file or
text file object which has been opened for reading
JSON files must be encoded in UTF-8, the default encoding for
text file.
This simple serialization technique can handle lists and dictionaries, but
serializing arbitrary class instances in JSO

Contoh Kode / Perintah CLI:

```
>>> import json
>>> x = [1, 'simple', 'list']
>>> json.dumps(x)
'[1, "simple", "list"]'

# ----------------------------------------

x = json.load(f)
```

[python_065] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Overview

Sumber: ht

Errors and Exceptions
Until now error messages haven't been more than mentioned, but if you have tried out the examples you have probably seen some. There are (at least) two distinguishable kinds of errors: *syntax errors* and *exceptions*.

[python_066] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Syntax Errors

Sumber: ht

Syntax errors, also known as parsing errors, are perhaps the most common kind of complaint you get while you are still learning Python
The parser repeats the offending line and displays little arrows pointing at the place where the error was detected. Note that this is not always the place that needs to be fixed. In the example, the error is detected at the function print, since a colon (``':'``) is missing just before it.
The file name (``<stdin>`` in our example) and line number are printed so you know where to look in case the input came from a file.

Contoh Kode / Perintah CLI:

```
>>> while True print('Hello world')
File "<stdin>", line 1
while True print('Hello world')
^^^^^
SyntaxError: invalid syntax
```

[python_067] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Exceptions

Sumber: ht

Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called *exceptions* and are not unconditionally fatal: you will soon learn how to handle them in Python programs. Most exceptions are not handled by programs, however, and result in error messages as shown here
The last line of the error message indicates what happened. Exceptions come in different types, and the type is printed as part of the message: the types in the example are ZeroDivisionError, NameError and TypeError.
The string printed as the exception type is the name of the built-in exception that occurred. This is true for all built-in exceptions, but need not be true for user-defined exceptions (although it is a useful convention). Standard exception names are built-in identifiers (not reserved keywords).
The rest of the line provides detail based on the type of exception and what caused it.
The preceding part of the error message shows the context where the exception occurred, in the form of a stack traceback. In general it contains a stack traceback listing source lines; however, it will not display lines read from standard input.
bltin-exceptions lists the built-in exceptions and their meanings.

Contoh Kode / Perintah CLI:

```
>>> 10 * (1/0)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
10 * (1/0)
~^~
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
4 + spam*3
^^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
'2' + 2
~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

[python_068] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Handling Exceptions

Sumber: ht

It is possible to write programs that handle selected exceptions. Look at the
following example, which asks the user for input until a valid integer has been
entered, but allows the user to interrupt the program (using Control-C or
whatever the operating system supports); note that a user-generated interruption
is signalled by raising the KeyboardInterrupt exception.
The try statement works as follows.
* First, the *try clause* (the statement(s) between the try and
except keywords) is executed.
* If no exception occurs, the *except clause* is skipped and execution of the
try statement is finished.
* If an exception occurs during execution of the try clause, the rest of the
clause is skipped. Then, if its type matches the exception named after the
except keyword, the *except clause* is executed, and then execution
continues after the try/except block.
* If an exception occurs which does not match the exception named in the *except
clause*, it is passed on to outer try statements; if no handler is
found, it is an *unhandled exception* and execution stops with an error message.
A try statement may have more than one *except clause*, to specify
handlers for different exceptions. At most one handler will be executed.
Handlers only handle exceptions that occur in the corresponding *try clause*,
not in other handlers of the same try statement. An *except clause*
may name multiple exceptions, for example
A class in an except clause matches exceptions which are instances of the
class itself or one of its derived classes (but not the other way around --- an
except clause listing a derived class does not match instances of its base classes).
For example, the following code will print B, C, D in that order
Note that if the *except clauses* were reversed (with ``except B`` firs

Contoh Kode / Perintah CLI:

```
>>> while True:
...     try:
...         x = int(input("Please enter a number: "))
...         break
...     except ValueError:
...         print("Oops!  That was no valid number.  Try again...")
```

```
...

# ----------------------------------------

class B(Exception):
pass
class C(B):
pass
class D(C):
pass
for cls in [B, C, D]:
try:
raise cls()
except D:
print("D")
except C:
print("C")
except B:
print("B")

# ----------------------------------------

>>> try:
...     raise Exception('spam', 'eggs')
... except Exception as inst:
...     print(type(inst))    # the exception type
...     print(inst.args)     # arguments stored in .args
...     print(inst)          # __str__ allows args to be printed directly,
...                          # but may be overridden in exception subclasses
...     x, y = inst.args     # unpack args
...     print('x =', x)
...     print('y =', y)
...
<class 'Exception'>
('spam', 'eggs')
('spam', 'eggs')
x = spam
y = eggs
```

[python_069] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Raising Exceptions

Sumber: ht

The raise statement allows the programmer to force a specified
exception to occur. For example
The sole argument to raise indicates the exception to be raised.
This must be either an exception instance or an exception class (a class that
derives from BaseException, such as Exception or one of its
subclasses). If an exception class is passed, it will be implicitly
instantiated by calling its constructor with no arguments
If you need to determine whether an exception was raised but don't intend to
handle it, a simpler form of the raise statement allows you to
re-raise the exception

Contoh Kode / Perintah CLI:

```
>>> raise NameError('HiThere')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
raise NameError('HiThere')
NameError: HiThere
```

```
# ----------------------------------------

>>> try:
...     raise NameError('HiThere')
... except NameError:
...     print('An exception flew by!')
...     raise
...
An exception flew by!
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
raise NameError('HiThere')
NameError: HiThere
```

[python_070] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Exception Chaining

Sumber: ht

If an unhandled exception occurs inside an except section, it will
have the exception being handled attached to it and included in the error
message
To indicate that an exception is a direct consequence of another, the
raise statement allows an optional raise clause
This can be useful when you are transforming exceptions. For example
It also allows disabling automatic exception chaining using the ``from None``
idiom
For more information about chaining mechanics, see bltin-exceptions.

Contoh Kode / Perintah CLI:

```
>>> try:
...     open("database.sqlite")
... except OSError:
...     raise RuntimeError("unable to handle error")
...
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
open("database.sqlite")
~~~~^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'database.sqlite'
<BLANKLINE>
During handling of the above exception, another exception occurred:
<BLANKLINE>
Traceback (most recent call last):
File "<stdin>", line 4, in <module>
raise RuntimeError("unable to handle error")
RuntimeError: unable to handle error

# ----------------------------------------

>>> def func():
...     raise ConnectionError
...
>>> try:
...     func()
... except ConnectionError as exc:
...     raise RuntimeError('Failed to open database') from exc
...
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
```

```
func()
~~~~^^
File "<stdin>", line 2, in func
ConnectionError
<BLANKLINE>
The above exception was the direct cause of the following exception:
<BLANKLINE>
Traceback (most recent call last):
File "<stdin>", line 4, in <module>
raise RuntimeError('Failed to open database') from exc
RuntimeError: Failed to open database

# ----------------------------------------

>>> try:
...     open('database.sqlite')
... except OSError:
...     raise RuntimeError from None
...
Traceback (most recent call last):
File "<stdin>", line 4, in <module>
raise RuntimeError from None
RuntimeError
```

[python_071] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): User-defined Exceptions

Sumber: ht

Programs may name their own exceptions by creating a new exception class (see
tut-classes for more about Python classes). Exceptions should typically
be derived from the Exception class, either directly or indirectly.
Exception classes can be defined which do anything any other class can do, but
are usually kept simple, often only offering a number of attributes that allow
information about the error to be extracted by handlers for the exception.
Most exceptions are defined with names that end in "Error", similar to the
naming of the standard exceptions.
Many standard modules define their own exceptions to report errors that may
occur in functions they define.

[python_072] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Defining Clean-up Actions

Sumber: ht

The try statement has another optional clause which is intended to
define clean-up actions that must be executed under all circumstances. For
example
If a finally clause is present, the finally
clause will execute as the last task before the try
statement completes. The finally clause runs whether or
not the try statement produces an exception. The following
points discuss more complex cases when an exception occurs:
* If an exception occurs during execution of the try
clause, the exception may be handled by an except
clause. If the exception is not handled by an except
clause, the exception is re-raised after the finally
clause has been executed.

* An exception could occur during execution of an except
or else clause. Again, the exception is re-raised after
the finally clause has been executed.
* If the finally clause executes a break,
continue or return statement, exceptions are not
re-raised. This can be confusing and is therefore discouraged. From
version 3.14 the compiler emits a SyntaxWarning for it
(see 765).
* If the try statement reaches a break,
continue or return statement, the
finally clause will execute just prior to the
break, continue or return
statement's execution.
* If a finally clause includes a return
statement, the returned value will be the one from the
finally clause's return statement, not the
value from the try clause's return
statement. This can be confusing and is therefore discouraged. From
version 3.14 the compiler emits a SyntaxWarning for it
(see 765).
For example
A more complicated example
As you can see, the finally clause is executed in any event. The
TypeError raised by dividing two strings is not handled by the
except clause and therefore re-raised after the finally
clause has been executed.
In real world applications, the finally c

Contoh Kode / Perintah CLI:

```
>>> try:
...     raise KeyboardInterrupt
... finally:
...     print('Goodbye, world!')
...
Goodbye, world!
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
raise KeyboardInterrupt
KeyboardInterrupt

# -----------------------------------------

>>> def bool_return():
...     try:
...         return True
...     finally:
...         return False
...
>>> bool_return()
False

# -----------------------------------------

>>> def divide(x, y):
...     try:
...         result = x / y
...     except ZeroDivisionError:
```

```
...         print("division by zero!")
...     else:
...         print("result is", result)
...     finally:
...         print("executing finally clause")
...
>>> divide(2, 1)
result is 2.0
executing finally clause
>>> divide(2, 0)
division by zero!
executing finally clause
>>> divide("2", "1")
executing finally clause
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
divide("2", "1")
~~~~~~^^^^^^^^^^
File "<stdin>", line 3, in divide
result = x / y
~~^~~
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

[python_073] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Predefined Clean-up Actions

Sumber: ht

Some objects define standard clean-up actions to be undertaken when the object is no longer needed, regardless of whether or not the operation using the object succeeded or failed. Look at the following example, which tries to open a file and print its contents to the screen.
The problem with this code is that it leaves the file open for an indeterminate amount of time after this part of the code has finished executing.
This is not an issue in simple scripts, but can be a problem for larger applications. The with statement allows objects like files to be used in a way that ensures they are always cleaned up promptly and correctly.
After the statement is executed, the file *f* is always closed, even if a problem was encountered while processing the lines. Objects which, like files, provide predefined clean-up actions will indicate this in their documentation.

Contoh Kode / Perintah CLI:

```
for line in open("myfile.txt"):
print(line, end="")

# ----------------------------------------

with open("myfile.txt") as f:
for line in f:
print(line, end="")
```

[python_074] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Raising and Handling Multiple Unrelated Exceptions

Sumber: ht

There are situations where it is necessary to report several exceptions that have occurred. This is often the case in concurrency frameworks, when several tasks may have failed in parallel, but there are also other use cases where it is desirable to continue execution and collect multiple errors rather than raise the first exception.
The builtin ExceptionGroup wraps a list of exception instances so that they can be raised together. It is an exception itself, so it can be caught like any other exception.
By using ``except*`` instead of ``except``, we can selectively handle only the exceptions in the group that match a certain type. In the following example, which shows a nested exception group, each ``except*`` clause extracts from the group exceptions of a certain type while letting all other exceptions propagate to other clauses and eventually to be reraised.
Note that the exceptions nested in an exception group must be instances, not types. This is because in practice the exceptions would typically be ones that have already been raised and caught by the program, along the following pattern

Contoh Kode / Perintah CLI:

```
>>> def f():
...     excs = [OSError('error 1'), SystemError('error 2')]
...     raise ExceptionGroup('there were problems', excs)
...
>>> f()
+ Exception Group Traceback (most recent call last):
|   File "<stdin>", line 1, in <module>
|     f()
|     ~^^
|   File "<stdin>", line 3, in f
|     raise ExceptionGroup('there were problems', excs)
| ExceptionGroup: there were problems (2 sub-exceptions)
+-+---------------- 1 ----------------
| OSError: error 1
+---------------- 2 ----------------
| SystemError: error 2
+------------------------------------
>>> try:
...     f()
... except Exception as e:
...     print(f'caught {type(e)}: {e}')
...
caught <class 'ExceptionGroup'>: there were problems (2 sub-exceptions)
>>>

# ----------------------------------------

>>> def f():
...     raise ExceptionGroup(
...         "group1",
...         [
...             OSError(1),
...             SystemError(2),
```

```
...             ExceptionGroup(
...                 "group2",
...                 [
...                     OSError(3),
...                     RecursionError(4)
...                 ]
...             )
...         ]
...     )
...
>>> try:
...     f()
... except* OSError as e:
...     print("There were OSErrors")
... except* SystemError as e:
...     print("There were SystemErrors")
...
There were OSErrors
There were SystemErrors
+ Exception Group Traceback (most recent call last):
|   File "<stdin>", line 2, in <module>
|     f()
|     ~^^
|   File "<stdin>", line 2, in f
|     raise ExceptionGroup(
|     ...<12 lines>...
|     )
| ExceptionGroup: group1 (1 sub-exception)
+-+---------------- 1 ----------------
| ExceptionGroup: group2 (1 sub-exception)
+-+---------------- 1 ----------------
| RecursionError: 4
+------------------------------------
>>>

# ----------------------------------------

>>> excs = []
>>> for test in tests:
...     try:
...         test.run()
...     except Exception as e:
...         excs.append(e)
...
>>> if excs:
...    raise ExceptionGroup("Test Failures", excs)
...
```

[python_075] Python - Error Handling & Exception Groups

Errors and Exceptions (try-except, ExceptionGroup, except*): Enriching Exceptions with Notes

Sumber: ht

When an exception is created in order to be raised, it is usually initialized with information that describes the error that has occurred. There are cases where it is useful to add information after the exception was caught. For this purpose, exceptions have a method ``add_note(note)`` that accepts a string and adds it to the exception's notes list. The standard traceback rendering includes all notes, in the order they were added, after the exception.

For example, when collecting exceptions into an exception group, we may want to add context information for the individual errors. In the following each exception in the group has a note indicating when this error has occurred.
>>>

Contoh Kode / Perintah CLI:

```
>>> try:
...     raise TypeError('bad type')
... except Exception as e:
...     e.add_note('Add some information')
...     e.add_note('Add some more information')
...     raise
...
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
raise TypeError('bad type')
TypeError: bad type
Add some information
Add some more information
>>>

# ----------------------------------------

>>> def f():
...     raise OSError('operation failed')
...
>>> excs = []
>>> for i in range(3):
...     try:
...         f()
...     except Exception as e:
...         e.add_note(f'Happened in Iteration {i+1}')
...         excs.append(e)
...
>>> raise ExceptionGroup('We have some problems', excs)
+ Exception Group Traceback (most recent call last):
|   File "<stdin>", line 1, in <module>
|     raise ExceptionGroup('We have some problems', excs)
| ExceptionGroup: We have some problems (3 sub-exceptions)
+-+---------------- 1 ----------------
| Traceback (most recent call last):
|   File "<stdin>", line 3, in <module>
|     f()
|     ~^^
|   File "<stdin>", line 2, in f
|     raise OSError('operation failed')
| OSError: operation failed
| Happened in Iteration 1
+---------------- 2 ----------------
| Traceback (most recent call last):
|   File "<stdin>", line 3, in <module>
|     f()
|     ~^^
|   File "<stdin>", line 2, in f
|     raise OSError('operation failed')
| OSError: operation failed
| Happened in Iteration 2
+---------------- 3 ----------------
| Traceback (most recent call last):
|   File "<stdin>", line 3, in <module>
|     f()
```

```
|     ~^^
|   File "<stdin>", line 2, in f
|     raise OSError('operation failed')
| OSError: operation failed
| Happened in Iteration 3
+-----------------------------------
```

[python_076] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Overview

Sumber: ht

Classes
Classes provide a means of bundling data and functionality together. Creating a new class creates a new *type* of object, allowing new *instances* of that type to be made. Each class instance can have attributes attached to it for maintaining its state. Class instances can also have methods (defined by its class) for modifying its state.
Compared with other programming languages, Python's class mechanism adds classes with a minimum of new syntax and semantics. It is a mixture of the class mechanisms found in C++ and Modula-3. Python classes provide all the standard features of Object Oriented Programming: the class inheritance mechanism allows multiple base classes, a derived class can override any methods of its base class or classes, and a method can call the method of a base class with the same name. Objects can contain arbitrary amounts and kinds of data. As is true for modules, classes partake of the dynamic nature of Python: they are created at runtime, and can be modified further after creation.
In C++ terminology, normally class members (including the data members) are *public* (except see below tut-private), and all member functions are *virtual*. As in Modula-3, there are no shorthands for referencing the object's members from its methods: the method function is declared with an explicit first argument representing the object, which is provided implicitly by the call. As in Smalltalk, classes themselves are objects. This provides semantics for importing and renaming. Unlike C++ and Modula-3, built-in types can be used as base classes for extension by the user. Also, like in C++, most built-in operators with special syntax (arithmetic operators, subscripting etc.) can be redefined for class instances.
(Lacking universally accepted terminolog

[python_077] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: A Word About Names and Objects

Sumber: ht

Objects have individuality, and multiple names (in multiple scopes) can be bound to the same object. This is known as aliasing in other languages. This is usually not appreciated on a first glance at Python, and can be safely ignored when dealing with immutable basic types (numbers, strings, tuples). However, aliasing has a possibly surprising effect on the semantics of Python code involving mutable objects such as lists, dictionaries, and most other types. This is usually used to the benefit of the program, since aliases behave like pointers in some respects. For example, passing an object is cheap since only a pointer is passed by the implementation; and if a function modifies an object passed as an argument, the caller will see the change --- this eliminates the need for two different argument passing mechanisms as in Pascal.

[python_078] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Python Scopes and Namespaces

Sumber: ht

Before introducing classes, I first have to tell you something about Python's
scope rules. Class definitions play some neat tricks with namespaces, and you
need to know how scopes and namespaces work to fully understand what's going on.
Incidentally, knowledge about this subject is useful for any advanced Python
programmer.
Let's begin with some definitions.
A *namespace* is a mapping from names to objects. Most namespaces are currently
implemented as Python dictionaries, but that's normally not noticeable in any
way (except for performance), and it may change in the future. Examples of
namespaces are: the set of built-in names (containing functions such as abs, and
built-in exception names); the global names in a module; and the local names in
a function invocation. In a sense the set of attributes of an object also form
a namespace. The important thing to know about namespaces is that there is
absolutely no relation between names in different namespaces; for instance, two
different modules may both define a function ``maximize`` without confusion ---
users of the modules must prefix it with the module name.
By the way, I use the word *attribute* for any name following a dot --- for
example, in the expression ``z.real``, ``real`` is an attribute of the object
``z``. Strictly speaking, references to names in modules are attribute
references: in the expression ``modname.funcname``, ``modname`` is a module
object and ``funcname`` is an attribute of it. In this case there happens to be
a straightforward mapping between the module's attributes and the global names
defined in the module: they share the same namespace! [#]_
Attributes may be read-only or writable. In the latter case, assignment to
attributes is possible. Module attributes are writable: you can write

[python_079] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Scopes and Namespaces Example

Sumber: ht

This is an example demonstrating how to reference the different scopes and
namespaces, and how global and nonlocal affect variable
binding
The output of the example code is:
After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam
Note how the *local* assignment (which is default) didn't change *scope_test*\'s
binding of *spam*. The nonlocal assignment changed *scope_test*\'s
binding of *spam*, and the global assignment changed the module-level
binding.
You can also see that there was no previous binding for *spam* before the
global assignment.

Contoh Kode / Perintah CLI:

```
def scope_test():
def do_local():
spam = "local spam"
def do_nonlocal():
```

```
nonlocal spam
spam = "nonlocal spam"
def do_global():
global spam
spam = "global spam"
spam = "test spam"
do_local()
print("After local assignment:", spam)
do_nonlocal()
print("After nonlocal assignment:", spam)
do_global()
print("After global assignment:", spam)
scope_test()
print("In global scope:", spam)
```

[python_080] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: A First Look at Classes

Sumber: ht

Classes introduce a little bit of new syntax, three new object types, and some new semantics.

[python_081] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Class Definition Syntax

Sumber: ht

The simplest form of class definition looks like this
Class definitions, like function definitions (def statements) must be executed before they have any effect. (You could conceivably place a class definition in a branch of an if statement, or inside a function.)
In practice, the statements inside a class definition will usually be function definitions, but other statements are allowed, and sometimes useful --- we'll come back to this later. The function definitions inside a class normally have a peculiar form of argument list, dictated by the calling conventions for methods --- again, this is explained later.
When a class definition is entered, a new namespace is created, and used as the local scope --- thus, all assignments to local variables go into this new namespace. In particular, function definitions bind the name of the new function here.
When a class definition is left normally (via the end), a *class object* is created. This is basically a wrapper around the contents of the namespace created by the class definition; we'll learn more about class objects in the next section. The original local scope (the one in effect just before the class definition was entered) is reinstated, and the class object is bound here to the class name given in the class definition header (ClassName in the example).

Contoh Kode / Perintah CLI:

```
class ClassName:
<statement-1>
.
.
.
<statement-N>
```

[python_082] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Class Objects

Sumber: ht

Class objects support two kinds of operations: attribute references and instantiation.
Attribute references use the standard syntax used for all attribute references in Python: ``obj.name``. Valid attribute names are all the names that were in the class's namespace when the class object was created. So, if the class definition looked like this
then ``MyClass.i`` and ``MyClass.f`` are valid attribute references, returning an integer and a function object, respectively. Class attributes can also be assigned to, so you can change the value of ``MyClass.i`` by assignment. ~type.__doc__ is also a valid attribute, returning the docstring belonging to the class: ``"A simple example class"``.
Class *instantiation* uses function notation. Just pretend that the class object is a parameterless function that returns a new instance of the class. For example (assuming the above class)
creates a new *instance* of the class and assigns this object to the local variable ``x``.
The instantiation operation ("calling" a class object) creates an empty object. Many classes like to create objects with instances customized to a specific initial state. Therefore a class may define a special method named ~object.__init__, like this
When a class defines an ~object.__init__ method, class instantiation automatically invokes __init__ for the newly created class instance. So in this example, a new, initialized instance can be obtained by
Of course, the ~object.__init__ method may have arguments for greater flexibility. In that case, arguments given to the class instantiation operator are passed on to __init__. For example,

Contoh Kode / Perintah CLI:

```
class MyClass:
"""A simple example class"""
i = 12345
def f(self):
return 'hello world'

# -----------------------------------------

x = MyClass()

# -----------------------------------------

def __init__(self):
self.data = []
```

[python_083] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Instance Objects

Sumber: ht

Now what can we do with instance objects? The only operations understood by instance objects are attribute references. There are two kinds of valid attribute names: data attributes and methods.
Data attributes correspond to "instance variables" in Smalltalk, and to "data members" in C++. Data attributes need not be declared; like local variables,

they spring into existence when they are first assigned to. For example, if
``x`` is the instance of MyClass created above, the following piece of
code will print the value ``16``, without leaving a trace
The other kind of instance attribute reference is a *method*. A method is a
function that "belongs to" an object.
Valid method names of an instance object depend on its class. By definition,
all attributes of a class that are function objects define corresponding
methods of its instances. So in our example, ``x.f`` is a valid method
reference, since ``MyClass.f`` is a function, but ``x.i`` is not, since
``MyClass.i`` is not. But ``x.f`` is not the same thing as ``MyClass.f`` --- it
is a *method object*, not a function object.

Contoh Kode / Perintah CLI:

```
x.counter = 1
while x.counter < 10:
x.counter = x.counter * 2
print(x.counter)
del x.counter
```

[python_084] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Method Objects

Sumber: ht

Usually, a method is called right after it is bound
If ``x = MyClass()``, as above, this will return the string ``'hello world'``.
However, it is not necessary to call a method right away: ``x.f`` is a method
object, and can be stored away and called at a later time. For example
will continue to print ``hello world`` until the end of time.
What exactly happens when a method is called? You may have noticed that
``x.f()`` was called without an argument above, even though the function
definition for f specified an argument. What happened to the argument?
Surely Python raises an exception when a function that requires an argument is
called without any --- even if the argument isn't actually used...
Actually, you may have guessed the answer: the special thing about methods is
that the instance object is passed as the first argument of the function. In our
example, the call ``x.f()`` is exactly equivalent to ``MyClass.f(x)``. In
general, calling a method with a list of *n* arguments is equivalent to calling
the corresponding function with an argument list that is created by inserting
the method's instance object before the first argument.
In general, methods work as follows. When a non-data attribute
of an instance is referenced, the instance's class is searched.
If the name denotes a valid class attribute that is a function object,
references to both the instance object and the function object
are packed into a method object. When the method object is called
with an argument list, a new argument list is constructed from the instance
object and the argument list, and the function object is called with this new
argument list.

Contoh Kode / Perintah CLI:

```
xf = x.f
while True:
print(xf())
```

[python_085] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Class and Instance Variables

Sumber: ht

Generally speaking, instance variables are for data unique to each instance and class variables are for attributes and methods shared by all instances of the class
As discussed in tut-object, shared data can have possibly surprising effects involving mutable objects such as lists and dictionaries.
For example, the *tricks* list in the following code should not be used as a class variable because just a single list would be shared by all *Dog* instances
Correct design of the class should use an instance variable instead

Contoh Kode / Perintah CLI:

```
class Dog:
kind = 'canine'         # class variable shared by all instances
def __init__(self, name):
self.name = name    # instance variable unique to each instance
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.kind                  # shared by all dogs
'canine'
>>> e.kind                  # shared by all dogs
'canine'
>>> d.name                  # unique to d
'Fido'
>>> e.name                  # unique to e
'Buddy'

# -----------------------------------------

class Dog:
tricks = []             # mistaken use of a class variable
def __init__(self, name):
self.name = name
def add_trick(self, trick):
self.tricks.append(trick)
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks                # unexpectedly shared by all dogs
['roll over', 'play dead']

# -----------------------------------------

class Dog:
def __init__(self, name):
self.name = name
self.tricks = []    # creates a new empty list for each dog
def add_trick(self, trick):
self.tricks.append(trick)
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> e.tricks
```

```
['play dead']
```

[python_086] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Random Remarks

Sumber: ht

.. These should perhaps be placed more carefully...
If the same attribute name occurs in both an instance and in a class,
then attribute lookup prioritizes the instance
Data attributes may be referenced by methods as well as by ordinary users
("clients") of an object. In other words, classes are not usable to implement
pure abstract data types. In fact, nothing in Python makes it possible to
enforce data hiding --- it is all based upon convention. (On the other hand,
the Python implementation, written in C, can completely hide implementation
details and control access to an object if necessary; this can be used by
extensions to Python written in C.)
Clients should use data attributes with care --- clients may mess up invariants
maintained by the methods by stamping on their data attributes. Note that
clients may add data attributes of their own to an instance object without
affecting the validity of the methods, as long as name conflicts are avoided ---
again, a naming convention can save a lot of headaches here.
There is no shorthand for referencing data attributes (or other methods!) from
within methods. I find that this actually increases the readability of methods:
there is no chance of confusing local variables and instance variables when
glancing through a method.
Often, the first argument of a method is called ``self``. This is nothing more
than a convention: the name ``self`` has absolutely no special meaning to
Python. Note, however, that by not following the convention your code may be
less readable to other Python programmers, and it is also conceivable that a
class browser program might be written that relies upon such a convention.
Any function object that is a class attribute defines a method for instances of
that class. It is not necessary that

Contoh Kode / Perintah CLI:

```
>>> class Warehouse:
...     purpose = 'storage'
...     region = 'west'
...
>>> w1 = Warehouse()
>>> print(w1.purpose, w1.region)
storage west
>>> w2 = Warehouse()
>>> w2.region = 'east'
>>> print(w2.purpose, w2.region)
storage east

# --------------------------------------

# Function defined outside the class
def f1(self, x, y):
return min(x, x+y)
class C:
f = f1
def g(self):
return 'hello world'
h = g
```

```
# ----------------------------------------

class Bag:
def __init__(self):
self.data = []
def add(self, x):
self.data.append(x)
def addtwice(self, x):
self.add(x)
self.add(x)
```

[python_087] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Inheritance

Sumber: ht

Of course, a language feature would not be worthy of the name "class" without supporting inheritance. The syntax for a derived class definition looks like this

The name BaseClassName must be defined in a namespace accessible from the scope containing the derived class definition. In place of a base class name, other arbitrary expressions are also allowed. This can be useful, for example, when the base class is defined in another module

Execution of a derived class definition proceeds the same as for a base class. When the class object is constructed, the base class is remembered. This is used for resolving attribute references: if a requested attribute is not found in the class, the search proceeds to look in the base class. This rule is applied recursively if the base class itself is derived from some other class.

There's nothing special about instantiation of derived classes: ``DerivedClassName()`` creates a new instance of the class. Method references are resolved as follows: the corresponding class attribute is searched, descending down the chain of base classes if necessary, and the method reference is valid if this yields a function object.

Derived classes may override methods of their base classes. Because methods have no special privileges when calling other methods of the same object, a method of a base class that calls another method defined in the same base class may end up calling a method of a derived class that overrides it. (For C++ programmers: all methods in Python are effectively ``virtual``.)

An overriding method in a derived class may in fact want to extend rather than simply replace the base class method of the same name. There is a simple way to call the base class method directly: just call ``BaseClassName.methodname(self, arguments)``. T

Contoh Kode / Perintah CLI:

```
class DerivedClassName(BaseClassName):
<statement-1>
.
.
.
<statement-N>

# ----------------------------------------

class DerivedClassName(modname.BaseClassName):
```

[python_088] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Multiple Inheritance

Sumber: ht

Python supports a form of multiple inheritance as well. A class definition with
multiple base classes looks like this
For most purposes, in the simplest cases, you can think of the search for
attributes inherited from a parent class as depth-first, left-to-right, not
searching twice in the same class where there is an overlap in the hierarchy.
Thus, if an attribute is not found in DerivedClassName, it is searched
for in Base1, then (recursively) in the base classes of Base1,
and if it was not found there, it was searched for in Base2, and so on.
In fact, it is slightly more complex than that; the method resolution order
changes dynamically to support cooperative calls to super. This
approach is known in some other multiple-inheritance languages as
call-next-method and is more powerful than the super call found in
single-inheritance languages.
Dynamic ordering is necessary because all cases of multiple inheritance exhibit
one or more diamond relationships (where at least one of the parent classes
can be accessed through multiple paths from the bottommost class). For example,
all classes inherit from object, so any case of multiple inheritance
provides more than one path to reach object. To keep the base classes
from being accessed more than once, the dynamic algorithm linearizes the search
order in a way that preserves the left-to-right ordering specified in each
class, that calls each parent only once, and that is monotonic (meaning that a
class can be subclassed without affecting the precedence order of its parents).
Taken together, these properties make it possible to design reliable and
extensible classes with multiple inheritance. For more detail, see
python_2.3_mro.
In some cases multiple inheritance is not allowed; see multiple-inheritance
for details.

Contoh Kode / Perintah CLI:

```
class DerivedClassName(Base1, Base2, Base3):
<statement-1>
.
.
.
<statement-N>
```

[python_089] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Private Variables

Sumber: ht

"Private" instance variables that cannot be accessed except from inside an
object don't exist in Python. However, there is a convention that is followed
by most Python code: a name prefixed with an underscore (e.g. ``_spam``) should
be treated as a non-public part of the API (whether it is a function, a method
or a data member). It should be considered an implementation detail and subject
to change without notice.
pair: name; mangling
Since there is a valid use-case for class-private members (namely to avoid name
clashes of names with names defined by subclasses), there is limited support for
such a mechanism, called name mangling. Any identifier of the form

``__spam`` (at least two leading underscores, at most one trailing underscore)
is textually replaced with ``_classname__spam``, where ``classname`` is the
current class name with leading underscore(s) stripped. This mangling is done
without regard to the syntactic position of the identifier, as long as it
occurs within the definition of a class.
.. seealso
Name mangling is helpful for letting subclasses override methods without
breaking intraclass method calls. For example
The above example would work even if ``MappingSubclass`` were to introduce a
``__update`` identifier since it is replaced with ``_Mapping__update`` in the
``Mapping`` class and ``_MappingSubclass__update`` in the ``MappingSubclass``
class respectively.
Note that the mangling rules are designed mostly to avoid accidents; it still is
possible to access or modify a variable that is considered private. This can
even be useful in special circumstances, such as in the debugger.
Notice that code passed to ``exec()`` or ``eval()`` does not consider the
classname of the invoking class to be the current class; this is similar to the
effect of the ``g

Contoh Kode / Perintah CLI:

```
class Mapping:
def __init__(self, iterable):
self.items_list = []
self.__update(iterable)
def update(self, iterable):
for item in iterable:
self.items_list.append(item)
__update = update   # private copy of original update() method
class MappingSubclass(Mapping):
def update(self, keys, values):
# provides new signature for update()
# but does not break __init__()
for item in zip(keys, values):
self.items_list.append(item)
```

[python_090] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Odds and Ends

Sumber: ht

Sometimes it is useful to have a data type similar to the Pascal "record" or C
"struct", bundling together a few named data items. The idiomatic approach
is to use dataclasses for this purpose
A piece of Python code that expects a particular abstract data type can often be
passed a class that emulates the methods of that data type instead. For
instance, if you have a function that formats some data from a file object, you
can define a class with methods ~io.TextIOBase.read and
~io.TextIOBase.readline that get the
data from a string buffer instead, and pass it as an argument.
.. (Unfortunately, this technique has its limitations: a class can't define
operations that are accessed by special syntax such as sequence subscripting
or arithmetic operators, and assigning such a "pseudo-file" to sys.stdin will
not cause the interpreter to read further input from it.)
instance-methods have attributes, too:
method.__self__ is the instance
object with the method m, and method.__func__ is

the user-defined-funcs
corresponding to the method.

Contoh Kode / Perintah CLI:

```
from dataclasses import dataclass
@dataclass
class Employee:
name: str
dept: str
salary: int

# ----------------------------------------

>>> john = Employee('john', 'computer lab', 1000)
>>> john.dept
'computer lab'
>>> john.salary
1000
```

[python_091] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Iterators

Sumber: ht

By now you have probably noticed that most container objects can be looped over
using a for statement
This style of access is clear, concise, and convenient. The use of iterators
pervades and unifies Python. Behind the scenes, the for statement
calls iter on the container object. The function returns an iterator
object that defines the method ~iterator.__next__ which accesses
elements in the container one at a time. When there are no more elements,
~iterator.__next__ raises a StopIteration exception which tells the
for loop to terminate. You can call the ~iterator.__next__ method
using the next built-in function; this example shows how it all works
Having seen the mechanics behind the iterator protocol, it is easy to add
iterator behavior to your classes. Define an ~container.__iter__ method which
returns an object with a ~iterator.__next__ method. If the class
defines __next__, then __iter__ can just return ``self``

Contoh Kode / Perintah CLI:

```
for element in [1, 2, 3]:
print(element)
for element in (1, 2, 3):
print(element)
for key in {'one':1, 'two':2}:
print(key)
for char in "123":
print(char)
for line in open("myfile.txt"):
print(line, end='')

# ----------------------------------------

>>> s = 'abc'
>>> it = iter(s)
>>> it
<str_iterator object at 0x10c90e650>
>>> next(it)
'a'
>>> next(it)
```

```
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
next(it)
StopIteration

# --------------------------------------

class Reverse:
"""Iterator for looping over a sequence backwards."""
def __init__(self, data):
self.data = data
self.index = len(data)
def __iter__(self):
return self
def __next__(self):
if self.index == 0:
raise StopIteration
self.index = self.index - 1
return self.data[self.index]
```

[python_092] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Generators

Sumber: ht

generator are a simple and powerful tool for creating iterators. They are written like regular functions but use the yield statement whenever they want to return data. Each time next is called on it, the generator resumes where it left off (it remembers all the data values and which statement was last executed). An example shows that generators can be trivially easy to create
Anything that can be done with generators can also be done with class-based iterators as described in the previous section. What makes generators so compact is that the ~iterator.__iter__ and ~generator.__next__ methods are created automatically.
Another key feature is that the local variables and execution state are automatically saved between calls. This made the function easier to write and much more clear than an approach using instance variables like ``self.index`` and ``self.data``.
In addition to automatic method creation and saving program state, when generators terminate, they automatically raise StopIteration. In combination, these features make it easy to create iterators with no more effort than writing a regular function.

Contoh Kode / Perintah CLI:

```
def reverse(data):
for index in range(len(data)-1, -1, -1):
yield data[index]

# --------------------------------------

>>> for char in reverse('golf'):
...     print(char)
...
f
```

```
l
o
g
```

[python_093] Python - Object-Oriented Programming

Classes, Inheritance, Iterators & Generators: Generator Expressions

Sumber: ht

Some simple generators can be coded succinctly as expressions using a syntax similar to list comprehensions but with parentheses instead of square brackets. These expressions are designed for situations where the generator is used right away by an enclosing function. Generator expressions are more compact but less versatile than full generator definitions and tend to be more memory friendly than equivalent list comprehensions.
Examples
In most cases, generator expressions must be wrapped in parentheses. As a special case, however, when provided as the sole argument to a function (as in the examples involving ``sum``, ``set``, ``max``, and ``list`` above), the generator expression does not need to be wrapped in an additional set of parentheses. That is to say, the following two pieces of code are semantically equivalent
as are the following
.. [#] Except for one thing. Module objects have a secret read-only attribute called ~object.__dict__ which returns the dictionary used to implement the module's namespace; the name ``__dict__`` is an attribute but not a global name. Obviously, using this violates the abstraction of namespace implementation, and should be restricted to things like post-mortem debuggers.

Contoh Kode / Perintah CLI:

```
>>> sum(i*i for i in range(10))                 # sum of squares
285
>>> xvec = [10, 20, 30]
>>> yvec = [7, 5, 3]
>>> sum(x*y for x,y in zip(xvec, yvec))         # dot product
260
>>> unique_words = set(word for line in page  for word in line.split())
>>> valedictorian = max((student.gpa, student.name) for student in graduates)
>>> data = 'golf'
>>> list(data[i] for i in range(len(data)-1, -1, -1))
['f', 'l', 'o', 'g']
>>> x = [[1,2,3], [], [4, 5]]
>>> g = (*i for i in x)
>>> list(g)
[1, 2, 3, 4, 5]

# ----------------------------------------

>>> f(x for x in y)
>>> f((x for x in y))

# ----------------------------------------

>>> f(*x for x in y)
>>> f((*x for x in y))
```

[python_094] Python - Environment & Package Management

Virtual Environments and Packages (venv & pip): Introduction

Sumber: ht

Python applications will often use packages and modules that don't
come as part of the standard library. Applications will sometimes
need a specific version of a library, because the application may
require that a particular bug has been fixed or the application may be
written using an obsolete version of the library's interface.
This means it may not be possible for one Python installation to meet
the requirements of every application. If application A needs version
1.0 of a particular module but application B needs version 2.0, then
the requirements are in conflict and installing either version 1.0 or 2.0
will leave one application unable to run.
The solution for this problem is to create a virtual environment, a
self-contained directory tree that contains a Python installation for a
particular version of Python, plus a number of additional packages.
Different applications can then use different virtual environments.
To resolve the earlier example of conflicting requirements,
application A can have its own virtual environment with version 1.0
installed while application B has another virtual environment with version 2.0.
If application B requires a library be upgraded to version 3.0, this will
not affect application A's environment.

[python_095] Python - Environment & Package Management

Virtual Environments and Packages (venv & pip): Creating Virtual Environments

Sumber: ht

The module used to create and manage virtual environments is called
venv. venv will install the Python version from which
the command was run (as reported by the --version option).
For instance, executing the command with ``python3.12`` will install
version 3.12.
To create a virtual environment, decide upon a directory where you want to
place it, and run the venv module as a script with the directory path
This will create the ``tutorial-env`` directory if it doesn't exist,
and also create directories inside it containing a copy of the Python
interpreter and various supporting files.
A common directory location for a virtual environment is ``.venv``.
This name keeps the directory typically hidden in your shell and thus
out of the way while giving it a name that explains why the directory
exists. It also prevents clashing with ``.env`` environment variable
definition files that some tooling supports.
Once you've created a virtual environment, you may activate it.
On Windows, run::
tutorial-env\Scripts\activate
On Unix or MacOS, run::
source tutorial-env/bin/activate
(This script is written for the bash shell. If you use the
csh or fish shells, there are alternate
``activate.csh`` and ``activate.fish`` scripts you should use
instead.)
Activating the virtual environment will change your shell's prompt to show what

virtual environment you're using, and modify the environment so that running
``python`` will get you that particular version and installation of Python.
For example:
$ source ~/envs/tutorial-env/bin/activate
(tutorial-env) $ python
Python 3.5.1 (default, May 6 2016, 10:59:36)
...
>>> import sys
>>> sys.path
['', '/usr/local/lib/python35.zip', ...,
'~/envs/tutorial-env/lib/python3.5/site-packages']
>>>
Note that the activated virtual environment does not alter t

[python_096] Python - Environment & Package Management

Virtual Environments and Packages (venv & pip): Managing Packages with pip

Sumber: ht

You can install, upgrade, and remove packages using a program called
pip. By default ``pip`` will install packages from the `Python
Package Index <https://pypi.org>`_. You can browse the Python
Package Index by going to it in your web browser.
``pip`` has a number of subcommands: "install", "uninstall",
"freeze", etc. (Consult the installing-index guide for
complete documentation for ``pip``.)
You can install the latest version of a package by specifying a package's name:
(tutorial-env) $ python -m pip install novas
Collecting novas
Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
Running setup.py install for novas
Successfully installed novas-3.1.1.3
You can also install a specific version of a package by giving the
package name followed by ``==`` and the version number:
(tutorial-env) $ python -m pip install requests==2.6.0
Collecting requests==2.6.0
Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0
If you re-run this command, ``pip`` will notice that the requested
version is already installed and do nothing. You can supply a
different version number to get that version, or you can run ``python
-m pip install --upgrade`` to upgrade the package to the latest version:
(tutorial-env) $ python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
Found existing installation: requests 2.6.0
Uninstalling requests-2.6.0:
Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0
``python -m pip uninstall`` followed by one or more package names will
remove the packages from the virtual environment.

``python -m pip show`` will display information about a particular package:

[python_097] Python - Environment & Package Management

Virtual Environments and Packages (venv & pip): (tutorial-env) $ python -m pip show requests

Sumber: ht

Metadata-Version: 2.0
Name: requests
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages
Requires:
``python -m pip list`` will display all of the packages installed in
the virtual environment:
(tutorial-env) $ python -m pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)
``python -m pip freeze`` will produce a similar list of the installed packages,
but the output uses the format that ``python -m pip install`` expects.
A common convention is to put this list in a ``requirements.txt`` file:
(tutorial-env) $ python -m pip freeze > requirements.txt
(tutorial-env) $ cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0
The ``requirements.txt`` can then be committed to version control and
shipped as part of an application. Users can then install all the
necessary packages with ``install -r``:
(tutorial-env) $ python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
...
Installing collected packages: novas, numpy, requests
Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
``pip`` has many more options. Consult the installing-index
guide for complete documentation for ``pip``. When you've written
a package and want to make it available on the Python Package Index,
consult the `Python packaging user guide`_.
.. _Python Packaging User Guide: https://packaging.python.org/en/latest/tutorials/packaging-projects/

[python_098] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Overview

Sumber: ht

What's New In Python 3.12
:Editor: Adam Turner
.. Rules for maintenance:
* Anyone can add text to this document. Do not spend very much time
on the wording of your changes, because your text will probably
get rewritten to some degree.
* The maintainer will go through Misc/NEWS periodically and add
changes; it's therefore more important to add your changes to
Misc/NEWS than to this file.
* This is not a complete list of every single change; completeness
is the purpose of Misc/NEWS. Some changes I consider too small
or esoteric to include. If such a change is added to the text,
I'll just remove it. (This is another reason you shouldn't spend
too much time on writing your addition.)
* If you want to draw your new text to the attention of the
maintainer, add 'XXX' to the beginning of the paragraph or
section.
* It's OK to just add a fragmentary note about a change. For
example: "XXX Describe the transmogrify() function added to the
socket module." The maintainer will research the change and
write the necessary text.
* You can comment out your additions if you like, but it's not
necessary (especially when a final release is some months away).
* Credit the author of a patch or bugfix.   Just the name is
sufficient; the e-mail address isn't necessary.
* It's helpful to add the issue number as a comment:
XXX Describe the transmogrify() function added to the socket
module.
(Contributed by P.Y. Developer in 12345.)
This saves the maintainer the effort of going through the VCS log when
researching a change.
This article explains the new features in Python 3.12, compared to 3.11.
Python 3.12 was released on October 2, 2023.
For full details, see the changelog.
.. seealso

[python_099] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Summary -- Release highlights

Sumber: ht

.. This section singles out the most important changes in Python 3.12.
Brevity is key.
Python 3.12 is a stable release of the Python programming language,
with a mix of changes to the language and the standard library.
The library changes focus on cleaning up deprecated APIs, usability, and correctness.
Of note, the distutils package has been removed from the standard library.
Filesystem support in os and pathlib has seen a number of improvements,
and several modules have better performance.

The language changes focus on usability,
as f-string have had many limitations removed
and 'Did you mean ...' suggestions continue to improve.
The new whatsnew312-pep695
and type statement improve ergonomics for using generic type and type alias with static type checkers.
This article doesn't attempt to provide a complete specification of all new features,
but instead gives a convenient overview.
For full details, you should refer to the documentation,
such as the library-index
and reference-index.
If you want to understand the complete implementation and design rationale for a change,
refer to the PEP for a particular new feature;
but note that PEPs usually are not kept up-to-date
once a feature has been fully implemented.

[python_100] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax):

Sumber: ht

.. PEP-sized items next.
New syntax features:
* whatsnew312-pep695, type parameter syntax and the type statement
New grammar features:
* whatsnew312-pep701, f-string in the grammar
Interpreter improvements:
* whatsnew312-pep684, a unique per-interpreter global interpreter lock
* whatsnew312-pep669, low impact monitoring
* `Improved 'Did you mean ...' suggestions <improved error messages_>`_
for NameError, ImportError, and SyntaxError exceptions
Python data model improvements:
* whatsnew312-pep688, using the bufferobjects from Python
Significant improvements in the standard library:
* The pathlib.Path class now supports subclassing
* The os module received several improvements for Windows support
* A sqlite3-cli has been added to the
sqlite3 module
* isinstance checks against typing.runtime_checkable enjoy a speed up of between two and 20 times
* The asyncio package has had a number of performance improvements,
with some benchmarks showing a 75% speed up.
* A uuid-cli has been added to the
uuid module
* Due to the changes in whatsnew312-pep701,
producing tokens via the tokenize module is up to 64% faster.
Security improvements:
* Replace the builtin hashlib implementations of
SHA1, SHA3, SHA2-384, SHA2-512, and MD5 with formally verified code from the
`HACL* <https://github.com/hacl-star/hacl-star/>`__ project.
These builtin implementations remain as fallbacks that are only used when
OpenSSL does not provide them.
C API improvements:
* whatsnew312-pep697, unstable C API tier
* whatsnew312-pep683, immortal objects

CPython implementation improvements:
* whatsnew312-pep709, comprehension inlining
* perf_profiling for the Linux ``perf`` profiler
* Implement stack overflow protection on supported platforms
New typing features:
* whatsnew312-pep692, using ~typing.TypedDict to

[python_101] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 695: Type Parameter Syntax

Sumber: ht

Generic classes and functions under 484 were declared using a verbose syntax
that left the scope of type parameters unclear and required explicit declarations of
variance.
695 introduces a new, more compact and explicit way to create
generic-classes and generic-functions
In addition, the PEP introduces a new way to declare type-aliases
using the type statement, which creates an instance of
~typing.TypeAliasType
Type aliases can also be generic-type-aliases
The new syntax allows declaring ~typing.TypeVarTuple
and ~typing.ParamSpec parameters, as well as ~typing.TypeVar
parameters with bounds or constraints
The value of type aliases and the bound and constraints of type variables
created through this syntax are evaluated only on demand (see
lazy-evaluation). This means type aliases are able to
refer to other types defined later in the file.
Type parameters declared through a type parameter list are visible within the
scope of the declaration and any nested scopes, but not in the outer scope. For
example, they can be used in the type annotations for the methods of a generic
class or in the class body. However, they cannot be used in the module scope after
the class is defined. See type-params for a detailed description of the
runtime semantics of type parameters.
In order to support these scoping semantics, a new kind of scope is introduced,
the annotation-scopes. Annotation scopes behave for the
most part like function scopes, but interact differently with enclosing class scopes.
In Python 3.13, annotation will also be evaluated in
annotation scopes.
See 695 for more details.
(PEP written by Eric Traut. Implementation by Jelle Zijlstra, Eric Traut,
and others in 103764.)

Contoh Kode / Perintah CLI:

```
def max[T](args: Iterable[T]) -> T:
...
class list[T]:
def __getitem__(self, index: int, /) -> T:
...
def append(self, element: T) -> None:
...

# --------------------------------------
```

```
type Point = tuple[float, float]

# ----------------------------------------

type Point[T] = tuple[T, T]
```

[python_102] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 701: Syntactic formalization of f-strings

Sumber: ht

701 lifts some restrictions on the usage of f-string.
Expression components inside f-strings can now be any valid Python expression,
including strings reusing the same quote as the containing f-string,
multi-line expressions, comments, backslashes, and unicode escape sequences.
Let's cover these in detail:
* Quote reuse: in Python 3.11, reusing the same quotes as the enclosing f-string
raises a SyntaxError, forcing the user to either use other available
quotes (like using double quotes or triple quotes if the f-string uses single
quotes). In Python 3.12, you can now do things like this:
>>> songs = ['Take me back to Eden', 'Alkaline', 'Ascensionism']
>>> f"This is the playlist: {", ".join(songs)}"
'This is the playlist: Take me back to Eden, Alkaline, Ascensionism'
Note that before this change there was no explicit limit in how f-strings can
be nested, but the fact that string quotes cannot be reused inside the
expression component of f-strings made it impossible to nest f-strings
arbitrarily. In fact, this is the most nested f-string that could be written:
>>> f"""{f'''{f'{f"{1+1}"}'}'''}"""
'2'
As now f-strings can contain any valid Python expression inside expression
components, it is now possible to nest f-strings arbitrarily:
>>> f"{f"{f"{f"{f"{f"{1+1}"}"}"}"}"}"
'2'
* Multi-line expressions and comments: In Python 3.11, f-string expressions
must be defined in a single line, even if the expression within the f-string
could normally span multiple lines
(like literal lists being defined over multiple lines),
making them harder to read. In Python 3.12 you can now define f-strings
spanning multiple lines, and add inline comments:
>>> f"This is the playlist: {", ".join([
... 'Take me back to Eden', # My, my, those eyes like fire
... 'Alkaline', # N

[python_103] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 684: A Per-Interpreter GIL

Sumber: ht

684 introduces a per-interpreter global interpreter lock,
so that sub-interpreters may now be created with a unique GIL per interpreter.
This allows Python programs to take full advantage of multiple CPU
cores. This is currently only available through the C-API,

though a Python API is 554.
Use the new :cPy_NewInterpreterFromConfig function to
create an interpreter with its own GIL:
PyInterpreterConfig config = {
.check_multi_interp_extensions = 1,
.gil = PyInterpreterConfig_OWN_GIL,
};
PyThreadState *tstate = NULL;
PyStatus status = Py_NewInterpreterFromConfig(&tstate, &config);
if (PyStatus_Exception(status)) {
return -1;
}
/* The new interpreter is now active in the current thread. */
For further examples how to use the C-API for sub-interpreters with a
per-interpreter GIL, see ``Modules/_xxsubinterpretersmodule.c``.
(Contributed by Eric Snow in 104210, etc.)

[python_104] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 669: Low impact monitoring for CPython

Sumber: ht

669 defines a new sys.monitoring for profilers,
debuggers, and other tools to monitor events in CPython.
It covers a wide range of events, including calls,
returns, lines, exceptions, jumps, and more.
This means that you only pay for what you use, providing support
for near-zero overhead debuggers and coverage tools.
See sys.monitoring for details.
(Contributed by Mark Shannon in 103082.)

[python_105] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 688: Making the buffer protocol accessible in Python

Sumber: ht

688 introduces a way to use the bufferobjects
from Python code. Classes that implement the ~object.__buffer__ method
are now usable as buffer types.
The new collections.abc.Buffer ABC provides a standard
way to represent buffer objects, for example in type annotations.
The new inspect.BufferFlags enum represents the flags that
can be used to customize buffer creation.
(Contributed by Jelle Zijlstra in 102500.)

[python_106] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 709: Comprehension inlining

Sumber: ht

Dictionary, list, and set comprehensions are now inlined, rather than creating a new single-use function object for each execution of the comprehension. This speeds up execution of a comprehension by up to two times.
See 709 for further details.
Comprehension iteration variables remain isolated and don't overwrite a variable of the same name in the outer scope, nor are they visible after the comprehension. Inlining does result in a few visible behavior changes:
* There is no longer a separate frame for the comprehension in tracebacks, and tracing/profiling no longer shows the comprehension as a function call.
* The symtable module will no longer produce child symbol tables for each comprehension; instead, the comprehension's locals will be included in the parent function's symbol table.
* Calling locals inside a comprehension now includes variables from outside the comprehension, and no longer includes the synthetic ``.0`` variable for the comprehension "argument".
* A comprehension iterating directly over ``locals()`` (e.g. ``[k for k in locals()]``) may see "RuntimeError: dictionary changed size during iteration" when run under tracing (e.g. code coverage measurement). This is the same behavior already seen in e.g. ``for k in locals():``. To avoid the error, first create a list of keys to iterate over: ``keys = list(locals()); [k for k in keys]``.
(Contributed by Carl Meyer and Vladimir Matveev in 709.)

[python_107] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Improved Error Messages

Sumber: ht

* Modules from the standard library are now potentially suggested as part of the error messages displayed by the interpreter when a NameError is raised to the top level. (Contributed by Pablo Galindo in 98254.)
>>> sys.version_info
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'sys' is not defined. Did you forget to import 'sys'?
* Improve the error suggestion for NameError exceptions for instances. Now if a NameError is raised in a method and the instance has an attribute that's exactly equal to the name in the exception, the suggestion will include ``self.<NAME>`` instead of the closest match in the method scope. (Contributed by Pablo Galindo in 99139.)
>>> class A:
... def __init__(self):
... self.blech = 1
...
... def foo(self):
... somethin = blech
...

```
>>> A().foo()
Traceback (most recent call last):
File "<stdin>", line 1
somethin = blech
^^^^^
NameError: name 'blech' is not defined. Did you mean: 'self.blech'?
```

* Improve the SyntaxError error message when the user types ``import x
from y`` instead of ``from y import x``. (Contributed by Pablo Galindo in 98931.)

```
>>> import a.y.z from b.y.z
Traceback (most recent call last):
File "<stdin>", line 1
import a.y.z from b.y.z
^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: Did you mean to use 'from ... import ...' instead?
```

* ImportError exceptions raised from failed ``from <module> import
<name>`` statements now include suggestions for the value of ``<name>`` based on the
available names in ``<module>``. (Contributed by Pablo Galindo in 91058.)

```
>>> from collections import chainmap
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ImportError: cannot import name 'chainmap' from 'collections'. Did you mean: 'ChainMap'?
```

[python_108] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): New Features Related to Type Hints

Sumber: ht

This section covers major changes affecting 484 and
the typing module.

[python_109] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 692: Using ``TypedDict`` for more precise ``**kwargs`` typing

Sumber: ht

Typing ``**kwargs`` in a function signature as introduced by 484 allowed
for valid annotations only in cases where all of the ``**kwargs`` were of the
same type.
692 specifies a more precise way of typing ``**kwargs`` by relying on
typed dictionaries
See 692 for more details.
(Contributed by Franek Magiera in 103629.)

Contoh Kode / Perintah CLI:

```
from typing import TypedDict, Unpack
class Movie(TypedDict):
name: str
year: int
def foo(**kwargs: Unpack[Movie]): ...
```

[python_110] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): PEP 698: Override Decorator for Static Typing

Sumber: ht

A new decorator typing.override has been added to the typing
module. It indicates to type checkers that the method is intended to override
a method in a superclass. This allows type checkers to catch mistakes where
a method that is intended to override something in a base class
does not in fact do so.
Example
See 698 for more details.
(Contributed by Steven Troxler in 101561.)

Contoh Kode / Perintah CLI:

```
from typing import override
class Base:
def get_color(self) -> str:
return "blue"
class GoodChild(Base):
@override  # ok: overrides Base.get_color
def get_color(self) -> str:
return "yellow"
class BadChild(Base):
@override  # type checker error: does not override Base.get_color
def get_colour(self) -> str:
return "red"
```

[python_111] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Other Language Changes

Sumber: ht

* The parser now raises SyntaxError when parsing source code containing
null bytes. (Contributed by Pablo Galindo in 96670.)
* A backslash-character pair that is not a valid escape sequence now generates
a SyntaxWarning, instead of DeprecationWarning.
For example, ``re.compile("\d+\.\d+")`` now emits a SyntaxWarning
(``"\d"`` is an invalid escape sequence, use raw strings for regular
expression: ``re.compile(r"\d+\.\d+")``).
In a future Python version, SyntaxError will eventually be raised,
instead of SyntaxWarning.
(Contributed by Victor Stinner in 98401.)
* Octal escapes with value larger than ``0o377`` (ex: ``"\477"``), deprecated
in Python 3.11, now produce a SyntaxWarning, instead of
DeprecationWarning.
In a future Python version they will be eventually a SyntaxError.
(Contributed by Victor Stinner in 98401.)
* Variables used in the target part of comprehensions that are not stored to
can now be used in assignment expressions (``:=``).
For example, in ``[(b := 1) for a, b.prop in some_iter]``, the assignment to
``b`` is now allowed. Note that assigning to variables stored to in the target
part of comprehensions (like ``a``) is still disallowed, as per 572.
(Contributed by Nikita Sobolev in 100581.)
* Exceptions raised in a class or type's ``__set_name__`` method are no longer

wrapped by a RuntimeError. Context information is added to the
exception as a 678 note. (Contributed by Irit Katriel in 77757.)
* When a ``try-except*`` construct handles the entire ExceptionGroup
and raises one other exception, that exception is no longer wrapped in an
ExceptionGroup. Also changed in version 3.11.4. (Contributed by Irit
Katriel in 103590.)
* The Garbage Collector now runs only on the eval breaker mechanism of the
Python bytecode evaluation loop instead of object allocations.

[python_112] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): array

Sumber: ht

* The array.array class now supports subscripting, making it a
generic type. (Contributed by Jelle Zijlstra in 98658.)

[python_113] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): asyncio

Sumber: ht

* The performance of writing to sockets in asyncio has been
significantly improved. ``asyncio`` now avoids unnecessary copying when
writing to sockets and uses ~socket.socket.sendmsg if the platform
supports it. (Contributed by Kumar Aditya in 91166.)
* Add asyncio.eager_task_factory and asyncio.create_eager_task_factory
functions to allow opting an event loop in to eager task execution,
making some use-cases 2x to 5x faster.
(Contributed by Jacob Bower & Itamar Oren in 102853, 104140, and 104138)
* On Linux, asyncio uses asyncio.PidfdChildWatcher by default
if os.pidfd_open is available and functional instead of
asyncio.ThreadedChildWatcher.
(Contributed by Kumar Aditya in 98024.)
* The event loop now uses the best available child watcher for each platform
(asyncio.PidfdChildWatcher if supported and
asyncio.ThreadedChildWatcher otherwise), so manually
configuring a child watcher is not recommended.
(Contributed by Kumar Aditya in 94597.)
* Add *loop_factory* parameter to asyncio.run to allow specifying
a custom event loop factory.
(Contributed by Kumar Aditya in 99388.)
* Add C implementation of asyncio.current_task for 4x-6x speedup.
(Contributed by Itamar Oren and Pranav Thulasiram Bhat in 100344.)
* asyncio.iscoroutine now returns ``False`` for generators as
asyncio does not support legacy generator-based coroutines.
(Contributed by Kumar Aditya in 102748.)
* asyncio.wait and asyncio.as_completed now accepts generators
yielding tasks.
(Contributed by Kumar Aditya in 78530.)

[python_114] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): calendar

Sumber: ht

* Add enums calendar.Month and calendar.Day
defining months of the year and days of the week.
(Contributed by Prince Roshan in 103636.)

[python_115] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): csv

Sumber: ht

* Add csv.QUOTE_NOTNULL and csv.QUOTE_STRINGS flags to
provide finer grained control of ``None`` and empty strings by
~csv.reader and ~csv.writer objects.

[python_116] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): dis

Sumber: ht

* Pseudo instruction opcodes (which are used by the compiler but
do not appear in executable bytecode) are now exposed in the
dis module.
HAVE_ARGUMENT is still relevant to real opcodes,
but it is not useful for pseudo instructions. Use the new
dis.hasarg collection instead.
(Contributed by Irit Katriel in 94216.)
* Add the dis.hasexc collection to signify instructions that set
an exception handler. (Contributed by Irit Katriel in 94216.)

[python_117] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): fractions

Sumber: ht

* Objects of type fractions.Fraction now support float-style
formatting. (Contributed by Mark Dickinson in 100161.)

[python_118] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): importlib.resources

Sumber: ht

* importlib.resources.as_file now supports resource directories.
(Contributed by Jason R. Coombs in 97930.)
* Rename first parameter of importlib.resources.files to *anchor*.
(Contributed by Jason R. Coombs in 100598.)

[python_119] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): inspect

Sumber: ht

* Add inspect.markcoroutinefunction to mark sync functions that return
a coroutine for use with inspect.iscoroutinefunction.
(Contributed by Carlton Gibson in 99247.)
* Add inspect.getasyncgenstate and inspect.getasyncgenlocals
for determining the current state of asynchronous generators.
(Contributed by Thomas Krennwallner in 79940.)
* The performance of inspect.getattr_static has been considerably
improved. Most calls to the function should be at least 2x faster than they
were in Python 3.11. (Contributed by Alex Waygood in 103193.)

[python_120] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): itertools

Sumber: ht

* Add itertools.batched for collecting into even-sized
tuples where the last batch may be shorter than the rest.
(Contributed by Raymond Hettinger in 98363.)

[python_121] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): math

Sumber: ht

* Add math.sumprod for computing a sum of products.
(Contributed by Raymond Hettinger in 100485.)
* Extend math.nextafter to include a *steps* argument
for moving up or down multiple steps at a time. (Contributed by
Matthias Goergens, Mark Dickinson, and Raymond Hettinger in 94906.)
os
--
* Add os.PIDFD_NONBLOCK to open a file descriptor
for a process with os.pidfd_open in non-blocking mode.
(Contributed by Kumar Aditya in 93312.)
* os.DirEntry now includes an os.DirEntry.is_junction
method to check if the entry is a junction.
(Contributed by Charles Machalow in 99547.)
* Add os.listdrives, os.listvolumes and os.listmounts
functions on Windows for enumerating drives, volumes and mount points.
(Contributed by Steve Dower in 102519.)
* os.stat and os.lstat are now more accurate on Windows.
The ``st_birthtime`` field will now be filled with the creation time
of the file, and ``st_ctime`` is deprecated but still contains the
creation time (but in the future will return the last metadata change,
for consistency with other platforms). ``st_dev`` may be up to 64 bits
and ``st_ino`` up to 128 bits depending on your file system, and
``st_rdev`` is always set to zero rather than incorrect values.
Both functions may be significantly faster on newer releases of
Windows. (Contributed by Steve Dower in 99726.)

[python_122] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): os.path

Sumber: ht

* Add os.path.isjunction to check if a given path is a junction.
(Contributed by Charles Machalow in 99547.)
* Add os.path.splitroot to split a path into a triad
``(drive, root, tail)``. (Contributed by Barney Gale in 101000.)

[python_123] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): pathlib

Sumber: ht

* Add support for subclassing pathlib.PurePath and
pathlib.Path, plus their Posix- and Windows-specific variants.
Subclasses may override the pathlib.PurePath.with_segments method
to pass information between path instances.
* Add pathlib.Path.walk for walking the directory trees and generating
all file or directory names within them, similar to os.walk.
(Contributed by Stanislav Zmiev in 90385.)
* Add *walk_up* optional parameter to pathlib.PurePath.relative_to
to allow the insertion of ``..`` entries in the result; this behavior is
more consistent with os.path.relpath.
(Contributed by Domenico Ragusa in 84538.)
* Add pathlib.Path.is_junction as a proxy to os.path.isjunction.
(Contributed by Charles Machalow in 99547.)
* Add *case_sensitive* optional parameter to pathlib.Path.glob,
pathlib.Path.rglob and pathlib.PurePath.match for matching
the path's case sensitivity, allowing for more precise control over the matching process.

[python_124] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): platform

Sumber: ht

* Add support for detecting Windows 11 and Windows Server releases past 2012.
Previously, lookups on Windows Server platforms newer than Windows Server 2012
and on Windows 11 would return ``Windows-10``.
(Contributed by Steve Dower in 89545.)

[python_125] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): pdb

Sumber: ht

* Add convenience variables to hold values temporarily for debug session
and provide quick access to values like the current frame or the return
value.
(Contributed by Tian Gao in 103693.)

[python_126] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): random

Sumber: ht

* Add random.binomialvariate.
(Contributed by Raymond Hettinger in 81620.)
* Add a default of ``lambd=1.0`` to random.expovariate.
(Contributed by Raymond Hettinger in 100234.)

[python_127] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): shutil

Sumber: ht

* shutil.make_archive now passes the *root_dir* argument to custom
archivers which support it.
In this case it no longer temporarily changes the current working directory
of the process to *root_dir* to perform archiving.
(Contributed by Serhiy Storchaka in 74696.)
* shutil.rmtree now accepts a new argument *onexc* which is an
error handler like *onerror* but which expects an exception instance
rather than a *(typ, val, tb)* triplet. *onerror* is deprecated.
(Contributed by Irit Katriel in 102828.)
* shutil.which now consults the *PATHEXT* environment variable to
find matches within *PATH* on Windows even when the given *cmd* includes
a directory component.
(Contributed by Charles Machalow in 103179.)
shutil.which will call ``NeedCurrentDirectoryForExePathW`` when
querying for executables on Windows to determine if the current working
directory should be prepended to the search path.
(Contributed by Charles Machalow in 103179.)
shutil.which will return a path matching the *cmd* with a component
from ``PATHEXT`` prior to a direct match elsewhere in the search path on
Windows.
(Contributed by Charles Machalow in 103179.)

[python_128] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): sqlite3

Sumber: ht

* Add a sqlite3-cli.
(Contributed by Erlend E. Aasland in 77617.)
* Add the sqlite3.Connection.autocommit attribute
to sqlite3.Connection
and the *autocommit* parameter to sqlite3.connect
to control 249-compliant
sqlite3-transaction-control-autocommit.
(Contributed by Erlend E. Aasland in 83638.)
* Add *entrypoint* keyword-only parameter to
sqlite3.Connection.load_extension,
for overriding the SQLite extension entry point.
(Contributed by Erlend E. Aasland in 103015.)
* Add sqlite3.Connection.getconfig and
sqlite3.Connection.setconfig to sqlite3.Connection

to make configuration changes to a database connection.
(Contributed by Erlend E. Aasland in 103489.)

[python_129] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): statistics

Sumber: ht

* Extend statistics.correlation to include as a ``ranked`` method
for computing the Spearman correlation of ranked data.
(Contributed by Raymond Hettinger in 95861.)

[python_130] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): sys

Sumber: ht

* Add the sys.monitoring namespace to expose the new whatsnew312-pep669 monitoring API.
(Contributed by Mark Shannon in 103082.)
* Add sys.activate_stack_trampoline and
sys.deactivate_stack_trampoline for activating and deactivating
stack profiler trampolines,
and sys.is_stack_trampoline_active for querying if stack profiler
trampolines are active.
(Contributed by Pablo Galindo and Christian Heimes
with contributions from Gregory P. Smith [Google] and Mark Shannon
in 96123.)
* Add sys.last_exc which holds the last unhandled exception that
was raised (for post-mortem debugging use cases). Deprecate the
three fields that have the same information in its legacy form:
sys.last_type, sys.last_value and sys.last_traceback.
(Contributed by Irit Katriel in 102778.)
* sys._current_exceptions now returns a mapping from thread-id to an
exception instance, rather than to a ``(typ, exc, tb)`` tuple.
(Contributed by Irit Katriel in 103176.)
* sys.setrecursionlimit and sys.getrecursionlimit.
The recursion limit now applies only to Python code. Builtin functions do
not use the recursion limit, but are protected by a different mechanism
that prevents recursion from causing a virtual machine crash.

[python_131] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): tempfile

Sumber: ht

* The tempfile.NamedTemporaryFile function has a new optional parameter
delete_on_close (Contributed by Evgeny Zorin in 58451.)
* tempfile.mkdtemp now always returns an absolute path, even if the
argument provided to the *dir* parameter is a relative path.

[python_132] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): threading

Sumber: ht

* Add threading.settrace_all_threads and
threading.setprofile_all_threads that allow to set tracing and
profiling functions in all running threads in addition to the calling one.
(Contributed by Pablo Galindo in 93503.)

[python_133] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): tkinter

Sumber: ht

* ``tkinter.Canvas.coords()`` now flattens its arguments.
It now accepts not only coordinates as separate arguments
(``x1, y1, x2, y2, ...``) and a sequence of coordinates
(``[x1, y1, x2, y2, ...]``), but also coordinates grouped in pairs
(``(x1, y1), (x2, y2), ...`` and ``[(x1, y1), (x2, y2), ...]``),
like ``create_*()`` methods.
(Contributed by Serhiy Storchaka in 94473.)

[python_134] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): tokenize

Sumber: ht

* The tokenize module includes the changes introduced in 701.
(Contributed by Marta Gómez Macías and Pablo Galindo in 102856.)
See whatsnew312-porting-to-python312 for more information on the
changes to the tokenize module.

[python_135] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): types

Sumber: ht

* Add types.get_original_bases to allow for further introspection of
user-defined-generics when subclassed. (Contributed by
James Hilton-Balfe and Alex Waygood in 101827.)

[python_136] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): typing

Sumber: ht

* isinstance checks against
typing.runtime_checkable now use
inspect.getattr_static rather than hasattr to lookup whether
attributes exist. This means that descriptors and ~object.__getattr__
methods are no longer unexpectedly evaluated during ``isinstance()`` checks
against runtime-checkable protocols. However, it may also mean that some
objects which used to be considered instances of a runtime-checkable protocol
may no longer be considered instances of that protocol on Python 3.12+, and
vice versa. Most users are unlikely to be affected by this change.
(Contributed by Alex Waygood in 102433.)
* The members of a runtime-checkable protocol are now considered "frozen" at
runtime as soon as the class has been created. Monkey-patching attributes

onto a runtime-checkable protocol will still work, but will have no impact on isinstance checks comparing objects to the protocol. For example runtime-checkable protocols.
* The performance profile of isinstance checks against typing.runtime_checkable has changed significantly. Most ``isinstance()`` checks against protocols with only a few members should be at least 2x faster than in 3.11, and some may be 20x faster or more. However, ``isinstance()`` checks against protocols with many members may be slower than in Python 3.11. (Contributed by Alex Waygood in 74690 and 103193.)
* All typing.TypedDict and typing.NamedTuple classes now have the ``__orig_bases__`` attribute. (Contributed by Adrian Garcia Badaracco in 103699.)
* Add ``frozen_default`` parameter to typing.dataclass_transform. (Contributed by Erik De Bonte in 99957.)

Contoh Kode / Perintah CLI:

```
>>> from typing import Protocol, runtime_checkable
>>> @runtime_checkable
... class HasX(Protocol):
...     x = 1
...
>>> class Foo: ...
...
>>> f = Foo()
>>> isinstance(f, HasX)
False
>>> f.x = 1
>>> isinstance(f, HasX)
True
>>> HasX.y = 2
>>> isinstance(f, HasX)  # unchanged, even though HasX now also has a "y" attribute
True
This change was made in order to speed up ``isinstance()`` checks against
```

[python_137] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): unicodedata

Sumber: ht

* The Unicode database has been updated to version 15.0.0. (Contributed by Benjamin Peterson in 96734).

[python_138] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): unittest

Sumber: ht

Add a ``--durations`` command line option, showing the N slowest test cases::
python3 -m unittest --durations=3 lib.tests.test_threading
.....

[python_139] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Slowest test durations

Sumber: ht

1.210s test_timeout (Lib.test.test_threading.BarrierTests)
1.003s test_default_timeout (Lib.test.test_threading.BarrierTests)
0.518s test_timeout (Lib.test.test_threading.EventTests)

[python_140] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): (0.000 durations hidden. Use -v to show these durations.)

Sumber: ht

Ran 158 tests in 9.869s
OK (skipped=3)
(Contributed by Giampaolo Rodola in 48330)

[python_141] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): uuid

Sumber: ht

* Add a uuid-cli.
(Contributed by Adam Chhina in 88597.)

[python_142] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Optimizations

Sumber: ht

* Remove ``wstr`` and ``wstr_length`` members from Unicode objects.
It reduces object size by 8 or 16 bytes on 64bit platform. (623)
(Contributed by Inada Naoki in 92536.)
* Add experimental support for using the BOLT binary optimizer in the build
process, which improves performance by 1-5%.
(Contributed by Kevin Modzelewski in 90536 and tuned by Donghee Na in 101525)
* Speed up the regular expression substitution (functions re.sub and
re.subn and corresponding re.Pattern methods) for
replacement strings containing group references by 2--3 times.
(Contributed by Serhiy Storchaka in 91524.)
* Speed up asyncio.Task creation by deferring expensive string formatting.
(Contributed by Itamar Oren in 103793.)
* The tokenize.tokenize and tokenize.generate_tokens functions are
up to 64% faster as a side effect of the changes required to cover 701 in
the tokenize module. (Contributed by Marta Gómez Macías and Pablo Galindo
in 102856.)
* Speed up super method calls and attribute loads via the
new LOAD_SUPER_ATTR instruction. (Contributed by Carl Meyer and
Vladimir Matveev in 103497.)

[python_143] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): CPython bytecode changes

Sumber: ht

* Remove the LOAD_METHOD instruction. It has been merged into
LOAD_ATTR. LOAD_ATTR will now behave like the old
LOAD_METHOD instruction if the low bit of its oparg is set.
(Contributed by Ken Jin in 93429.)
* Remove the JUMP_IF_FALSE_OR_POP and JUMP_IF_TRUE_OR_POP
instructions. (Contributed by Irit Katriel in 102859.)
* Remove the PRECALL instruction. (Contributed by Mark Shannon in
92925.)
* Add the BINARY_SLICE and STORE_SLICE instructions.
(Contributed by Mark Shannon in 94163.)
* Add the CALL_INTRINSIC_1 instructions.
(Contributed by Mark Shannon in 99005.)
* Add the CALL_INTRINSIC_2 instruction.
(Contributed by Irit Katriel in 101799.)
* Add the CLEANUP_THROW instruction.
(Contributed by Brandt Bucher in 90997.)
* Add the END_SEND instruction.
(Contributed by Mark Shannon in 103082.)
* Add the LOAD_FAST_AND_CLEAR instruction as part of the
implementation of 709. (Contributed by Carl Meyer in 101441.)
* Add the LOAD_FAST_CHECK instruction.
(Contributed by Dennis Sweeney in 93143.)
* Add the LOAD_FROM_DICT_OR_DEREF, LOAD_FROM_DICT_OR_GLOBALS,
and LOAD_LOCALS opcodes as part of the implementation of 695.
Remove the LOAD_CLASSDEREF opcode, which can be replaced with
LOAD_LOCALS plus LOAD_FROM_DICT_OR_DEREF. (Contributed
by Jelle Zijlstra in 103764.)
* Add the LOAD_SUPER_ATTR instruction. (Contributed by Carl Meyer and
Vladimir Matveev in 103497.)
* Add the ``RETURN_CONST`` instruction. (Contributed by Wenyang Wang in 101632.)

[python_144] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Demos and Tools

Sumber: ht

* Remove the ``Tools/demo/`` directory which contained old demo scripts. A copy
can be found in the `old-demos project
<https://github.com/gvanrossum/old-demos>`_.
(Contributed by Victor Stinner in 97681.)
* Remove outdated example scripts of the ``Tools/scripts/`` directory.
A copy can be found in the `old-demos project
<https://github.com/gvanrossum/old-demos>`_.
(Contributed by Victor Stinner in 97669.)

[python_145] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Deprecated

Sumber: ht

* argparse: The *type*, *choices*, and *metavar* parameters
of argparse.BooleanOptionalAction are deprecated
and will be removed in 3.14.
(Contributed by Nikita Sobolev in 92248.)
* ast: The following ast features have been deprecated in documentation since
Python 3.8, now cause a DeprecationWarning to be emitted at runtime
when they are accessed or used, and will be removed in Python 3.14:
* ast.Num
* ast.Str
* ast.Bytes
* ast.NameConstant
* ast.Ellipsis
Use ast.Constant instead.
(Contributed by Serhiy Storchaka in 90953.)
* asyncio:
* The child watcher classes asyncio.MultiLoopChildWatcher,
asyncio.FastChildWatcher, asyncio.AbstractChildWatcher
and asyncio.SafeChildWatcher are deprecated and
will be removed in Python 3.14.
(Contributed by Kumar Aditya in 94597.)
* asyncio.set_child_watcher, asyncio.get_child_watcher,
asyncio.AbstractEventLoopPolicy.set_child_watcher and
asyncio.AbstractEventLoopPolicy.get_child_watcher are deprecated
and will be removed in Python 3.14.
(Contributed by Kumar Aditya in 94597.)
* The ~asyncio.get_event_loop method of the
default event loop policy now emits a DeprecationWarning if there
is no current event loop set and it decides to create one.
(Contributed by Serhiy Storchaka and Guido van Rossum in 100160.)
* calendar: ``calendar.January`` and ``calendar.February`` constants are deprecated and
replaced by calendar.JANUARY and calendar.FEBRUARY.
(Contributed by Prince Roshan in 103636.)
* collections.abc: Deprecated collections.abc.ByteString.
Use ``isinstance(obj, collections.abc.Buffer)`` to test if ``obj`` implements
the bufferobjects at runtime. For use in type
annotations, either use ~collections.abc.Buffer or a union
that explicitly specifies the types your code supports (e.g.,
``bytes | bytearray | memoryview``).
ByteString was or

[python_146] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): asynchat and asyncore

Sumber: ht

* These two modules have been removed
according to the schedule in 594,
having been deprecated in Python 3.6.
Use asyncio instead.

(Contributed by Nikita Sobolev in 96580.)

[python_147] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): configparser

Sumber: ht

* Several names deprecated in the configparser way back in 3.2 have
been removed per 89336:
* configparser.ParsingError no longer has a ``filename`` attribute
or argument. Use the ``source`` attribute and argument instead.
* configparser no longer has a ``SafeConfigParser`` class. Use the
shorter ~configparser.ConfigParser name instead.
* configparser.ConfigParser no longer has a ``readfp`` method.
Use ~configparser.ConfigParser.read_file instead.

[python_148] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): distutils

Sumber: ht

* Remove the :pydistutils package. It was deprecated in Python 3.10 by
632 "Deprecate distutils module". For projects still using
``distutils`` and cannot be updated to something else, the ``setuptools``
project can be installed: it still provides ``distutils``.
(Contributed by Victor Stinner in 92584.)

[python_149] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): ensurepip

Sumber: ht

* Remove the bundled setuptools wheel from ensurepip,
and stop installing setuptools in environments created by venv.
``pip (>= 22.1)`` does not require setuptools to be installed in the
environment. ``setuptools``-based (and ``distutils``-based) packages
can still be used with ``pip install``, since pip will provide
``setuptools`` in the build environment it uses for building a
package.
``easy_install``, ``pkg_resources``, ``setuptools`` and ``distutils``
are no longer provided by default in environments created with
``venv`` or bootstrapped with ``ensurepip``, since they are part of
the ``setuptools`` package. For projects relying on these at runtime,
the ``setuptools`` project should be declared as a dependency and
installed separately (typically, using pip).
(Contributed by Pradyun Gedam in 95299.)

[python_150] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): enum

Sumber: ht

* Remove enum's ``EnumMeta.__getattr__``, which is no longer needed for
enum attribute access.
(Contributed by Ethan Furman in 95083.)

[python_151] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): ftplib

Sumber: ht

* Remove ftplib's ``FTP_TLS.ssl_version`` class attribute: use the
context parameter instead.
(Contributed by Victor Stinner in 94172.)

[python_152] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): gzip

Sumber: ht

* Remove the ``filename`` attribute of gzip's gzip.GzipFile,
deprecated since Python 2.6, use the ~gzip.GzipFile.name attribute
instead. In write mode, the ``filename`` attribute added ``'.gz'`` file
extension if it was not present.
(Contributed by Victor Stinner in 94196.)

[python_153] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): hashlib

Sumber: ht

* Remove the pure Python implementation of hashlib's
hashlib.pbkdf2_hmac, deprecated in Python 3.10. Python 3.10 and
newer requires OpenSSL 1.1.1 (644): this OpenSSL version provides
a C implementation of ~hashlib.pbkdf2_hmac which is faster.
(Contributed by Victor Stinner in 94199.)

[python_154] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): importlib

Sumber: ht

* Many previously deprecated cleanups in importlib have now been
completed:
* References to, and support for module_repr has been removed.
(Contributed by Barry Warsaw in 97850.)
* ``importlib.util.set_package``, ``importlib.util.set_loader`` and
``importlib.util.module_for_loader`` have all been removed. (Contributed by
Brett Cannon and Nikita Sobolev in 65961 and 97850.)
* Support for ``find_loader()`` and ``find_module()`` APIs have been
removed. (Contributed by Barry Warsaw in 98040.)
* ``importlib.abc.Finder``, ``pkgutil.ImpImporter``, and ``pkgutil.ImpLoader``
have been removed. (Contributed by Barry Warsaw in 98040.)

[python_155] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): imp

Sumber: ht

* The imp module has been removed. (Contributed by Barry Warsaw in
98040.)
To migrate, consult the following correspondence table:

imp	importlib

``imp.NullImporter`` Insert ``None`` into ``sys.path_importer_cache``
``imp.cache_from_source()`` importlib.util.cache_from_source
``imp.find_module()`` importlib.util.find_spec
``imp.get_magic()`` importlib.util.MAGIC_NUMBER
``imp.get_suffixes()`` importlib.machinery.SOURCE_SUFFIXES, importlib.machinery.EXTENSION_SUFFIXES, and importlib.machinery.BYTECODE_SUFFIXES
``imp.get_tag()`` sys.implementation
``imp.load_module()`` importlib.import_module
``imp.new_module(name)`` ``types.ModuleType(name)``
``imp.reload()`` importlib.reload
``imp.source_from_cache()`` importlib.util.source_from_cache
``imp.load_source()`` *See below*
========== ==========
Replace ``imp.load_source()`` with
* Remove imp functions and attributes with no replacements:
* Undocumented functions:
* ``imp.init_builtin()``
* ``imp.load_compiled()``
* ``imp.load_dynamic()``
* ``imp.load_package()``
* ``imp.lock_held()``, ``imp.acquire_lock()``, ``imp.release_lock()``:
the locking scheme has changed in Python 3.3 to per-module locks.
* ``imp.find_module()`` constants: ``SEARCH_ERROR``, ``PY_SOURCE``, ``PY_COMPILED``, ``C_EXTENSION``, ``PY_RESOURCE``, ``PKG_DIRECTORY``, ``C_BUILTIN``, ``PY_FROZEN``, ``PY_CODERESOURCE``, ``IMP_HOOK``.
io
--
* Remove io's ``io.OpenWrapper`` and ``_pyio.OpenWrapper``,

Contoh Kode / Perintah CLI:

```
import importlib.util
import importlib.machinery
def load_source(modname, filename):
loader = importlib.machinery.SourceFileLoader(modname, filename)
spec = importlib.util.spec_from_file_location(modname, filename, loader=loader)
module = importlib.util.module_from_spec(spec)
# The module is always executed and not cached in sys.modules.
# Uncomment the following line to cache the module.
# sys.modules[module.__name__] = module
loader.exec_module(module)
return module
```

[python_156] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): locale

Sumber: ht

* Remove locale's locale.format function, deprecated in Python 3.7:
use locale.format_string instead.
(Contributed by Victor Stinner in 94226.)

[python_157] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): smtpd

Sumber: ht

* The ``smtpd`` module has been removed according to the schedule in 594,
having been deprecated in Python 3.4.7 and 3.5.4.
Use the aiosmtpd PyPI module or any other
asyncio-based server instead.
(Contributed by Oleg Iarygin in 93243.)

[python_158] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): sqlite3

Sumber: ht

* The following undocumented sqlite3 features, deprecated in Python
3.10, are now removed:
* ``sqlite3.enable_shared_cache()``
* ``sqlite3.OptimizedUnicode``
If a shared cache must be used, open the database in URI mode using the
``cache=shared`` query parameter.
The ``sqlite3.OptimizedUnicode`` text factory has been an alias for
str since Python 3.3. Code that previously set the text factory to
``OptimizedUnicode`` can either use ``str`` explicitly, or rely on the
default value which is also ``str``.
(Contributed by Erlend E. Aasland in 92548.)

[python_159] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): ssl

Sumber: ht

* Remove ssl's ssl.RAND_pseudo_bytes function, deprecated in Python 3.6:
use os.urandom or ssl.RAND_bytes instead.
(Contributed by Victor Stinner in 94199.)
* Remove the ssl.match_hostname function.
It was deprecated in Python 3.7. OpenSSL performs
hostname matching since Python 3.7, Python no longer uses the
ssl.match_hostname function.
(Contributed by Victor Stinner in 94199.)
* Remove the ssl.wrap_socket function, deprecated in Python 3.7:
instead, create a ssl.SSLContext object and call its
ssl.SSLContext.wrap_socket method. Any package that still uses
ssl.wrap_socket is broken and insecure. The function neither sends a
SNI TLS extension nor validates the server hostname. Code is subject to 295
(Improper Certificate Validation).
(Contributed by Victor Stinner in 94199.)

[python_160] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): unittest

Sumber: ht

* Remove many long-deprecated unittest features:
.. _unittest-TestCase-removed-aliases:
* A number of ~unittest.TestCase method aliases:

Deprecated alias	Method Name	Deprecated in
``failUnless``	.assertTrue	3.1
``failIf``	.assertFalse	3.1
``failUnlessEqual``	.assertEqual	3.1
``failIfEqual``	.assertNotEqual	3.1
``failUnlessAlmostEqual``	.assertAlmostEqual	3.1
``failIfAlmostEqual``	.assertNotAlmostEqual	3.1
``failUnlessRaises``	.assertRaises	3.1
``assert_``	.assertTrue	3.2
``assertEquals``	.assertEqual	3.2
``assertNotEquals``	.assertNotEqual	3.2
``assertAlmostEquals``	.assertAlmostEqual	3.2
``assertNotAlmostEquals``	.assertNotAlmostEqual	3.2
``assertRegexpMatches``	.assertRegex	3.2
``assertRaisesRegexp``	.assertRaisesRegex	3.2
``assertNotRegexpMatches``	.assertNotRegex	3.5

You can use https://github.com/isidentical/teyit to automatically modernise
your unit tests.
* Undocumented and broken ~unittest.TestCase method
``assertDictContainsSubset`` (deprecated in Python 3.2).
* Undocumented unittest.TestLoader.loadTestsFromModule parameter *use_load_tests*
(deprecated and ignored since Python 3.5).
* An alias of the ~unittest.TextTestResult class:
``_TextTestResult`` (deprecated in Python 3.2).
(Contributed by Serhiy Storchaka in 89325.)

[python_161] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): webbrowser

Sumber: ht

* Remove support for obsolete browsers from webbrowser.
The removed browsers include: Grail, Mosaic, Netscape, Galeon, Skipstone,
Iceape, Firebird, and Firefox versions 35 and below (102871).

[python_162] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): xml.etree.ElementTree

Sumber: ht

* Remove the ``ElementTree.Element.copy()`` method of the
pure Python implementation, deprecated in Python 3.10, use the
copy.copy function instead. The C implementation of xml.etree.ElementTree
has no ``copy()`` method, only a ``__copy__()`` method.

(Contributed by Victor Stinner in 94383.)

[python_163] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): zipimport

Sumber: ht

* Remove zipimport's ``find_loader()`` and ``find_module()`` methods,
deprecated in Python 3.10: use the ``find_spec()`` method instead. See
451 for the rationale.
(Contributed by Victor Stinner in 94379.)

[python_164] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Others

Sumber: ht

* Remove the ``suspicious`` rule from the documentation Makefile and
Doc/tools/rstlint.py, both in favor of `sphinx-lint
<https://github.com/sphinx-contrib/sphinx-lint>`_.
(Contributed by Julien Palard in 98179.)
* Remove the *keyfile* and *certfile* parameters from the
ftplib, imaplib, poplib and smtplib modules,
and the *key_file*, *cert_file* and *check_hostname* parameters from the
http.client module,
all deprecated since Python 3.6. Use the *context* parameter
(*ssl_context* in imaplib) instead.
(Contributed by Victor Stinner in 94172.)
* Remove ``Jython`` compatibility hacks from several stdlib modules and tests.
(Contributed by Nikita Sobolev in 99482.)
* Remove ``_use_broken_old_ctypes_structure_semantics_`` flag
from ctypes module.
(Contributed by Nikita Sobolev in 99285.)

[python_165] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Porting to Python 3.12

Sumber: ht

This section lists previously described changes and other bugfixes
that may require changes to your code.

[python_166] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Changes in the Python API

Sumber: ht

* More strict rules are now applied for numerical group references and
group names in regular expressions.
Only sequence of ASCII digits is now accepted as a numerical reference.
The group name in bytes patterns and replacement strings can now only
contain ASCII letters and digits and underscore.
(Contributed by Serhiy Storchaka in 91760.)
* Remove ``randrange()`` functionality deprecated since Python 3.10. Formerly,
``randrange(10.0)`` losslessly converted to ``randrange(10)``. Now, it raises a

TypeError. Also, the exception raised for non-integer values such as
``randrange(10.5)`` or ``randrange('10')`` has been changed from ValueError to
TypeError. This also prevents bugs where ``randrange(1e25)`` would silently
select from a larger range than ``randrange(10**25)``.
(Originally suggested by Serhiy Storchaka 86388.)
* argparse.ArgumentParser changed encoding and error handler
for reading arguments from file (e.g. ``fromfile_prefix_chars`` option)
from default text encoding (e.g. locale.getpreferredencoding)
to filesystem encoding and error handler.
Argument files should be encoded in UTF-8 instead of ANSI Codepage on Windows.
* Remove the ``asyncore``-based ``smtpd`` module deprecated in Python 3.4.7
and 3.5.4. A recommended replacement is the
asyncio-based aiosmtpd PyPI module.
* shlex.split: Passing ``None`` for *s* argument now raises an
exception, rather than reading sys.stdin. The feature was deprecated
in Python 3.9.
(Contributed by Victor Stinner in 94352.)
* The os module no longer accepts bytes-like paths, like
bytearray and memoryview types: only the exact
bytes type is accepted for bytes strings.
(Contributed by Victor Stinner in 98393.)
* syslog.openlog and syslog.closelog now fail if used in subinterpreters.
syslog.syslog may still be used in subinterp

[python_167] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Build Changes

Sumber: ht

* Python no longer uses setup.py to build shared C extension modules.
Build parameters like headers and libraries are detected in ``configure``
script. Extensions are built by Makefile. Most extensions use
``pkg-config`` and fall back to manual detection.
(Contributed by Christian Heimes in 93939.)
* ``va_start()`` with two parameters, like ``va_start(args, format),``
is now required to build Python.
``va_start()`` is no longer called with a single parameter.
(Contributed by Kumar Aditya in 93207.)
* CPython now uses the ThinLTO option as the default link time optimization policy
if the Clang compiler accepts the flag.
(Contributed by Donghee Na in 89536.)
* Add ``COMPILEALL_OPTS`` variable in Makefile to override compileall
options (default: ``-j0``) in ``make install``. Also merged the 3
``compileall`` commands into a single command to build .pyc files for all
optimization levels (0, 1, 2) at once.
(Contributed by Victor Stinner in 99289.)
* Add platform triplets for 64-bit LoongArch:
* loongarch64-linux-gnusf
* loongarch64-linux-gnuf32
* loongarch64-linux-gnu
(Contributed by Zhang Na in 90656.)
* ``PYTHON_FOR_REGEN`` now require Python 3.10 or newer.
* Autoconf 2.71 and aclocal 1.16.4 is now required to regenerate

configure.
(Contributed by Christian Heimes in 89886.)
* Windows builds and macOS installers from python.org now use OpenSSL 3.0.

[python_168] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): New Features

Sumber: ht

* 697: Introduce the unstable-c-api,
intended for low-level tools like debuggers and JIT compilers.
This API may change in each minor release of CPython without deprecation
warnings.
Its contents are marked by the ``PyUnstable_`` prefix in names.
Code object constructors:
- ``PyUnstable_Code_New()`` (renamed from ``PyCode_New``)
- ``PyUnstable_Code_NewWithPosOnlyArgs()`` (renamed from ``PyCode_NewWithPosOnlyArgs``)
Extra storage for code objects (523):
- ``PyUnstable_Eval_RequestCodeExtraIndex()`` (renamed from ``_PyEval_RequestCodeExtraIndex``)
- ``PyUnstable_Code_GetExtra()`` (renamed from ``_PyCode_GetExtra``)
- ``PyUnstable_Code_SetExtra()`` (renamed from ``_PyCode_SetExtra``)
The original names will continue to be available until the respective
API changes.
(Contributed by Petr Viktorin in 101101.)
* 697: Add an API for extending types whose instance memory layout is
opaque:
- :cPyType_Spec.basicsize can be zero or negative to specify
inheriting or extending the base class size.
- :cPyObject_GetTypeData and :cPyType_GetTypeDataSize
added to allow access to subclass-specific instance data.
- :cPy_TPFLAGS_ITEMS_AT_END and :cPyObject_GetItemData
added to allow safely extending certain variable-sized types, including
:cPyType_Type.
- :cPy_RELATIVE_OFFSET added to allow defining
:cPyMemberDef in terms of a subclass-specific struct.
(Contributed by Petr Viktorin in 103509.)
* Add the new limited-c-api function :cPyType_FromMetaclass,
which generalizes the existing :cPyType_FromModuleAndSpec using
an additional metaclass argument.
(Contributed by Wenzel Jakob in 93012.)
* API for creating objects that can be called using
vectorcall was added to the
stable:
* :cPy_TPFLAGS_HAVE_VECTORCALL
* :cPyVectorcall_NARGS
* :cPyVectorcall_Call
* :cvectorcallfunc
The :cPy_TPFLAGS_HAVE_

[python_169] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Porting to Python 3.12

Sumber: ht

* Legacy Unicode APIs based on ``Py_UNICODE*`` representation has been removed.
Please migrate to APIs based on UTF-8 or ``wchar_t*``.
* Argument parsing functions like :cPyArg_ParseTuple doesn't support
``Py_UNICODE*`` based format (e.g. ``u``, ``Z``) anymore. Please migrate
to other formats for Unicode like ``s``, ``z``, ``es``, and ``U``.
* ``tp_weaklist`` for all static builtin types is always ``NULL``.
This is an internal-only field on ``PyTypeObject``
but we're pointing out the change in case someone happens to be
accessing the field directly anyway. To avoid breakage, consider
using the existing public C-API instead, or, if necessary, the
(internal-only) ``_PyObject_GET_WEAKREFS_LISTPTR()`` macro.
* This internal-only :cPyTypeObject.tp_subclasses may now not be
a valid object pointer. Its type was changed to :cvoid * to
reflect this. We mention this in case someone happens to be accessing the
internal-only field directly.
To get a list of subclasses, call the Python method
:py~type.__subclasses__ (using :cPyObject_CallMethod,
for example).
* Add support of more formatting options (left aligning, octals, uppercase
hexadecimals, :cintmax_t, :cptrdiff_t, :cwchar_t C
strings, variable width and precision) in :cPyUnicode_FromFormat and
:cPyUnicode_FromFormatV.
(Contributed by Serhiy Storchaka in 98836.)
* An unrecognized format character in :cPyUnicode_FromFormat and
:cPyUnicode_FromFormatV now sets a SystemError.
In previous versions it caused all the rest of the format string to be
copied as-is to the result string, and any extra arguments discarded.
(Contributed by Serhiy Storchaka in 95781.)
* Fix wrong sign placement in :cPyUnicode_FromFormat and
:cPyUnicode_FromFormatV.
(Contributed by Philip Georgi in 95504.)
* Extension classes wanting to add a ~object.__di

[python_170] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Deprecated

Sumber: ht

* In accordance with 699, the ``ma_version_tag`` field in :cPyDictObject
is deprecated for extension modules. Accessing this field will generate a compiler
warning at compile time. This field will be removed in Python 3.14.
(Contributed by Ramvikrams and Kumar Aditya in 101193. PEP by Ken Jin.)
* Deprecate global configuration variable:
* :cPy_DebugFlag: use :cPyConfig.parser_debug
* :cPy_VerboseFlag: use :cPyConfig.verbose
* :cPy_QuietFlag: use :cPyConfig.quiet
* :cPy_InteractiveFlag: use :cPyConfig.interactive
* :cPy_InspectFlag: use :cPyConfig.inspect
* :cPy_OptimizeFlag: use :cPyConfig.optimization_level

* :cPy_NoSiteFlag: use :cPyConfig.site_import
* :cPy_BytesWarningFlag: use :cPyConfig.bytes_warning
* :cPy_FrozenFlag: use :cPyConfig.pathconfig_warnings
* :cPy_IgnoreEnvironmentFlag: use :cPyConfig.use_environment
* :cPy_DontWriteBytecodeFlag: use :cPyConfig.write_bytecode
* :cPy_NoUserSiteDirectory: use :cPyConfig.user_site_directory
* :cPy_UnbufferedStdioFlag: use :cPyConfig.buffered_stdio
* :cPy_HashRandomizationFlag: use :cPyConfig.use_hash_seed
and :cPyConfig.hash_seed
* :cPy_IsolatedFlag: use :cPyConfig.isolated
* :cPy_LegacyWindowsFSEncodingFlag: use :cPyPreConfig.legacy_windows_fs_encoding
* :cPy_LegacyWindowsStdioFlag: use :cPyConfig.legacy_windows_stdio
* :cPy_FileSystemDefaultEncoding: use :cPyConfig.filesystem_encoding
* :cPy_HasFileSystemDefaultEncoding: use :cPyConfig.filesystem_encoding
* :cPy_FileSystemDefaultEncodeErrors: use :cPyConfig.filesystem_errors
* :cPy_UTF8Mode: use :cPyPreConfig.utf8_mode (see :cPy_PreInitialize)
The :cPy_InitializeFromConfig API should be used with
:cPyConfig instead.
(Contributed by Victor Stinner in 77782.)
* Creating :cPy_TPFLAGS_IMMUTABLETYPE with mutable
bases is deprecated and will be disabled in Python 3.14. (9538

[python_171] Python - Modern Features - Python 3.12

What's New in Python 3.12 (PEP 695 Type Parameters, F-String Syntax): Removed

Sumber: ht

* Remove the token.h header file. There was never any public tokenizer C
API. The token.h header file was only designed to be used by Python
internals.
(Contributed by Victor Stinner in 92651.)
* Legacy Unicode APIs have been removed. See 623 for detail.
* :cPyUnicode_WCHAR_KIND
* :cPyUnicode_AS_UNICODE
* :cPyUnicode_AsUnicode
* :cPyUnicode_AsUnicodeAndSize
* :cPyUnicode_AS_DATA
* :cPyUnicode_FromUnicode
* :cPyUnicode_GET_SIZE
* :cPyUnicode_GetSize
* :cPyUnicode_GET_DATA_SIZE
* Remove the ``PyUnicode_InternImmortal()`` function macro.
(Contributed by Victor Stinner in 85858.)

[python_172] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Overview

Sumber: ht

What's New In Python 3.13
:Editors: Adam Turner and Thomas Wouters
.. Rules for maintenance:
* Anyone can add text to this document. Do not spend very much time
on the wording of your changes, because your text will probably
get rewritten to some degree.

* The maintainer will go through Misc/NEWS periodically and add
changes; it's therefore more important to add your changes to
Misc/NEWS than to this file.
* This is not a complete list of every single change; completeness
is the purpose of Misc/NEWS. Some changes I consider too small
or esoteric to include. If such a change is added to the text,
I'll just remove it. (This is another reason you shouldn't spend
too much time on writing your addition.)
* If you want to draw your new text to the attention of the
maintainer, add 'XXX' to the beginning of the paragraph or
section.
* It's OK to just add a fragmentary note about a change. For
example: "XXX Describe the transmogrify() function added to the
socket module." The maintainer will research the change and
write the necessary text.
* You can comment out your additions if you like, but it's not
necessary (especially when a final release is some months away).
* Credit the author of a patch or bugfix.   Just the name is
sufficient; the e-mail address isn't necessary.
* It's helpful to add the issue number as a comment:
XXX Describe the transmogrify() function added to the socket
module.
(Contributed by P.Y. Developer in 12345.)
This saves the maintainer the effort of going through the VCS log
when researching a change.
This article explains the new features in Python 3.13, compared to 3.12.
Python 3.13 was released on October 7, 2024.
For full details, see the changelog.
.. seealso

[python_173] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Summary -- Release Highlights

Sumber: ht

.. This section singles out the most important changes in Python 3.13.
Brevity is key.
Python 3.13 is a stable release of the Python programming
language, with a mix of changes to the language, the implementation
and the standard library.
The biggest changes include a new `interactive interpreter
<whatsnew313-better-interactive-interpreter_>`_,
experimental support for running in a `free-threaded mode
<whatsnew313-free-threaded-cpython_>`_ (703),
and a `Just-In-Time compiler <whatsnew313-jit-compiler_>`_ (744).
Error messages continue to improve, with tracebacks now highlighted in color
by default. The locals builtin now has whatsnew313-locals-semantics for changing the returned mapping,
and type parameters now support default values.
The library changes contain removal of deprecated APIs and modules,
as well as the usual improvements in user-friendliness and correctness.
Several legacy standard library modules have now `been removed
<whatsnew313-pep594_>`_ following their deprecation in Python 3.11 (594).

This article doesn't attempt to provide a complete specification
of all new features, but instead gives a convenient overview.
For full details refer to the documentation,
such as the library-index
and reference-index.
To understand the complete implementation and design rationale for a change,
refer to the PEP for a particular new feature;
but note that PEPs usually are not kept up-to-date
once a feature has been fully implemented.
See `Porting to Python 3.13`_ for guidance on upgrading from
earlier versions of Python.

[python_174] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL):

Sumber: ht

.. PEP-sized items next.
Interpreter improvements:
* A greatly improved whatsnew313-better-interactive-interpreter and
whatsnew313-improved-error-messages.
* 667: The locals builtin now has
whatsnew313-locals-semantics when mutating the
returned mapping. Python debuggers and similar tools may now more reliably
update local variables in optimized scopes even during concurrent code
execution.
* 703: CPython 3.13 has experimental support for running with the
global interpreter lock disabled. See whatsnew313-free-threaded-cpython for more details.
* 744: A basic whatsnew313-jit-compiler was added.
It is currently disabled by default (though we may turn it on later).
Performance improvements are modest -- we expect to improve this
over the next few releases.
* Color support in the new whatsnew313-better-interactive-interpreter,
as well as in whatsnew313-improved-error-messages
and whatsnew313-doctest output.
This can be disabled through the PYTHON_COLORS and |NO_COLOR|_
environment variables.
Python data model improvements:
* ~type.__static_attributes__ stores the names of attributes accessed
through ``self.X`` in any function in a class body.
* ~type.__firstlineno__ records the first line number of a class
definition.
Significant improvements in the standard library:
* Add a new PythonFinalizationError exception, raised when an operation
is blocked during interpreter shutdown.
* The argparse module now supports deprecating command-line options,
positional arguments, and subcommands.
* The new functions base64.z85encode and base64.z85decode
support encoding and decoding `Z85 data`_.
* The copy module now has a copy.replace function,
with support for many builtin types and any class defining
the ~object.__replace__ method.
* The new dbm.sqlite3 module is now the default dbm b

[python_175] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): A better interactive interpreter

Sumber: ht

Python now uses a new interactive shell by default, based on code
from the `PyPy project`_.
When the user starts the REPL from an interactive terminal,
the following new features are now supported:
* Multiline editing with history preservation.
* Direct support for REPL-specific commands like help, exit,
and quit, without the need to call them as functions.
* Prompts and tracebacks with using-on-controlling-color.
* Interactive help browsing using F1 with a separate command
history.
* History browsing using F2 that skips output as well as the
:term:`>>>` and ... prompts.
* "Paste mode" with F3 that makes pasting larger blocks of code
easier (press F3 again to return to the regular prompt).
To disable the new interactive shell,
set the PYTHON_BASIC_REPL environment variable.
For more on interactive mode, see tut-interac.
(Contributed by Pablo Galindo Salgado, ?ukasz Langa, and
Lysandros Nikolaou in 111201 based on code from the PyPy project.
Windows support contributed by Dino Viehland and Anthony Shaw.)
.. _`PyPy project`: https://pypy.org/

[python_176] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Improved error messages

Sumber: ht

* The interpreter now uses color by default when displaying tracebacks in the
terminal. This feature using-on-controlling-color
via the new PYTHON_COLORS environment variable as well as
the canonical |NO_COLOR|_ and |FORCE_COLOR|_ environment variables.
(Contributed by Pablo Galindo Salgado in 112730.)
* A common mistake is to write a script with the same name as a
standard library module. When this results in errors, we now
display a more helpful error message:
.. code-block:: pytb
$ python random.py
Traceback (most recent call last):
File "/home/me/random.py", line 1, in <module>
import random
File "/home/me/random.py", line 3, in <module>
print(random.randint(5))
^^^^^^^^^^^^^^
AttributeError: module 'random' has no attribute 'randint' (consider renaming '/home/me/random.py' since it has the same name as the standard library module named 'random' and prevents importing that standard library module)
Similarly, if a script has the same name as a third-party
module that it attempts to import and this results in errors,

we also display a more helpful error message:
.. code-block:: pytb
$ python numpy.py
Traceback (most recent call last):
File "/home/me/numpy.py", line 1, in <module>
import numpy as np
File "/home/me/numpy.py", line 3, in <module>
np.array([1, 2, 3])
^^^^^^^^
AttributeError: module 'numpy' has no attribute 'array' (consider renaming '/home/me/numpy.py' if it has the same name as a library you intended to import)
(Contributed by Shantanu Jain in 95754.)
* The error message now tries to suggest the correct keyword argument
when an incorrect keyword argument is passed to a function.
.. code-block:: pycon
>>> "Better error messages!".split(max_split=1)
Traceback (most recent call last):
File "<python-input-0>", line 1, in <module>
"Better error messages!".spl

[python_177] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Free-threaded CPython

Sumber: ht

CPython now has experimental support for running in a free-threaded mode,
with the global interpreter lock (GIL) disabled.
This is an experimental feature and therefore is not enabled by default.
The free-threaded mode requires a different executable,
usually called ``python3.13t`` or ``python3.13t.exe``.
Pre-built binaries marked as *free-threaded* can be installed as part of
the official install-freethreaded-windows
and install-freethreaded-macos installers,
or CPython can be built from source with the --disable-gil option.
Free-threaded execution allows for full utilization of the available
processing power by running threads in parallel on available CPU cores.
While not all software will benefit from this automatically, programs
designed with threading in mind will run faster on multi-core hardware.
The free-threaded mode is experimental and work is ongoing to improve it:
expect some bugs and a substantial single-threaded performance hit.
Free-threaded builds of CPython support optionally running with the GIL
enabled at runtime using the environment variable PYTHON_GIL or
the command-line option -X gil=1.
To check if the current interpreter supports free-threading, -V
and sys.version contain "experimental free-threading build".
The new sys._is_gil_enabled function can be used to check whether
the GIL is actually disabled in the running process.
C-API extension modules need to be built specifically for the free-threaded
build. Extensions that support running with the GIL disabled should
use the :cPy_mod_gil slot. Extensions using single-phase init should
use :cPyUnstable_Module_SetGIL to indicate whether they support
running with the GIL disabled. Importing C extensions that don't use these

mechanisms will cause the GIL to be enabled, unless the GIL was explicitl

[python_178] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): An experimental just-in-time (JIT) compiler

Sumber: ht

When CPython is configured and built using
the --enable-experimental-jit option,
a just-in-time (JIT) compiler is added which may speed up some Python programs.
On Windows, use ``PCbuild/build.bat --experimental-jit`` to enable the JIT
or ``--experimental-jit-interpreter`` to enable the Tier 2 interpreter.
Build requirements and further supporting information `are contained at`__
Tools/jit/README.md.
__ https://github.com/python/cpython/blob/main/Tools/jit/README.md
The --enable-experimental-jit option takes these (optional) values,
defaulting to ``yes`` if --enable-experimental-jit is present
without the optional value.
* ``no``: Disable the entire Tier 2 and JIT pipeline.
* ``yes``: Enable the JIT.
To disable the JIT at runtime, pass the environment variable ``PYTHON_JIT=0``.
* ``yes-off``: Build the JIT but disable it by default.
To enable the JIT at runtime, pass the environment variable ``PYTHON_JIT=1``.
* ``interpreter``: Enable the Tier 2 interpreter but disable the JIT.
The interpreter can be disabled by running with ``PYTHON_JIT=0``.
The internal architecture is roughly as follows:
* We start with specialized *Tier 1 bytecode*.
See whatsnew311-pep659 for details.
* When the Tier 1 bytecode gets hot enough, it gets translated
to a new purely internal intermediate representation (IR),
called the *Tier 2 IR*, and sometimes referred to as micro-ops ("uops").
* The Tier 2 IR uses the same stack-based virtual machine as Tier 1,
but the instruction format is better suited to translation to machine code.
* We have several optimization passes for Tier 2 IR, which are applied
before it is interpreted or translated to machine code.
* There is a Tier 2 interpreter, but it is mostly intended for debugging
the earlier stages of the optimization pipeline.
The Tier 2 interpret

[python_179] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Defined mutation semantics for :pylocals

Sumber: ht

Historically, the expected result of mutating the return value of
locals has been left to individual Python implementations to define.
Starting from Python 3.13, 667 standardises
the historical behavior of CPython for most code execution scopes,
but changes optimized scope
(functions, generators, coroutines, comprehensions, and generator expressions)
to explicitly return independent snapshots of the currently assigned local
variables, including locally referenced nonlocal variables captured in closures.

This change to the semantics of locals in optimized scopes also
affects the default behavior of code execution functions that implicitly
target locals if no explicit namespace is provided
(such as exec and eval).
In previous versions, whether or not changes could be accessed by calling
locals after calling the code execution function was
implementation-dependent. In CPython specifically, such code would typically
appear to work as desired, but could sometimes fail in optimized scopes based
on other code (including debuggers and code execution tracing tools)
potentially resetting the shared snapshot in that scope.
Now, the code will always run against an independent snapshot of
the local variables in optimized scopes, and hence the changes will never
be visible in subsequent calls to locals.
To access the changes made in these cases, an explicit namespace reference
must now be passed to the relevant function.
Alternatively, it may make sense to update affected code to use a higher level
code execution API that returns the resulting code execution namespace
(e.g. runpy.run_path when executing Python files from disk).
To ensure debuggers and similar tools can reliably update local variables in
scopes affected by this change, frame.f_locals now
returns a write-through proxy

[python_180] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Support for mobile platforms

Sumber: ht

730: iOS is now a 11 supported platform, with the
``arm64-apple-ios`` and ``arm64-apple-ios-simulator`` targets at tier 3
(iPhone and iPad devices released after 2013 and the Xcode iOS simulator
running on Apple silicon hardware, respectively).
``x86_64-apple-ios-simulator``
(the Xcode iOS simulator running on older ``x86_64`` hardware)
is not a tier 3 supported platform, but will have best-effort support.
(PEP written and implementation contributed by Russell Keith-Magee in
114099.)
738: Android is now a 11 supported platform, with the
``aarch64-linux-android`` and ``x86_64-linux-android`` targets at tier 3.
The 32-bit targets ``arm-linux-androideabi`` and ``i686-linux-android``
are not tier 3 supported platforms, but will have best-effort support.
(PEP written and implementation contributed by Malcolm Smith in
116622.)

[python_181] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Other Language Changes

Sumber: ht

* The compiler now strips common leading whitespace
from every line in a docstring.
This reduces the size of the bytecode
(such as ``.pyc`` files), with reductions in file size of around 5%,

for example in sqlalchemy.orm.session from SQLAlchemy 2.0.
This change affects tools that use docstrings, such as doctest.
.. doctest
* annotation-scopes within class scopes
can now contain lambdas and comprehensions.
Comprehensions that are located within class scopes
are not inlined into their parent scope.
.. code-block:: python
class C[T]:
type Alias = lambda: T
(Contributed by Jelle Zijlstra in 109118 and 118160.)
* future are no longer triggered by
relative imports of the __future__ module,
meaning that statements of the form ``from .__future__ import ...``
are now simply standard relative imports, with no special features activated.
(Contributed by Jeremiah Gabriel Pascual in 118216.)
* global declarations are now permitted in except blocks
when that global is used in the else block.
Previously this raised an erroneous SyntaxError.
(Contributed by Irit Katriel in 111123.)
* Add PYTHON_FROZEN_MODULES, a new environment variable that
determines whether frozen modules are ignored by the import machinery,
equivalent to the -X command-line option.
(Contributed by Yilei Yang in 111374.)
* Add perf_profiling working
without `frame pointers <https://en.wikipedia.org/wiki/Call_stack>`_ through
the new environment variable PYTHON_PERF_JIT_SUPPORT
and command-line option -X.
(Contributed by Pablo Galindo in 118518.)
* The location of a .python_history file can be changed via the
new PYTHON_HISTORY environment variable.
(Contributed by Levi Sabah, Zackery Spytz and Hugo van Kemenade
in 73965.)
* Classes have a new ~type.__static_attributes__ attribute.
This is populated by the compiler w

Contoh Kode / Perintah CLI:

```
>>> def spam():
...     """
...         This is a docstring with
...           leading whitespace.
...
...         It even has multiple paragraphs!
...     """
...
>>> spam.__doc__
'\nThis is a docstring with\n  leading whitespace.\n\nIt even has multiple paragraphs!\n'
(Contributed by Inada Naoki in :gh:`81283`.)
```

[python_182] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): New Modules

Sumber: ht

* dbm.sqlite3: An SQLite backend for dbm.
(Contributed by Raymond Hettinger and Erlend E. Aasland in 100414.)

[python_183] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): argparse

Sumber: ht

* Add the *deprecated* parameter to the
~argparse.ArgumentParser.add_argument
and add_parser methods, to enable deprecating
command-line options, positional arguments, and subcommands.
(Contributed by Serhiy Storchaka in 83648.)

[python_184] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): array

Sumber: ht

* Add the ``'w'`` type code (``Py_UCS4``) for Unicode characters.
It should be used instead of the deprecated ``'u'`` type code.
(Contributed by Inada Naoki in 80480.)
* Register array.array as a ~collections.abc.MutableSequence
by implementing the ~array.array.clear method.
(Contributed by Mike Zimin in 114894.)

[python_185] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): ast

Sumber: ht

* The constructors of node types in the ast module are now
stricter in the arguments they accept,
with more intuitive behavior when arguments are omitted.
If an optional field on an AST node is not included as an argument when
constructing an instance, the field will now be set to ``None``. Similarly,
if a list field is omitted, that field will now be set to an empty list,
and if an expr_context field is omitted, it defaults to
ast.Load.
(Previously, in all cases, the attribute would be missing on the newly
constructed AST node instance.)
In all other cases, where a required argument is omitted,
the node constructor will emit a DeprecationWarning.
This will raise an exception in Python 3.15.
Similarly, passing a keyword argument to the constructor
that does not map to a field on the AST node is now deprecated,
and will raise an exception in Python 3.15.
These changes do not apply to user-defined subclasses of ast.AST
unless the class opts in to the new behavior
by defining the .AST._field_types mapping.
(Contributed by Jelle Zijlstra in 105858, 117486, and 118851.)
* ast.parse now accepts an optional argument *optimize*
which is passed on to compile.

This makes it possible to obtain an optimized AST.
(Contributed by Irit Katriel in 108113.)

[python_186] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): asyncio

Sumber: ht

* asyncio.as_completed now returns an object that is both an
asynchronous iterator and a plain iterator
of awaitable.
The awaitables yielded by asynchronous iteration include original task
or future objects that were passed in,
making it easier to associate results with the tasks being completed.
(Contributed by Justin Arthur in 77714.)
* asyncio.loop.create_unix_server will now automatically remove
the Unix socket when the server is closed.
(Contributed by Pierre Ossman in 111246.)
* .DatagramTransport.sendto will now send zero-length
datagrams if called with an empty bytes object.
The transport flow control also now accounts for the datagram header
when calculating the buffer size.
(Contributed by Jamie Phan in 115199.)
* Add asyncio.Queue.shutdown
and ~asyncio.QueueShutDown to manage queue termination.
(Contributed by Laurie Opperman and Yves Duprat in 104228.)
* Add the .Server.close_clients and .Server.abort_clients
methods, which more forcefully close an asyncio server.
(Contributed by Pierre Ossman in 113538.)
* Accept a tuple of separators in .StreamReader.readuntil,
stopping when any one of them is encountered.
(Contributed by Bruce Merry in 81322.)
* Improve the behavior of ~asyncio.TaskGroup when
an external cancellation collides with an internal cancellation.
For example, when two task groups are nested
and both experience an exception in a child task simultaneously,
it was possible that the outer task group would hang,
because its internal cancellation was swallowed by the inner task group.
In the case where a task group is cancelled externally
and also must raise an ExceptionGroup,
it will now call the parent task's ~asyncio.Task.cancel method.
This ensures that a ~asyncio.CancelledError will be raised
at the next await, so the cancellation is not lost.
An

[python_187] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): base64

Sumber: ht

* Add ~base64.z85encode and ~base64.z85decode functions
for encoding bytes as `Z85 data`_
and decoding Z85-encoded data to bytes.
(Contributed by Matan Perelman in 75299.)
.. _Z85 data: https://rfc.zeromq.org/spec/32/

[python_188] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): compileall

Sumber: ht

* The default number of worker threads and processes is now selected using
os.process_cpu_count instead of os.cpu_count.
(Contributed by Victor Stinner in 109649.)

[python_189] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): concurrent.futures

Sumber: ht

* The default number of worker threads and processes is now selected using
os.process_cpu_count instead of os.cpu_count.
(Contributed by Victor Stinner in 109649.)

[python_190] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): configparser

Sumber: ht

* ~configparser.ConfigParser now has support for unnamed sections,
which allows for top-level key-value pairs.
This can be enabled with the new *allow_unnamed_section* parameter.
(Contributed by Pedro Sousa Lacerda in 66449.)

[python_191] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): copy

Sumber: ht

* The new ~copy.replace function and the object.__replace__ make creating modified copies of objects much simpler.
This is especially useful when working with immutable objects.
The following types support the ~copy.replace function
and implement the replace protocol:
* collections.namedtuple
* dataclasses.dataclass
* datetime.datetime, datetime.date, datetime.time
* inspect.Signature, inspect.Parameter
* types.SimpleNamespace
* code-objects
Any user-defined class can also support copy.replace by defining
the ~object.__replace__ method.
(Contributed by Serhiy Storchaka in 108751.)

[python_192] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): ctypes

Sumber: ht

* As a consequence of necessary internal refactoring, initialization of
internal metaclasses now happens in ``__init__`` rather
than in ``__new__``. This affects projects that subclass these internal
metaclasses to provide custom initialization.
Generally:
- Custom logic that was done in ``__new__`` after calling ``super().__new__``

should be moved to ``__init__``.
- To create a class, call the metaclass, not only the metaclass's
``__new__`` method.
See 124520 for discussion and links to changes in some affected
projects.
* ctypes.Structure objects have a new ~ctypes.Structure._align_
attribute which allows the alignment of the structure being packed to/from
memory to be specified explicitly.
(Contributed by Matt Sanderson in 112433)

[python_193] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): dbm

Sumber: ht

* Add dbm.sqlite3, a new module which implements an SQLite backend,
and make it the default dbm backend.
(Contributed by Raymond Hettinger and Erlend E. Aasland in 100414.)
* Allow removing all items from the database through
the new clear methods of the GDBM and NDBM database objects.
(Contributed by Donghee Na in 107122.)

[python_194] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): dis

Sumber: ht

* Change the output of dis module functions to show logical
labels for jump targets and exception handlers, rather than offsets.
The offsets can be added with the new
dis --show-offsets command-line option
or the *show_offsets* argument.
(Contributed by Irit Katriel in 112137.)
* ~dis.get_instructions no longer represents cache entries
as separate instructions.
Instead, it returns them as part of the ~dis.Instruction,
in the new *cache_info* field.
The *show_caches* argument to ~dis.get_instructions is deprecated
and no longer has any effect.
(Contributed by Irit Katriel in 112962.)

[python_195] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): doctest

Sumber: ht

* doctest output is now colored by default.
This can be controlled via the new PYTHON_COLORS environment
variable as well as the canonical |NO_COLOR|_
and |FORCE_COLOR|_ environment variables.
See also using-on-controlling-color.
(Contributed by Hugo van Kemenade in 117225.)
* The .DocTestRunner.run method now counts the number of skipped tests.
Add the .DocTestRunner.skips and .TestResults.skipped attributes.
(Contributed by Victor Stinner in 108794.)

[python_196] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): email

Sumber: ht

* Headers with embedded newlines are now quoted on output.
The ~email.generator will now refuse to serialize (write) headers
that are improperly folded or delimited, such that they would be parsed as
multiple headers or joined with adjacent data.
If you need to turn this safety feature off,
set ~email.policy.Policy.verify_generated_headers.
(Contributed by Bas Bloemsaat and Petr Viktorin in 121650.)
* ~email.utils.getaddresses and ~email.utils.parseaddr now
return ``('', '')`` pairs in more situations where invalid email addresses
are encountered instead of potentially inaccurate values.
The two functions have a new optional *strict* parameter (default ``True``).
To get the old behavior (accepting malformed input), use ``strict=False``.
``getattr(email.utils, 'supports_strict_parsing', False)`` can be used
to check if the *strict* parameter is available.
(Contributed by Thomas Dwyer and Victor Stinner for 102988 to improve
the 2023-27043 fix.)

[python_197] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): enum

Sumber: ht

* ~enum.EnumDict has been made public to better support subclassing
~enum.EnumType.

[python_198] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): fractions

Sumber: ht

* ~fractions.Fraction objects now support the standard
formatspec rules
for fill, alignment, sign handling, minimum width, and grouping.
(Contributed by Mark Dickinson in 111320.)

[python_199] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): glob

Sumber: ht

* Add ~glob.translate, a function to convert a path specification
with shell-style wildcards to a regular expression.
(Contributed by Barney Gale in 72904.)

[python_200] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): importlib

Sumber: ht

* The following functions in importlib.resources now allow accessing
a directory (or tree) of resources, using multiple positional arguments
(the *encoding* and *errors* arguments in the text-reading functions
are now keyword-only):
* ~importlib.resources.is_resource

* ~importlib.resources.open_binary
* ~importlib.resources.open_text
* ~importlib.resources.path
* ~importlib.resources.read_binary
* ~importlib.resources.read_text

These functions are no longer deprecated and are not scheduled for removal.
(Contributed by Petr Viktorin in 116608.)

* ~importlib.resources.contents remains deprecated in favor of
the fully-featured ~importlib.resources.abc.Traversable API.
However, there is now no plan to remove it.
(Contributed by Petr Viktorin in 116608.)

io

--

* The ~io.IOBase finalizer now logs any errors raised by
the ~io.IOBase.close method with sys.unraisablehook.
Previously, errors were ignored silently by default,
and only logged in devmode
or when using a debug-build.
(Contributed by Victor Stinner in 62948.)

[python_201] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): ipaddress

Sumber: ht

* Add the .IPv4Address.ipv6_mapped property,
which returns the IPv4-mapped IPv6 address.
(Contributed by Charles Machalow in 109466.)

* Fix ``is_global`` and ``is_private`` behavior in
~ipaddress.IPv4Address, ~ipaddress.IPv6Address,
~ipaddress.IPv4Network, and ~ipaddress.IPv6Network.
(Contributed by Jakub Stasiak in 113171.)

[python_202] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): itertools

Sumber: ht

* ~itertools.batched has a new *strict* parameter,
which raises a ValueError if the final batch is shorter
than the specified batch size.
(Contributed by Raymond Hettinger in 113202.)

[python_203] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): marshal

Sumber: ht

* Add the *allow_code* parameter in module functions.
Passing ``allow_code=False`` prevents serialization and de-serialization
of code objects which are incompatible between Python versions.
(Contributed by Serhiy Storchaka in 113626.)

[python_204] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): math

Sumber: ht

* The new function ~math.fma performs fused multiply-add operations.
This computes ``x * y + z`` with only a single round,
and so avoids any intermediate loss of precision.
It wraps the ``fma()`` function provided by C99,
and follows the specification of the IEEE 754 "fusedMultiplyAdd" operation
for special cases.
(Contributed by Mark Dickinson and Victor Stinner in 73468.)

[python_205] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): mimetypes

Sumber: ht

* Add the ~mimetypes.guess_file_type function to guess a MIME type
from a filesystem path.
Using paths with ~mimetypes.guess_type is now soft deprecated.
(Contributed by Serhiy Storchaka in 66543.)

[python_206] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): mmap

Sumber: ht

* ~mmap.mmap is now protected from crashing on Windows when the
mapped memory is inaccessible due to file system errors or access violations.
(Contributed by Jannis Weigend in 118209.)
* ~mmap.mmap has a new ~mmap.mmap.seekable method
that can be used when a seekable file-like object is required.
The ~mmap.mmap.seek method now returns the new absolute position.
(Contributed by Donghee Na and Sylvie Liberman in 111835.)
* The new UNIX-only *trackfd* parameter for ~mmap.mmap controls
file descriptor duplication;
if false, the file descriptor specified by *fileno* will not be duplicated.
(Contributed by Zackery Spytz and Petr Viktorin in 78502.)

[python_207] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): multiprocessing

Sumber: ht

* The default number of worker threads and processes is now selected using
os.process_cpu_count instead of os.cpu_count.
(Contributed by Victor Stinner in 109649.)
os
--
* Add ~os.process_cpu_count function to get the number
of logical CPU cores usable by the calling thread of the current process.
(Contributed by Victor Stinner in 109649.)
* ~os.cpu_count and ~os.process_cpu_count can be overridden
through the new environment variable PYTHON_CPU_COUNT
or the new command-line option -X.

This option is useful for users who need to limit CPU resources
of a container system without having to modify application code
or the container itself.
(Contributed by Donghee Na in 109595.)
* Add a os-timerfd to Linux's
timerfd_create(2)
via ~os.timerfd_create,
~os.timerfd_settime, ~os.timerfd_settime_ns,
~os.timerfd_gettime, ~os.timerfd_gettime_ns,
~os.TFD_NONBLOCK, ~os.TFD_CLOEXEC,
~os.TFD_TIMER_ABSTIME, and ~os.TFD_TIMER_CANCEL_ON_SET
(Contributed by Masaru Tsuchiyama in 108277.)
* ~os.lchmod and the *follow_symlinks* argument of ~os.chmod
are both now available on Windows.
Note that the default value of *follow_symlinks*
in lchmod is ``False`` on Windows.
(Contributed by Serhiy Storchaka in 59616.)
* ~os.fchmod and support for file descriptors in ~os.chmod
are both now available on Windows.
(Contributed by Serhiy Storchaka in 113191.)
* On Windows, ~os.mkdir and ~os.makedirs now support passing
a *mode* value of ``0o700`` to apply access control to the new directory.
This implicitly affects tempfile.mkdtemp
and is a mitigation for 2024-4030.
Other values for *mode* continue to be ignored.
(Contributed by Steve Dower in 118486.)
* ~os.posix_spawn now accepts ``None`` for the *env* argument,
which makes the newly spawned process use the current process environment.
(Contributed by Jak

[python_208] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): os.path

Sumber: ht

* Add ~os.path.isreserved to check if a path is reserved
on the current system.
This function is only available on Windows.
(Contributed by Barney Gale in 88569.)
* On Windows, ~os.path.isabs no longer considers paths
starting with exactly one slash (``\`` or ``/``) to be absolute.
(Contributed by Barney Gale and Jon Foster in 44626.)
* ~os.path.realpath now resolves MS-DOS style file names
even if the file is not accessible.
(Contributed by Moonsik Park in 82367.)

[python_209] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): pathlib

Sumber: ht

* Add ~pathlib.UnsupportedOperation, which is raised instead of
NotImplementedError when a path operation isn't supported.
(Contributed by Barney Gale in 89812.)
* Add a new constructor for creating ~pathlib.Path objects
from 'file' URIs (``file:///``), .Path.from_uri.
(Contributed by Barney Gale in 107465.)
* Add .PurePath.full_match for matching paths with
shell-style wildcards, including the recursive wildcard "``**``".
(Contributed by Barney Gale in 73435.)
* Add the .PurePath.parser class attribute to store the
implementation of os.path used
for low-level path parsing and joining.
This will be either posixpath or ntpath.
* Add *recurse_symlinks* keyword-only argument to
.Path.glob and ~pathlib.Path.rglob.
(Contributed by Barney Gale in 77609.)
* .Path.glob and ~pathlib.Path.rglob
now return files and directories when given a pattern that ends with "``**``".
Previously, only directories were returned.
(Contributed by Barney Gale in 70303.)
* Add the *follow_symlinks* keyword-only argument to
pathlib.Path.is_file,
pathlib.Path.is_dir,
.Path.owner, and .Path.group.
(Contributed by Barney Gale in 105793 and Kamil Turek in 107962.)

[python_210] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): pdb

Sumber: ht

* breakpoint and ~pdb.set_trace now enter the debugger immediately
rather than on the next line of code to be executed. This change prevents the
debugger from breaking outside of the context when breakpoint is positioned
at the end of the context.
(Contributed by Tian Gao in 118579.)
* ``sys.path[0]`` is no longer replaced by the directory of the script
being debugged when sys.flags.safe_path is set.
(Contributed by Tian Gao and Christian Walther in 111762.)
* zipapp is now supported as a debugging target.
(Contributed by Tian Gao in 118501.)
* Add ability to move between chained exceptions during
post-mortem debugging in ~pdb.pm using
the new exceptions command for Pdb.
(Contributed by Matthias Bussonnier in 106676.)
* Expressions and statements whose prefix is a pdb command are now correctly
identified and executed.
(Contributed by Tian Gao in 108464.)

[python_211] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): queue

Sumber: ht

* Add queue.Queue.shutdown and ~queue.ShutDown
to manage queue termination.
(Contributed by Laurie Opperman and Yves Duprat in 104750.)

[python_212] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): random

Sumber: ht

* Add a random-cli.
(Contributed by Hugo van Kemenade in 118131.)
re
--
* Rename re.error to ~re.PatternError for improved clarity.
re.error is kept for backward compatibility.

[python_213] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): shutil

Sumber: ht

* Support the *dir_fd* and *follow_symlinks* keyword arguments
in ~shutil.chown.
(Contributed by Berker Peksag and Tahia K in 62308)

[python_214] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): site

Sumber: ht

* .pth files are now decoded using UTF-8 first,
and then with the locale encoding if UTF-8 decoding fails.
(Contributed by Inada Naoki in 117802.)

[python_215] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): sqlite3

Sumber: ht

* A ResourceWarning is now emitted if a ~sqlite3.Connection
object is not sqlite3.Connection.close explicitly.
(Contributed by Erlend E. Aasland in 105539.)
* Add the *filter* keyword-only parameter to .Connection.iterdump
for filtering database objects to dump.
(Contributed by Mariusz Felisiak in 91602.)

[python_216] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): ssl

Sumber: ht

* The ~ssl.create_default_context API now includes
~ssl.VERIFY_X509_PARTIAL_CHAIN and ~ssl.VERIFY_X509_STRICT
in its default flags.
.. note

Contoh Kode / Perintah CLI:

```
:data:`~ssl.VERIFY_X509_STRICT` may reject pre-:rfc:`5280`
or malformed certificates that the underlying OpenSSL implementation
might otherwise accept.
Whilst disabling this is not recommended, you can do so using:
.. code-block:: python
import ssl
ctx = ssl.create_default_context()
ctx.verify_flags &= ~ssl.VERIFY_X509_STRICT
(Contributed by William Woodruff in :gh:`112389`.)
```

[python_217] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): statistics

Sumber: ht

* Add ~statistics.kde for kernel density estimation.
This makes it possible to estimate a continuous probability density function
from a fixed number of discrete samples.
(Contributed by Raymond Hettinger in 115863.)
* Add ~statistics.kde_random for sampling from an
estimated probability density function created by ~statistics.kde.
(Contributed by Raymond Hettinger in 115863.)

[python_218] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): subprocess

Sumber: ht

* The subprocess module now uses the ~os.posix_spawn function in
more situations.
Notably, when *close_fds* is ``True`` (the default),
~os.posix_spawn will be used when the C library provides
:cposix_spawn_file_actions_addclosefrom_np,
which includes recent versions of Linux, FreeBSD, and Solaris.
On Linux, this should perform similarly to the existing
Linux :cvfork based code.
A private control knob subprocess._USE_POSIX_SPAWN can
be set to ``False`` if you need to force subprocess
to never use ~os.posix_spawn.
Please report your reason and platform details in
the using-the-tracker if you set this
so that we can improve our API selection logic for everyone.
(Contributed by Jakub Kulik in 113117.)

[python_219] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): sys

Sumber: ht

* Add the ~sys._is_interned function to test if a string was interned.
This function is not guaranteed to exist in all implementations of Python.
(Contributed by Serhiy Storchaka in 78573.)

[python_220] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): tempfile

Sumber: ht

* On Windows, the default mode ``0o700`` used by tempfile.mkdtemp now
limits access to the new directory due to changes to os.mkdir.
This is a mitigation for 2024-4030.
(Contributed by Steve Dower in 118486.)

[python_221] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): time

Sumber: ht

* On Windows, ~time.monotonic now uses the
``QueryPerformanceCounter()`` clock for a resolution of 1 microsecond,
instead of the ``GetTickCount64()`` clock which has
a resolution of 15.6 milliseconds.
(Contributed by Victor Stinner in 88494.)
* On Windows, ~time.time now uses the
``GetSystemTimePreciseAsFileTime()`` clock for a resolution of 1 microsecond,
instead of the ``GetSystemTimeAsFileTime()`` clock which has
a resolution of 15.6 milliseconds.
(Contributed by Victor Stinner in 63207.)

[python_222] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): tkinter

Sumber: ht

* Add tkinter widget methods:
tk_busy_hold, tk_busy_configure,
tk_busy_cget, tk_busy_forget,
tk_busy_current, and tk_busy_status.
(Contributed by Miguel, klappnase and Serhiy Storchaka in 72684.)
* The tkinter widget method wm_attributes now accepts
the attribute name without the minus prefix to get window attributes,
for example ``w.wm_attributes('alpha')``
and allows specifying attributes and values to set as keyword arguments,
for example ``w.wm_attributes(alpha=0.5)``.
(Contributed by Serhiy Storchaka in 43457.)
* wm_attributes can now return attributes as a dict,
by using the new optional keyword-only parameter *return_python_dict*.
(Contributed by Serhiy Storchaka in 43457.)
* Text.count can now return a simple int
when the new optional keyword-only parameter *return_ints* is used.
Otherwise, the single count is returned as a 1-tuple or ``None``.
(Contributed by Serhiy Storchaka in 97928.)

* Support the "vsapi" element type in
the ~tkinter.ttk.Style.element_create method of
tkinter.ttk.Style.
(Contributed by Serhiy Storchaka in 68166.)
* Add the after_info method for Tkinter widgets.
(Contributed by Cheryl Sabella in 77020.)
* Add a new copy_replace method to PhotoImage
to copy a region from one image to another,
possibly with pixel zooming, subsampling, or both.
(Contributed by Serhiy Storchaka in 118225.)
* Add *from_coords* parameter to the PhotoImage methods
copy, zoom and subsample.
Add *zoom* and *subsample* parameters to the PhotoImage method
copy.
(Contributed by Serhiy Storchaka in 118225.)
* Add the PhotoImage methods
read to read an image from a file
and data to get the image data.
Add *background* and *grayscale* parameters to the write method.
(Contributed by Serhiy Storchaka in 118271.)

[python_223] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): traceback

Sumber: ht

* Add the ~traceback.TracebackException.exc_type_str attribute
to ~traceback.TracebackException,
which holds a string display of the *exc_type*.
Deprecate the ~traceback.TracebackException.exc_type attribute,
which holds the type object itself.
Add parameter *save_exc_type* (default ``True``)
to indicate whether ``exc_type`` should be saved.
(Contributed by Irit Katriel in 112332.)
* Add a new *show_group* keyword-only parameter to
.TracebackException.format_exception_only to (recursively) format
the nested exceptions of a BaseExceptionGroup instance.
(Contributed by Irit Katriel in 105292.)

[python_224] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): types

Sumber: ht

* ~types.SimpleNamespace can now take a single positional argument
to initialise the namespace's arguments.
This argument must either be a mapping or an iterable of key-value pairs.
(Contributed by Serhiy Storchaka in 108191.)

[python_225] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): typing

Sumber: ht

* 705: Add ~typing.ReadOnly, a special typing construct
to mark a ~typing.TypedDict item as read-only for type checkers.
* 742: Add ~typing.TypeIs, a typing construct
that can be used to instruct a type checker how to narrow a type.
* Add ~typing.NoDefault, a sentinel object used to represent
the defaults of some parameters in the typing module.
(Contributed by Jelle Zijlstra in 116126.)
* Add ~typing.get_protocol_members to return the set of members
defining a typing.Protocol.
(Contributed by Jelle Zijlstra in 104873.)
* Add ~typing.is_protocol to check whether a class
is a ~typing.Protocol.
(Contributed by Jelle Zijlstra in 104873.)
* ~typing.ClassVar can now be nested in ~typing.Final,
and vice versa.
(Contributed by Mehdi Drissi in 89547.)

[python_226] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): unicodedata

Sumber: ht

* Update the Unicode database to `version 15.1.0`__.
(Contributed by James Gerity in 109559.)
__ https://www.unicode.org/versions/Unicode15.1.0/

[python_227] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): venv

Sumber: ht

* Add support for creating source control management (SCM) ignore files
in a virtual environment's directory.
By default, Git is supported.
This is implemented as opt-in via the API,
which can be extended to support other SCMs
(~venv.EnvBuilder and ~venv.create),
and opt-out via the CLI, using --without-scm-ignore-files.
(Contributed by Brett Cannon in 108125.)

[python_228] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): warnings

Sumber: ht

* 702: The new warnings.deprecated decorator provides a way to
communicate deprecations to a static type checker
and to warn on usage of deprecated classes and functions.
A DeprecationWarning may also be emitted when
a decorated function or class is used at runtime.
(Contributed by Jelle Zijlstra in 104003.)

[python_229] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): xml

Sumber: ht

* Allow controlling Expat >=2.6.0 reparse deferral (2023-52425)
by adding five new methods:
* xml.etree.ElementTree.XMLParser.flush
* xml.etree.ElementTree.XMLPullParser.flush
* xml.parsers.expat.xmlparser.GetReparseDeferralEnabled
* xml.parsers.expat.xmlparser.SetReparseDeferralEnabled
* xml.sax.expatreader.ExpatParser.flush
(Contributed by Sebastian Pipping in 115623.)
* Add the close method for the iterator returned by
~xml.etree.ElementTree.iterparse for explicit cleanup.
(Contributed by Serhiy Storchaka in 69893.)

[python_230] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): zipimport

Sumber: ht

* Add support for ZIP64_ format files.
Everybody loves huge data, right?
(Contributed by Tim Hatch in 94146.)
.. _ZIP64: https://en.wikipedia.org/wiki/Zip_(file_format)#ZIP64

[python_231] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Optimizations

Sumber: ht

* Several standard library modules have had
their import times significantly improved.
For example, the import time of the typing module
has been reduced by around a third by removing dependencies
on re and contextlib.
Other modules to enjoy import-time speedups include
email.utils, enum, functools,
importlib.metadata, and threading.
(Contributed by Alex Waygood, Shantanu Jain, Adam Turner, Daniel Hollas,
and others in 109653.)
* textwrap.indent is now around 30% faster than before for large input.
(Contributed by Inada Naoki in 107369.)
* The subprocess module now uses the ~os.posix_spawn function in
more situations, including when *close_fds* is ``True`` (the default)
on many modern platforms.
This should provide a notable performance increase
when launching processes on FreeBSD and Solaris.
See the whatsnew313-subprocess section above for details.
(Contributed by Jakub Kulik in 113117.)

[python_232] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): PEP 594: Remove "dead batteries" from the standard library

Sumber: ht

594 proposed removing 19 modules from the standard library,
colloquially referred to as 'dead batteries' due to their
historic, obsolete, or insecure status.
All of the following modules were deprecated in Python 3.11,
and are now removed:
* aifc
* standard-aifc:
Use the redistribution of ``aifc`` library from PyPI.
* audioop
* audioop-lts:
Use ``audioop-lts`` library from PyPI.
* chunk
* standard-chunk:
Use the redistribution of ``chunk`` library from PyPI.
* cgi and cgitb
* cgi.FieldStorage can typically be replaced with
urllib.parse.parse_qsl for ``GET`` and ``HEAD`` requests,
and the email.message module or the multipart library
for ``POST`` and ``PUT`` requests.
* cgi.parse can be replaced by calling
urllib.parse.parse_qs directly on the desired query string,
unless the input is ``multipart/form-data``,
which should be replaced as described below for cgi.parse_multipart.
* cgi.parse_header can be replaced with the functionality
in the email package, which implements the same MIME RFCs.
For example, with email.message.EmailMessage:
.. code-block:: python
from email.message import EmailMessage
msg = EmailMessage()
msg['content-type'] = 'application/json; charset="utf8"'
main, params = msg.get_content_type(), msg['content-type'].params
* cgi.parse_multipart can be replaced with the functionality
in the email package, which implements the same MIME RFCs,
or with the multipart library.
For example, the email.message.EmailMessage
and email.message.Message classes.
* standard-cgi: and standard-cgitb:
Use the redistribution of ``cgi`` and ``cgitb`` library from PyPI.
* crypt and the private _crypt extension.
The hashlib module may be an appropriate replacement
when simply hashing a value is required.
Otherwise, various third-party libraries on PyPI are available:
* bcrypt:

[python_233] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): 2to3

Sumber: ht

* Remove the 2to3 program and the lib2to3 module,
previously deprecated in Python 3.11.
(Contributed by Victor Stinner in 104780.)

[python_234] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): builtins

Sumber: ht

* Remove support for chained classmethod descriptors
(introduced in 63272).
These can no longer be used to wrap other descriptors,
such as property.
The core design of this feature was flawed and led to several problems.
To "pass-through" a classmethod, consider using
the __wrapped__ attribute that was added in Python 3.10.
(Contributed by Raymond Hettinger in 89519.)
* Raise a RuntimeError when calling frame.clear
on a suspended frame (as has always been the case for an executing frame).
(Contributed by Irit Katriel in 79932.)

[python_235] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): configparser

Sumber: ht

* Remove the undocumented LegacyInterpolation class,
deprecated in the docstring since Python 3.2,
and at runtime since Python 3.11.
(Contributed by Hugo van Kemenade in 104886.)

[python_236] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): importlib.metadata

Sumber: ht

* Remove deprecated subscript (~object.__getitem__) access for
entry-points objects.
(Contributed by Jason R. Coombs in 113175.)

[python_237] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): locale

Sumber: ht

* Remove the locale.resetlocale function, deprecated in Python 3.11.
Use ``locale.setlocale(locale.LC_ALL, "")`` instead.
(Contributed by Victor Stinner in 104783.)

[python_238] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): opcode

Sumber: ht

* Move opcode.ENABLE_SPECIALIZATION to _opcode.ENABLE_SPECIALIZATION.
This field was added in 3.12, it was never documented,
and is not intended for external use.
(Contributed by Irit Katriel in 105481.)
* Remove opcode.is_pseudo, opcode.MIN_PSEUDO_OPCODE,
and opcode.MAX_PSEUDO_OPCODE, which were added in Python 3.12,
but were neither documented nor exposed through dis,
and were not intended to be used externally.
(Contributed by Irit Katriel in 105481.)

[python_239] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): optparse

Sumber: ht

* This module is no longer considered soft deprecated.
While argparse remains preferred for new projects that
aren't using a third party command line argument processing
library, there are aspects of the way ``argparse`` works that
mean the lower level ``optparse`` module may provide a better
foundation for *writing* argument processing libraries, and
for implementing command line applications which adhere more
strictly than ``argparse`` does to various Unix command line
processing conventions that originate in the behaviour of the
C :cgetopt function .
(Contributed by Alyssa Coghlan and Serhiy Storchaka in 126180.)

[python_240] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): pathlib

Sumber: ht

* Remove the ability to use ~pathlib.Path objects as context managers.
This functionality was deprecated and has had no effect since Python 3.9.
(Contributed by Barney Gale in 83863.)
re
--
* Remove the undocumented, deprecated, and broken
re.template function and re.TEMPLATE / re.T flag.
(Contributed by Serhiy Storchaka and Nikita Sobolev in 105687.)

[python_241] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): tkinter.tix

Sumber: ht

* Remove the tkinter.tix module, deprecated in Python 3.6.
The third-party Tix library which the module wrapped is unmaintained.
(Contributed by Zachary Ware in 75552.)

[python_242] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): turtle

Sumber: ht

* Remove the RawTurtle.settiltangle method,
deprecated in the documentation since Python 3.1
and at runtime since Python 3.11.
(Contributed by Hugo van Kemenade in 104876.)

[python_243] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): typing

Sumber: ht

* Remove the typing.io and typing.re namespaces,
deprecated since Python 3.8.
The items in those namespaces can be imported directly
from the typing module.
(Contributed by Sebastian Rittau in 92871.)
* Remove the keyword-argument method of creating
~typing.TypedDict types, deprecated in Python 3.11.
(Contributed by Tomas Roun in 104786.)

[python_244] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): unittest

Sumber: ht

* Remove the following unittest functions, deprecated in Python 3.11:
* unittest.findTestCases
* unittest.makeSuite
* unittest.getTestCaseNames
Use ~unittest.TestLoader methods instead:
* ~unittest.TestLoader.loadTestsFromModule
* ~unittest.TestLoader.loadTestsFromTestCase
* ~unittest.TestLoader.getTestCaseNames
(Contributed by Hugo van Kemenade in 104835.)
* Remove the untested and undocumented TestProgram.usageExit
method, deprecated in Python 3.11.
(Contributed by Hugo van Kemenade in 104992.)

[python_245] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): urllib

Sumber: ht

* Remove the *cafile*, *capath*, and *cadefault* parameters of the
urllib.request.urlopen function, deprecated in Python 3.6.
Use the *context* parameter instead with an ~ssl.SSLContext instance.
The ssl.SSLContext.load_cert_chain function
can be used to load specific certificates,
or let ssl.create_default_context select
the operating system's trusted certificate authority (CA) certificates.
(Contributed by Victor Stinner in 105382.)

[python_246] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): webbrowser

Sumber: ht

* Remove the untested and undocumented MacOSX class,
deprecated in Python 3.11.
Use the MacOSXOSAScript class (introduced in Python 3.2) instead.
(Contributed by Hugo van Kemenade in 104804.)
* Remove the deprecated MacOSXOSAScript._name attribute.
Use the webbrowser.controller.name
attribute instead.
(Contributed by Nikita Sobolev in 105546.)

[python_247] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): New Deprecations

Sumber: ht

* user-defined-funcs:
* Deprecate assignment to a function's ~function.__code__ attribute,
where the new code object's type does not match the function's type.
The different types are:
plain function, generator, async generator, and coroutine.
(Contributed by Irit Katriel in 81137.)
* array:
* Deprecate the ``'u'`` format code (:cwchar_t) at runtime.
This format code has been deprecated in documentation since Python 3.3,
and will be removed in Python 3.16.
Use the ``'w'`` format code (:cPy_UCS4)
for Unicode characters instead.
(Contributed by Hugo van Kemenade in 80480.)
* ctypes:
* Deprecate the undocumented SetPointerType function,
to be removed in Python 3.15.
(Contributed by Victor Stinner in 105733.)
* soft deprecated the ~ctypes.ARRAY
function in favour of ``type * length`` multiplication.
(Contributed by Victor Stinner in 105733.)
* decimal:
* Deprecate the non-standard and undocumented ~decimal.Decimal
format specifier ``'N'``,
which is only supported in the decimal module's C implementation.
Scheduled to be removed in Python 3.18.
(Contributed by Serhiy Storchaka in 89902.)
* dis:
* Deprecate the HAVE_ARGUMENT separator.
Check membership in ~dis.hasarg instead.
(Contributed by Irit Katriel in 109319.)
* gettext:
* Deprecate non-integer numbers as arguments to functions and methods
that consider plural forms in the gettext module,
even if no translation was found.
(Contributed by Serhiy Storchaka in 88434.)

* glob:
* Deprecate the undocumented glob0 and glob1 functions.
Use ~glob.glob and pass a path-like object specifying
the root directory to the *root_dir* parameter instead.
(Contributed by Barney Gale in 117337.)
* http.server:
* Deprecate CGIHTTPRequestHandler,
to be removed in Python 3.15.
Process-based CGI HTTP servers have been out of favor for a very lo

[python_248] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): CPython Bytecode Changes

Sumber: ht

* The oparg of YIELD_VALUE is now
``1`` if the yield is part of a yield-from or await, and ``0`` otherwise.
The oparg of RESUME was changed to add a bit indicating
if the except-depth is 1, which is needed to optimize closing of generators.
(Contributed by Irit Katriel in 111354.)

[python_249] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): New Features

Sumber: ht

* Add the c-api-monitoring
for generating 669 monitoring events:
* :cPyMonitoringState
* :cPyMonitoring_FirePyStartEvent
* :cPyMonitoring_FirePyResumeEvent
* :cPyMonitoring_FirePyReturnEvent
* :cPyMonitoring_FirePyYieldEvent
* :cPyMonitoring_FireCallEvent
* :cPyMonitoring_FireLineEvent
* :cPyMonitoring_FireJumpEvent
* ``PyMonitoring_FireBranchEvent``
* :cPyMonitoring_FireCReturnEvent
* :cPyMonitoring_FirePyThrowEvent
* :cPyMonitoring_FireRaiseEvent
* :cPyMonitoring_FireCRaiseEvent
* :cPyMonitoring_FireReraiseEvent
* :cPyMonitoring_FireExceptionHandledEvent
* :cPyMonitoring_FirePyUnwindEvent
* :cPyMonitoring_FireStopIterationEvent
* :cPyMonitoring_EnterScope
* :cPyMonitoring_ExitScope
(Contributed by Irit Katriel in 111997).
* Add :cPyMutex, a lightweight mutex that occupies a single byte,
and the new :cPyMutex_Lock and :cPyMutex_Unlock functions.
:cPyMutex_Lock will release the GIL (if currently held)
if the operation needs to block.
(Contributed by Sam Gross in 108724.)

* Add the c-api-time to provide access to system clocks:
* :cPyTime_t.
* :cPyTime_MIN and :cPyTime_MAX.
* :cPyTime_AsSecondsDouble.
* :cPyTime_Monotonic.
* :cPyTime_MonotonicRaw.
* :cPyTime_PerfCounter.
* :cPyTime_PerfCounterRaw.
* :cPyTime_Time.
* :cPyTime_TimeRaw.
(Contributed by Victor Stinner and Petr Viktorin in 110850.)
* Add the :cPyDict_ContainsString function
with the same behavior as :cPyDict_Contains,
but *key* is specified as a :cconst char* UTF-8 encoded bytes string,
rather than a :cPyObject*.
(Contributed by Victor Stinner in 108314.)
* Add the :cPyDict_GetItemRef and :cPyDict_GetItemStringRef
functions,
which behave similarly to :cPyDict_GetItemWithError,
but return a strong reference instead of a borrowed reference.
Moreover, these functions return ``-1`` on error,
removing the need to

[python_250] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Changed C APIs

Sumber: ht

* The *keywords* parameter of :cPyArg_ParseTupleAndKeywords
and :cPyArg_VaParseTupleAndKeywords
now has type :cchar * const * in C
and :cconst char * const * in C++,
instead of :cchar **.
In C++, this makes these functions compatible with arguments
of type :cconst char * const *, :cconst char **,
or :cchar * const * without an explicit type cast.
In C, the functions only support arguments of type :cchar * const *.
This can be overridden with the :cPY_CXX_CONST macro.
(Contributed by Serhiy Storchaka in 65210.)
* :cPyArg_ParseTupleAndKeywords now supports
non-ASCII keyword parameter names.
(Contributed by Serhiy Storchaka in 110815.)
* The :cPyCode_GetFirstFree function is now unstable API
and is now named :cPyUnstable_Code_GetFirstFree.
(Contributed by Bogdan Romanyuk in 115781.)
* The :cPyDict_GetItem, :cPyDict_GetItemString,
:cPyMapping_HasKey, :cPyMapping_HasKeyString,
:cPyObject_HasAttr, :cPyObject_HasAttrString,
and :cPySys_GetObject functions,
each of which clears all errors which occurred when calling them
now reports these errors using sys.unraisablehook.
You may replace them with other functions as recommended in the documentation.

(Contributed by Serhiy Storchaka in 106672.)
* Add support for the ``%T``, ``%#T``, ``%N`` and ``%#N`` formats
to :cPyUnicode_FromFormat:
* ``%T``: Get the fully qualified name of an object type
* ``%#T``: As above, but use a colon as the separator
* ``%N``: Get the fully qualified name of a type
* ``%#N``: As above, but use a colon as the separator
See 737 for more information.
(Contributed by Victor Stinner in 111696.)
* You no longer have to define the ``PY_SSIZE_T_CLEAN`` macro before
including Python.h when using ``#`` formats in
arg-parsing-string-and-buffers.
APIs accepting the format codes always use ``Py_ssize_t`` for ``#``

[python_251] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Limited C API Changes

Sumber: ht

* The following functions are now included in the Limited C API:
* :cPyMem_RawMalloc
* :cPyMem_RawCalloc
* :cPyMem_RawRealloc
* :cPyMem_RawFree
* :cPySys_Audit
* :cPySys_AuditTuple
* :cPyType_GetModuleByDef
(Contributed by Victor Stinner in 85283 and 116936.)
* Python built with --with-trace-refs (tracing references)
now supports the limited-c-api.
(Contributed by Victor Stinner in 108634.)

[python_252] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Removed C APIs

Sumber: ht

* Remove several functions, macros, variables, etc
with names prefixed by ``_Py`` or ``_PY`` (which are considered private).
If your project is affected by one of these removals
and you believe that the removed API should remain available,
please using-the-tracker to request a public C API
and add ``cc: @vstinner`` to the issue to notify Victor Stinner.
(Contributed by Victor Stinner in 106320.)
* Remove old buffer protocols deprecated in Python 3.0.
Use bufferobjects instead.
* :cPyObject_CheckReadBuffer:
Use :cPyObject_CheckBuffer to test
whether the object supports the buffer protocol.
Note that :cPyObject_CheckBuffer doesn't guarantee
that :cPyObject_GetBuffer will succeed.
To test if the object is actually readable,

see the next example of :cPyObject_GetBuffer.
* :cPyObject_AsCharBuffer, :cPyObject_AsReadBuffer:
Use :cPyObject_GetBuffer and :cPyBuffer_Release instead:
.. code-block:: c
Py_buffer view;
if (PyObject_GetBuffer(obj, &view, PyBUF_SIMPLE) < 0) {
return NULL;
}
// Use `view.buf` and `view.len` to read from the buffer.
// You may need to cast buf as `(const char*)view.buf`.
PyBuffer_Release(&view);
* :cPyObject_AsWriteBuffer:
Use :cPyObject_GetBuffer and :cPyBuffer_Release instead:
.. code-block:: c
Py_buffer view;
if (PyObject_GetBuffer(obj, &view, PyBUF_WRITABLE) < 0) {
return NULL;
}
// Use `view.buf` and `view.len` to write to the buffer.
PyBuffer_Release(&view);
(Contributed by Inada Naoki in 85275.)
* Remove various functions deprecated in Python 3.9:
* :cPyEval_CallObject, :cPyEval_CallObjectWithKeywords:
Use :cPyObject_CallNoArgs or :cPyObject_Call instead.
.. warning
* :cPyEval_CallMethod:
Use :cPyObject_CallMethod instead.
* :cPyCFunction_Call:
Use :cPyObject_Call instead.
(Contributed by Victor Stinner in 105107.)
* Remove the following old fun

[python_253] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Deprecated C APIs

Sumber: ht

* Deprecate old Python initialization functions:
* :cPySys_ResetWarnOptions:
Clear sys.warnoptions and warnings.filters instead.
* :cPy_GetExecPrefix:
Get sys.exec_prefix instead.
* :cPy_GetPath:
Get sys.path instead.
* :cPy_GetPrefix:
Get sys.prefix instead.
* :cPy_GetProgramFullPath:
Get sys.executable instead.
* :cPy_GetProgramName:
Get sys.executable instead.
* :cPy_GetPythonHome:
Get :cPyConfig.home

or the PYTHONHOME environment variable instead.
(Contributed by Victor Stinner in 105145.)
* soft deprecated the
:cPyEval_GetBuiltins, :cPyEval_GetGlobals,
and :cPyEval_GetLocals functions,
which return a borrowed reference.
(Soft deprecated as part of 667.)
* Deprecate the :cPyImport_ImportModuleNoBlock function,
which is just an alias to :cPyImport_ImportModule since Python 3.3.
(Contributed by Victor Stinner in 105396.)
* soft deprecated the
:cPyModule_AddObject function.
It should be replaced with :cPyModule_Add
or :cPyModule_AddObjectRef.
(Contributed by Serhiy Storchaka in 86493.)
* Deprecate the old ``Py_UNICODE`` and ``PY_UNICODE_TYPE`` types
and the :cPy_UNICODE_WIDE define.
Use the :cwchar_t type directly instead.
Since Python 3.3, ``Py_UNICODE`` and ``PY_UNICODE_TYPE``
are just aliases to :cwchar_t.
(Contributed by Victor Stinner in 105156.)
* Deprecate the :cPyWeakref_GetObject and
:cPyWeakref_GET_OBJECT functions,
which return a borrowed reference.
Replace them with the new :cPyWeakref_GetRef function,
which returns a strong reference.
The `pythoncapi-compat project`_ can be used to get
:cPyWeakref_GetRef on Python 3.12 and older.
(Contributed by Victor Stinner in 105927.)
.. Add deprecations above alphabetically, not here at the end.
.. _pythoncapi-compat project: https://github.com/python/pythoncapi-compat/

[python_254] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Build Changes

Sumber: ht

* ``arm64-apple-ios`` and ``arm64-apple-ios-simulator`` are both
now 11 tier 3 platforms.
(whatsnew313-platform-support written
and implementation contributed by Russell Keith-Magee in 114099.)
* ``aarch64-linux-android`` and ``x86_64-linux-android`` are both
now 11 tier 3 platforms.
(whatsnew313-platform-support written
and implementation contributed by Malcolm Smith in 116622.)
* ``wasm32-wasi`` is now a 11 tier 2 platform.
(Contributed by Brett Cannon in 115192.)
* ``wasm32-emscripten`` is no longer a 11 supported platform.
(Contributed by Brett Cannon in 115192.)
* Building CPython now requires a compiler with support for the C11 atomic
library, GCC built-in atomic functions, or MSVC interlocked intrinsics.
* Autoconf 2.71 and aclocal 1.16.5 are now required to regenerate

the configure script.
(Contributed by Christian Heimes in 89886 and by Victor Stinner in 112090.)
* SQLite 3.15.2 or newer is required to build
the sqlite3 extension module.
(Contributed by Erlend Aasland in 105875.)
* CPython now bundles the `mimalloc library`_ by default.
It is licensed under the MIT license;
see mimalloc-license.
The bundled mimalloc has custom changes, see 113141 for details.
(Contributed by Dino Viehland in 109914.)
.. _mimalloc library: https://github.com/microsoft/mimalloc/
* The configure option --with-system-libmpdec
now defaults to ``yes``.
The bundled copy of ``libmpdec`` will be removed in Python 3.16.
* Python built with configure --with-trace-refs
(tracing references) is now ABI compatible with the Python release build
and debug-build.
(Contributed by Victor Stinner in 108634.)
* On POSIX systems, the pkg-config (``.pc``) filenames now include the ABI
flags. For example, the free-threaded build generates ``python-3.13t.pc``
and the debug build generates ``python-3.13

[python_255] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Porting to Python 3.13

Sumber: ht

This section lists previously described changes and other bugfixes
that may require changes to your code.

[python_256] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Changes in the Python API

Sumber: ht

* whatsnew313-locals-semantics introduces several changes
to the semantics of locals and frame.f_locals:
* Calling locals in an optimized scope now produces an
independent snapshot on each call, and hence no longer implicitly updates
previously returned references. Obtaining the legacy CPython behavior now
requires explicit calls to update the initially returned dictionary with the
results of subsequent calls to locals. Code execution functions that
implicitly target locals (such as ``exec`` and ``eval``) must be
passed an explicit namespace to access their results in an optimized scope.
(Changed as part of 667.)
* Calling locals from a comprehension at module or class scope
(including via ``exec`` or ``eval``) once more behaves as if the comprehension
were running as an independent nested function (i.e. the local variables from
the containing scope are not included). In Python 3.12, this had changed
to include the local variables from the containing scope when implementing
709. (Changed as part of 667.)
* Accessing frame.f_locals in an

optimized scope now returns a write-through proxy rather than a
snapshot that gets updated at ill-specified times. If a snapshot is desired,
it must be created explicitly with ``dict`` or the proxy's ``.copy()`` method.
(Changed as part of 667.)
* functools.partial now emits a FutureWarning
when used as a method.
The behavior will change in future Python versions.
Wrap it in staticmethod if you want to preserve the old behavior.
(Contributed by Serhiy Storchaka in 121027.)
* An OSError is now raised by getpass.getuser
for any failure to retrieve a username,
instead of ImportError on non-Unix platforms
or KeyError on Unix platforms where the password database is empty.
* The value of the mode attribute of gzip.GzipFile
is now a string (``

[python_257] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Changes in the C API

Sumber: ht

* ``Python.h`` no longer includes the ``<ieeefp.h>`` standard header. It was
included for the :cfinite function which is now provided by the
``<math.h>`` header. It should now be included explicitly if needed. Remove
also the ``HAVE_IEEEFP_H`` macro.
(Contributed by Victor Stinner in 108765.)
* ``Python.h`` no longer includes these standard header files: ``<time.h>``,
``<sys/select.h>`` and ``<sys/time.h>``. If needed, they should now be
included explicitly. For example, ``<time.h>`` provides the :cclock and
:cgmtime functions, ``<sys/select.h>`` provides the :cselect
function, and ``<sys/time.h>`` provides the :cfutimes, :cgettimeofday
and :csetitimer functions.
(Contributed by Victor Stinner in 108765.)
* On Windows, ``Python.h`` no longer includes the ``<stddef.h>`` standard
header file. If needed, it should now be included explicitly. For example, it
provides :coffsetof function, and ``size_t`` and ``ptrdiff_t`` types.
Including ``<stddef.h>`` explicitly was already needed by all other
platforms, the ``HAVE_STDDEF_H`` macro is only defined on Windows.
(Contributed by Victor Stinner in 108765.)
* If the :cPy_LIMITED_API macro is defined, :cPy_BUILD_CORE,
:cPy_BUILD_CORE_BUILTIN and :cPy_BUILD_CORE_MODULE macros
are now undefined by ``<Python.h>``.
(Contributed by Victor Stinner in 85283.)
* The old trashcan macros ``Py_TRASHCAN_SAFE_BEGIN`` and ``Py_TRASHCAN_SAFE_END``
were removed. They should be replaced by the new macros ``Py_TRASHCAN_BEGIN``
and ``Py_TRASHCAN_END``.
A ``tp_dealloc`` function that has the old macros, such as
should be the deallocation function it is in. The new macros were
added in Python 3.8 and the old macros were deprecated in Python 3.11.
(Contributed by Irit Katriel in 105111.)
* whatsnew313-locals-semantics introduces several changes
to fram

[python_258] Python - Modern Features - Python 3.13

What's New in Python 3.13 (Free-threaded GIL-free, JIT, Enhanced REPL): Regression Test Changes

Sumber: ht

* Python built with configure --with-pydebug now
supports a -X command-line
option. If used, it specifies a module that should be imported early
in the lifecycle of the interpreter, before ``site.py`` is executed.
(Contributed by ?ukasz Langa in 110769.)